

Learn Angular by Building Compass

*A project-driven path from your first line of
TypeScript to a deployed Angular app*

First edition, 2026

Prepared for personal use.

Preface

If you have never built a web application before, this book is written for you. It assumes you can read English, use a computer, and are willing to type things into an editor and see what happens. It does not assume you know what a framework is, what TypeScript is, or why anyone would voluntarily use something called *dependency injection*. By the last chapter, all of those words will feel like tools you reach for, not jargon you tolerate.

The book is built around a single application called **Compass** — a personal task and habit tracker. In Chapter 4 you will type `ng new compass` and see an empty starter appear on your screen. Every chapter after that adds one real capability to Compass. By the end you will have written a real app: it will let you sign in, plan your week, track streaks, work offline, sync when you reconnect, and run in a browser on the public internet.

Learning by building has two advantages that reading-first books tend to give up. The first is that every concept lands somewhere. When you meet a new idea — say, an *interceptor* — you do not just read a definition; you write one, watch it change the behavior of your app, and then break it on purpose to see what happens. The second is that you end with something. Compass is small enough to hold in

your head and rich enough to show a friend. It is also yours: nothing in this book is a puzzle whose answer is hidden at the back.

How the book is organized

The book has five parts and a set of appendices.

Part I — Foundations. Four chapters that get you from zero to a running Angular project. You will meet TypeScript, the modern JavaScript features Angular actually uses, and the Angular CLI. You will not touch a component here; the goal is to have a workshop before you pick up a hammer.

Part II — Components and templates. The core mechanics: what a component is, how templates and data binding work, and how *signals* — Angular's modern reactive primitive — let you describe changing state without ceremony. By the end of Part II, Compass will look and feel like a real app.

Part III — Services and the outside world. Applications don't live in a single component. This part covers dependency injection, HTTP, RxJS, and forms — the four things you need to talk to servers, gather user input, and share behavior across your app without copying and pasting.

Part IV — Building a real app. Routing, state that outlives a single screen, custom directives and pipes, and performance. This is where Compass grows from "one page that works" into "several pages that stay fast."

Part V — Production. Testing, server-side rendering, deployment, and the ecosystem you are joining. When you finish Part V you

will have a Compass instance on the internet, and you will know what to search for next.

Appendices. A machine setup guide, a debugging checklist, and a short tour of older Angular idioms you will encounter in existing codebases. Read them when you need them; they are reference material, not sequential.

How to read this book

Read it at the keyboard. Every chapter has code you should type — not paste. The typing itself does work: it slows you down enough to see what changed and forces you to notice syntax that a copy would gloss over. When a chapter ends with an *Exercises* section, do them before starting the next chapter. They are not optional in the sense that the next chapter assumes you did them; they are optional in the sense that if you skip them, you will lose the muscle memory the book is trying to build.

Callouts appear throughout:

Note. A useful aside — deeper context, a related concept, or a piece of history.

Watch out. A common mistake or a subtle behavior that will save you an afternoon later.

Try it. A one-minute experiment to change some code and see the result. Do these; they are cheap.

What you need

- A laptop or desktop computer running macOS, Windows, or Linux.
- About 5 GB of free disk space (mostly for `node_modules`, which we will meet in Chapter 4).
- A stable internet connection while working through Parts I and V; the rest of the book works offline.
- Roughly 30 to 50 hours across however many sittings suit you. Some readers finish in two weekends; some spread it over three months. The book does not care.

You do not need any prior experience with Angular, React, Vue, or any other framework. You do not need to know TypeScript; Chapter 2 teaches enough of it. You do not need to know how servers work; when we need a server in Part III, we spin one up together in a few lines of code.

A word on Angular versions

Angular ships a new major version roughly every six months, but the *ideas* in this book — components, signals, dependency injection, routing, forms — have been stable and central for years and will remain so. The specific syntax targets Angular 18 and later, which is the first version where the modern control flow (`@if`, `@for`, `@switch`), signals, and standalone components are the recommended defaults. If you are reading this and Angular is on version 22 by then, most of the code will still run untouched; anything that changes will be a rename, not a rethink.

Turn the page. Chapter 1 is a short one — no code, just a map of where we are and where we are going.

Chapter 1. What Angular is, and when to reach for it

This chapter has no code. It is a map. Before you install anything, it helps to know what you are installing, what problem it solves, and what kind of application you are about to build. Read it once, slowly. Everything from Chapter 2 onward assumes you have this map in your head.

The shape of a web app

Imagine, for a minute, a website you use often. Maybe it is a calendar, a to-do list, a music streaming service, or an email inbox. Now imagine what it does when you interact with it.

You click a message and its full text appears in a pane on the right. You type a search query and results filter as you type, without the page reloading. You mark a task done and it moves into a "completed" list; another user, on another continent, sees the change a second later. You drag an event across your calendar and it snaps to a new time.

None of this is a page. In the earliest years of the web, every one of those interactions would have meant a round trip to a server: click, wait, watch the page go white, wait, see a new page. A modern web application does something different. It loads once, then it maintains a *living picture* of what the user is looking at, updating parts of the

screen as data arrives and as the user acts. When it needs data or wants to save something, it talks to a server quietly in the background, without disrupting the picture.

Keeping that living picture up to date is harder than it sounds. Every piece of the screen depends on some piece of data — the count of unread messages in the sidebar depends on the message list, the "reply" button's enabled state depends on whether a message is selected, the color of a task depends on whether it is overdue. When any piece of data changes, every part of the screen that depends on it has to change with it. If you tried to write that by hand — every button that has to enable and disable, every counter that has to increment, every list that has to re-render — you would spend most of your time gluing things together rather than building anything new. You would also introduce bugs constantly: forgetting to update a counter, forgetting to disable a button, forgetting to remove an old item from a list.

A **framework** is a piece of software that handles most of that gluing for you. Angular is a framework.

What "framework" actually means

If you have written a computer program before — even a small one — you have written *your* code and, at some point, imported someone else's code to do a job you didn't want to do yourself. That someone else's code is called a **library**. Your code is in charge; when it needs help, it *calls* the library.

A framework is the opposite. In a framework, the framework's code is in charge; your code fills in the parts that the framework does

not know. A useful metaphor: a library is a hammer, and you decide when to swing it. A framework is a workshop, and you decide what to build inside the workshop — but the workshop has already picked the layout of the workbench, the location of the outlets, and the tool caddy on the wall. You gain a lot: the workshop is well-organized, standard tools are always where you expect them, and other people who have worked in the same kind of workshop can find their way around your code immediately. You give up something too: if you want to tear out the workbench and put the tool caddy on the ceiling, the workshop will fight you.

This trade-off is the central choice you make when you pick a framework. In return for giving up some flexibility, you get consistency, community, and a lot of decisions you no longer have to make yourself. Angular sits toward the "opinionated" end of the framework spectrum. It has opinions about how you organize code, how components talk to each other, how you handle side effects, how you write tests, and how you build for production. Some newcomers find this stifling for the first few days and liberating for the rest of their careers.

So — what is Angular?

Angular is a framework, made by Google, for building applications that run in a web browser. In one sentence: **you describe the screens of your application as a tree of small, self-contained pieces called *components*, and Angular takes care of turning that description into pixels, keeping the pixels in sync with your data, and shipping the whole thing to a browser.**

Let's unpack that sentence a piece at a time, because every noun in it will get a chapter of its own later.

A tree of components. You will not build Compass as one enormous file. You will build it as a tree of small parts: a top-level shell, inside that a sidebar and a main area, inside the main area a header and a list, inside the list many task rows, and so on. Each part — each component — is responsible for one thing: how it looks, how it behaves when you interact with it, and how it exposes itself to the parts around it. Components in the tree talk to their parents and children through well-defined, one-directional connections. Angular calls those connections *inputs* and *outputs*; you will meet them properly in Chapter 8.

Self-contained. A component's HTML, CSS, and behavior live together. This is a deliberate reversal of the older web development model, where a project had a folder of HTML files, a folder of CSS files, and a folder of JavaScript files, and any single feature touched one file in each. Angular's argument, born from a decade of watching people build large applications, is that features rarely respect those boundaries — a button's markup, its style, and its click handler are one feature — and it is more useful to keep them together and separate them along feature lines instead.

Angular turns your description into pixels. You describe what the screen should look like at any given moment, given some data. Angular figures out what to actually put on the screen, and what to change when the data changes. This is subtle. In the very old web, you wrote code that said "when the user clicks reply, remove the reply button, add a *send* button, put a textarea in the middle, focus it." In Angular, you write: "the reply button is visible when there is no

draft; the send button is visible when there is a draft; the textarea's content is bound to the draft's text." You describe *the states*, not *the transitions*. Angular does the transitions.

Angular keeps the pixels in sync with your data. Because your description says which pieces of the screen depend on which pieces of data, Angular knows how to update them when data changes. Modern Angular does this with a mechanism called *signals*, which you will meet in Chapter 7. A signal is a piece of data that knows who is looking at it; when the data changes, everyone looking at it is notified, and just those pieces of the screen re-render. This sounds like a nicety and turns out to be transformative: it means your application stays fast even when it grows large, because the framework never has to re-check the entire screen after every change.

Angular ships the whole thing to a browser. Modern web apps go through a *build* step that turns your source code (many files in a friendly authoring format) into a small number of files, heavily optimized, ready to be sent to a browser. Angular ships with a build tool that handles this for you. In Chapter 4 you will run `ng build` for the first time and see megabytes of your source code compressed into a few hundred kilobytes of production-ready output.

What makes Angular Angular

Angular is not the only framework of its kind. If you have heard the name Angular, you have probably heard the names of a few others. This book will not compare them in depth — you do not need that comparison to learn Angular any more than you need to compare

Spanish to Italian to learn Spanish — but it is fair to name the qualities that distinguish Angular from its peers.

Opinionated and batteries-included. Angular arrives with routing, forms, HTTP, animation, testing, internationalization, and server-side rendering already in the box, all designed to work with one another. Frameworks that take the opposite approach — ship a small core and let you pick libraries for everything else — give you more flexibility and force more decisions on you. Angular's bet is that most of those decisions have right answers most of the time, and it makes them for you so that you can spend your attention on your application instead.

TypeScript-first. Angular is written in TypeScript, and every Angular application is written in TypeScript. TypeScript is a superset of JavaScript: it is JavaScript with types. If that sounds intimidating, don't worry — Chapter 2 teaches enough TypeScript to be productive, and Angular's tooling means the types mostly help you rather than get in your way. The payoff is real: when you rename a field, your editor renames every use of it across your whole app; when you break a component's contract with its parent, your editor tells you before you refresh the browser. In a large application, this is the difference between a codebase you can keep in your head and a codebase you fight.

Dependency injection. Angular has a built-in system for one component to say, "I need a thing that does X," and get a suitable thing without having to know how it was made. This sounds abstract now; it will feel obvious after Chapter 9. It is the single biggest reason Angular applications tend to be testable, because "give me a fake

version of the thing that talks to the server" turns out to be one line of code.

Signals as the reactive primitive. For most of its history, Angular's reactive model was built on a library called RxJS, which represents changes over time as *observable streams*. RxJS is powerful and still important — you will meet it in Chapter 11 — but the learning cliff was steep and the ceremony was heavy for everyday state. Recent Angular versions introduced *signals*, a lighter-weight primitive that feels like plain variables but auto-updates the screen when they change. Signals are how you will manage most state in this book; RxJS is how you will handle streams that inherently unfold over time, like HTTP responses and event streams.

A convention-driven CLI. Angular ships a command-line tool, `ng`, that scaffolds projects, generates new components and services, runs your app, runs your tests, and builds for production. This is the tool that removes most of the setup ceremony that used to gate web development. In Chapter 4 you will use it to create Compass with a single command.

When Angular is a good fit — and when it isn't

Angular is not the right choice for every project on the web, and part of learning the framework is knowing where it shines.

Angular is at its best when the application is one you expect to work on for a long time, with a team, with real business complexity: dashboards, admin panels, internal tools, customer portals, banking

and healthcare and e-commerce apps that will grow features every quarter for years. In those settings, Angular's opinionated structure, TypeScript foundation, and testing story pay for themselves many times over. New engineers can join and find their way around; refactors are safe because the compiler catches contract breakages; features can be added without accidentally breaking three other features.

Angular is a heavier tool than you need for a single marketing page, a landing site, a blog, or a one-off interactive demo. For those, plain HTML — or a lightweight static site generator — will start you faster, ship less code to your users, and be easier to hand off. Choosing Angular for a five-page marketing site is like renting a warehouse to make a sandwich. You can do it; it will work; you should not.

Angular is not the best choice for game engines, heavy visualizations, or applications whose primary surface is a canvas or WebGL scene. Angular does not know or care what a pixel of your canvas looks like; you would use it only for the UI *around* the canvas, and even then a lighter framework or plain code might be more appropriate.

Compass, the app you are about to build, sits squarely in Angular's sweet spot: several screens, real data, a persistent server, forms, authentication, and a feature list that will grow across chapters. It is representative of the kind of application Angular teams build every day.

What Compass will be, at the end

By the last chapter, Compass will be a real, if small, application. Here is a preview so you know where the road leads.

You will open Compass in your browser and see a login screen. You will sign in with an email address; a confirmation link will arrive; you will click it, and Compass will remember you on this device. You will land on a dashboard showing today's tasks and the habits you are tracking this week.

You will add a task by pressing a button, filling out a form, and hitting save. It will appear immediately. If your internet is off, it will still appear immediately, marked with a small icon that says "not yet synced," and when your internet comes back it will save quietly in the background. You will drag tasks between "today," "this week," and "later." You will mark habits done with a click; a streak counter next to each habit will update.

You will navigate to a *stats* page and see a chart of your habit completions across the last thirty days. You will click a habit to see its history. You will visit a settings page where you can change your display name and pick a theme.

Every one of those features is a place in the book. The login screen and email confirmation live in the routing and forms chapters. The offline-first behavior lives in the HTTP and state chapters. The chart lives in the components-and-templates chapter, and again in the performance chapter when we make it fast enough not to matter on a phone. The tests you will write for all of it live in Part V.

You will finish the book with an application that is small enough to hold in your head and rich enough to justify every idea Angular offers.

What comes next

The next three chapters get you ready to type `ng new compass`. They are unusual for a "build an app" book because they contain almost no Angular. That is deliberate. Angular sits on top of TypeScript, TypeScript sits on top of modern JavaScript, and both sit on top of a set of tools — a package manager, a runtime, an editor. Skipping past those, as many books do, produces readers who can copy Angular examples but who freeze the first time an error message mentions a term the book never defined.

Chapter 2 covers TypeScript. Not all of it — TypeScript is a large language with an even larger set of possible types — just the parts you will actually meet while writing Angular. Chapter 3 covers the modern JavaScript features Angular relies on: destructuring, spread, arrow functions, modules, promises and `async / await`. Chapter 4 walks you through installing Node and the Angular CLI, running `ng new compass`, and taking a slow tour of every file the CLI produces so that nothing in your new project looks mysterious.

By the end of Part I, you will have a running Angular application on your screen and you will understand every part of what generated it. Then, in Part II, we will start building.

Exercises

1. Open a web application you use often — an email client, a music service, a project management tool, anything with a rich interface. Watch what happens as you click around. Try to identify three interactions that would have required

a full page reload in an old-school website and instead update just a piece of the screen. Write them down.

2. In your own words, in three or four sentences, describe the difference between a library and a framework to someone who has never written code. Do not use the words *library* or *framework* in your answer. If you can explain the trade-off without them, you understand it.
3. Think of an application idea of your own that you might like to build someday. It does not have to be original or good; a personal recipe box, a music practice log, or a birthday reminder is fine. Ask yourself: is this application in Angular's sweet spot, or would something lighter serve it better? Write down your answer and your reasoning. In a few months, come back and see if you still agree.

Chapter 2. TypeScript enough for Angular

This chapter is a working knowledge of TypeScript, written for people who have not used it before and do not want to become full-time TypeScript scholars. By the end you will be able to read every TypeScript expression the rest of the book uses, and write your own without guessing. You will not, and should not, know everything TypeScript can do. Its type system is deep enough to have its own bookshelf; we are not visiting that bookshelf today.

The examples in this chapter are motivated by Compass. Every time we introduce a new TypeScript feature, we will show it with a shape you will later have in your app: a task, a habit, a user, a completion event. That way, by the time we start writing Angular components in Chapter 5, the types will already feel familiar.

Why types at all

If you have written a small script in a language like Python or plain JavaScript, you probably wrote something like this:

```
function totalMinutes(sessions) {  
  let sum = 0;  
  for (const s of sessions) sum += s.duration;  
  return sum;  
}
```

This works. It also gives you no help. Somewhere else in the program, a colleague passes you an array of *strings* by mistake, or an object that spells the field `durationMinutes` instead of `duration`. The function still runs; it produces `NaN` or `"0[object Object]"` or something else surreal, and by the time you notice, you are hunting through the output for the source of the weirdness.

TypeScript adds a small amount of ceremony to prevent that hunt. You annotate what a function expects; the TypeScript compiler checks every place that function is called; if a caller violates the contract, you find out immediately, before the code runs. In an editor, this manifests as a red squiggle under the offending line and a specific message about what is wrong. In a build, it stops the build.

The same function in TypeScript:

```
interface Session {  
  duration: number;  
}  
  
function totalMinutes(sessions: Session[]): number {  
  let sum = 0;  
  for (const s of sessions) sum += s.duration;  
  return sum;  
}
```

Now the compiler knows: the function takes an array of things that each have a `duration` field which is a number, and returns a number. Pass it anything else and TypeScript objects. There is nothing magic about this — the compiler is just checking that names and shapes agree — but the payoff in a codebase larger than one file is substantial. Angular is built to lean on that payoff.

Running TypeScript, in one paragraph

TypeScript is not something browsers run directly. A tool called the TypeScript compiler (`tsc`) reads `.ts` files, checks the types, and produces plain JavaScript files that a browser (or Node.js) can run. In Angular projects, you almost never invoke `tsc` yourself; the Angular CLI runs it as part of `ng serve` and `ng build`. For this chapter, if you want to try TypeScript in isolation, use the online **TypeScript Playground** at `typescriptlang.org/play` — it lets you type TypeScript on the left and see the compiled JavaScript on the right, no installation required. From Chapter 4 onward, the CLI takes over.

The primitives

TypeScript's basic types are `string`, `number`, `boolean`, `null`, and `undefined`. There are others, but these are the ones you will type a hundred times a day.

```
let title: string = "Buy milk";
let priority: number = 2;
let done: boolean = false;
let due: string | null = null; // more on the "|" in a moment
```

TypeScript is smart about inferring types when you assign a value at the point of declaration:

```
let title = "Buy milk"; // inferred as string
```

You do not need to write `: string` when the value already tells TypeScript

what the variable holds. In practice, you annotate types on *function parameters and return types*, on *object shapes*, and on things you're going to assign to later; you leave everything else to inference. A codebase that annotates every trivial `let` is a codebase full of noise.

Note. `null` and `undefined` are distinct in TypeScript, as they are in JavaScript. Angular tends to prefer `null` for "we know there's no value here" and reserves `undefined` for "the value was never set." A field that is *optional* is one that might be `undefined`; a field that is *nullable* is one that is present but might be `null`. It is a subtle distinction that will matter in Chapter 12 when we talk about forms.

Arrays and tuples

An array's type is `Type[]`, or equivalently `Array<Type>`:

```
const priorities: number[] = [1, 2, 3];
const tags: string[] = ["home", "work"];
```

Empty arrays need an annotation because there is nothing to infer from:

```
const completed: string[] = []; // fine
const uh = []; // inferred as any[], which you
```

Tuples are fixed-length, position-typed arrays. You will use them rarely in application code:

```
const point: [number, number] = [42, 7];
```

Object types with `interface`

The bread and butter of TypeScript in an Angular application is describing the shape of objects. TypeScript offers two ways to do this — `interface` and `type` — and for object shapes they are almost interchangeable. Use `interface` for object shapes; use `type` for unions, intersections, and things that are not just an object.

Here is Compass's central type, as it will appear in Chapter 5:

```
interface Task {  
  id: string;  
  title: string;  
  done: boolean;  
  createdAt: string;    // ISO 8601 date  
  dueDate: string | null;  
  tags: string[];  
}
```

A value that claims to be a `Task` must have all six of these fields, with these types. Extra fields are an error (in most contexts). Missing fields are an error. Wrong types are an error. This shape becomes a contract that every part of the app can rely on.

Optional properties, readonly, and index signatures

You will meet three annotations that modify how a field behaves.

```
interface Habit {
  id: string;
  name: string;
  color?: string; // may be undefined
  readonly createdAt: string; // cannot be reassigned after creation
  metadata: { [key: string]: string }; // arbitrary string-string map
}
```

`color?: string` means the field is optional; a value satisfying `Habit` need not include it, and if it is there it may be `string` or `undefined`. `readonly` prevents reassignment through this type — helpful for id-like fields that must never be mutated in place. The last form, `{ [key: string]: string }`, is called an *index signature*; it says "any string key maps to a string value" and is useful for open-ended dictionaries.

Union types and literal types

A *union* type is a value that could be one of several types. You saw one already: `string | null`. Unions are constrained "or"s: this value is *either* a string *or* the value `null`.

Unions of literal strings are one of TypeScript's most useful patterns:

```
type Status = "todo" | "doing" | "done" | "archived";

interface Task {
  id: string;
  title: string;
  status: Status;
}
```

Now `status` cannot be an arbitrary string; it must be exactly one of those four. Try to write `task.status = "in-progress"` and TypeScript will refuse. This is how you enforce a closed set of options — a poor person's enum, and in most cases the better choice.

Watch out. TypeScript does have an `enum` keyword, and Angular's early years used it heavily. Modern practice is to prefer union types of string literals. `enum` compiles into runtime JavaScript objects, which bloat your bundle; union types compile into nothing (they exist only at type-check time). Chapter 5 uses unions throughout.

Type narrowing

If a variable has a union type, TypeScript will *narrow* it inside a branch of code where it can tell what the value actually is:


```
function greeting(name: string | null): string {
  if (name === null) {
    return "Hello, stranger.";
  }
  // Here, TypeScript knows name is string, not string | null.
  return `Hello, ${name}.`;
}
```

The three most common narrowing patterns:

- `typeof x === "string"` narrows `x` to `string`.
- `x === null` or `x === undefined` narrows through `if`s.
- `"field" in x` narrows to the subset of a union that has that field.
- `x instanceof Foo` narrows to `Foo`.

Narrowing is one of the features that makes TypeScript feel like it is *helping* rather than *fighting*. If TypeScript ever says "possibly undefined" and you know it isn't, the answer is almost always to check with an `if`, not to bypass the check.

Functions

Function types annotate parameters and, optionally, return types:

```
function isOverdue(task: Task, now: Date): boolean {
  if (task.dueDate === null) return false;
  return new Date(task.dueDate) < now;
}
```

Arrow functions work the same way:

```
const isOverdue = (task: Task, now: Date): boolean => {
  if (task.dueDate === null) return false;
  return new Date(task.dueDate) < now;
};
```

If TypeScript can infer the return type from the body, you can omit the annotation — but it is good practice to annotate return types on exported functions, because it forces you to think about what a function actually promises to return.

Function parameters can be optional or have defaults:

```
function label(task: Task, prefix?: string): string {
  return `${prefix ?? ""}${task.title}`;
}

function paginate<T>(items: T[], page: number, size = 20): T[] {
  return items.slice(page * size, (page + 1) * size);
}
```

The second example uses a *generic*, which we look at next.

Generics, gently

A generic is a type parameter — a placeholder for "some type we will fill in at the call site." The classic use is a function that works over any array without losing type information:

```
function first<T>(items: T[]): T | undefined {
  return items[0];
}

first([1, 2, 3]);           // T is number; returns number | und
first(["a", "b"]);         // T is string; returns string | und
```

The name `T` is convention. Use single letters for very general parameters (`T`, `U`, `K`, `V`), or descriptive names when clarity helps (`<Item>`, `<Payload>`).

You will see generics constantly in Angular. `signal<Task[]>([])` creates a signal holding an array of `Task`. `HttpClient.get<Task[]>('/api/tasks')` tells the compiler what shape to expect back from the server. In each case, the generic is the answer to "what type is inside the thing?"

Do not write your own generic functions when you are learning. Reach for them the third or fourth time you copy-paste a function and only change its types. Until then, plain typed functions are almost always the right tool.

Type inference is your friend

TypeScript is aggressive about inferring types, and you should let it. Consider:

```
const tasks = tasksFromServer();           // inferred as Task[]
const titles = tasks.map(t => t.title);     // inferred as string[]
const activeCount = tasks.filter(t => !t.done).length; // inferred as number
```

No annotations were needed — TypeScript walked through the whole chain and figured it out. The general rule: annotate the *inputs* and *outputs* of your public functions, and let TypeScript infer everything else.

any, **unknown**, and the honest escape hatches

Sometimes you truly do not know the type of a value — you have just parsed some JSON off the wire, or you are calling into an old library with no types. TypeScript has two escape hatches:

- **any** turns off type checking for that value. Anything goes. This is dangerous; if you write **any** you are signing away every benefit of using TypeScript for that piece of data.
- **unknown** is the safe alternative. A value of type **unknown** can be anything, but you cannot do anything *with* it until you narrow it to a specific type first.

```
function parseTask(raw: unknown): Task {
  if (typeof raw !== "object" || raw === null) {
    throw new Error("Not an object");
  }
  const r = raw as { id?: unknown; title?: unknown; /* ... */ };
  if (typeof r.id !== "string" || typeof r.title !== "string") {
    throw new Error("Missing required fields");
  }
  // ...more checks...
  return { id: r.id, title: r.title, done: false, createdAt: "", ... };
}
```

Reach for `unknown` at the boundary of your app — parsing JSON, reading `localStorage`, dealing with untyped libraries. Reach for `any` almost never.

Decorators, briefly

Angular relies heavily on *decorators* — functions written with an `@` prefix that attach metadata to a class or its members. You will see them constantly:

```
@Component({
  selector: "app-task-row",
  template: `<div>{{ task.title }}</div>`,
})
export class TaskRow {
  @Input() task!: Task;
}
```

For now, know only this: `@Component`, `@Input`, `@Output`, and their friends are how you tell Angular "this class is a component" or "this field is an input from a parent." Chapter 5 will explain what each does. TypeScript's role is simply to allow the syntax; the meaning is Angular's.

Note. Decorators are officially standardized in modern JavaScript now, but Angular still uses the older *experimental* decorator flavor because it carries more metadata that Angular relies on. The Angular CLI configures your project so this Just Works; you should never have to think about it. If you ever see `experimentalDecorators` in a `tsconfig.json`, that is why.

The `!` and `?` operators

Two small TypeScript-only sigils will appear often:

- `x!` is the *non-null assertion*. It tells the compiler "trust me, this is not null or undefined here." Use it sparingly; every `!` is a place where a runtime error can appear that the type system does not warn about. In component code, `@Input() task!: Task` uses `!` to promise "Angular will assign this before I use it," which is a promise Angular does keep — but only after component construction, not during it.
- `x?.y` is the *optional chaining* operator (which is actually plain JavaScript). It returns `undefined` if `x` is `null` or `undefined`, instead of throwing. `x?.y ?? "default"` combines it with the *nullish coalescing* operator we will meet in Chapter 3.

What we deliberately skipped

TypeScript has conditional types, mapped types, template literal types, `infer`, discriminated unions in their full generality, and a hundred other tools. You will encounter some of them, mostly in error messages, and you will be tempted to dive down that rabbit hole. Resist. The features in this chapter are the ones you will use every single day in Angular. The rest is available when you need it.

What comes next

Chapter 3 covers the modern JavaScript features Angular assumes you know: modules, arrow functions, destructuring, spread and rest, `async / await`, and the array methods you will lean on constantly. If you have already written recent JavaScript, you can skim it; if not, take it slow — Chapter 4 will assume every idea from Chapter 3.

Exercises

1. Write an interface `Habit` with these fields: `id` (string), `name` (string), `frequency` (one of `"daily"`, `"weekly"`, `"custom"`), `color` (optional string), `createdAt` (a `readonly` string). Then write a function `describe(habit: Habit): string` that returns something like `"Read (daily)"`.
2. Given `type Result<T> = { ok: true; value: T } | { ok: false; error: string }`, write a function `unwrap<T>(r: Result<T>): T` that re-

turns the value if `ok` is true and throws otherwise. Try to write it with narrowing rather than an `as` assertion.

3. In the Playground (typescriptlang.org/play), paste an object of shape `{ title: "test", priority: "high" }` and try to assign it to a variable of type `{ title: string; priority: number }`. Read the error message carefully. Then fix the object so it type-checks. Understanding this error format is a skill worth having.

Chapter 3. Modern JavaScript you'll actually use

Angular is TypeScript on the outside and JavaScript on the inside. Everything the compiler produces, the browser runs. If a piece of JavaScript syntax looks unfamiliar in an Angular example later in the book, the odds are it is one of the features in this chapter. There are dozens more we will not cover — the modern JavaScript language is very large — but the subset here is roughly a hundred percent of what Angular application code actually uses.

If you have written recent JavaScript before, you can read this chapter as a checklist and skim past what you already know. If not, plan on trying every example in a browser's developer console (open your browser, press F12, click "Console") so the syntax lands in your fingers.

Modules: `import` and `export`

Every file in a modern JavaScript or TypeScript project is a *module*. A module is a file that can *export* things (functions, classes, constants, types) for other files to use, and *import* things from other modules. This is the mechanism by which your app is split into files without polluting a global namespace.

There are two flavors of exports:

```
// tasks.ts

// Named export: exports a symbol under its own name.
export interface Task {
  id: string;
  title: string;
}

export function isOverdue(task: Task): boolean { /* ... */ return ... }

// Default export: one per file, imported without curly braces.
export default class TaskService {
  // ...
}
```

And correspondingly, two flavors of imports:

```
// app.ts

import TaskService from "./tasks";           // default
import { Task, isOverdue } from "./tasks";    // named
import * as tasks from "./tasks";             // everything, namespace
```

Angular's style guide, and this book, prefer *named exports* almost universally. Default exports save one pair of braces but cost clarity: a default export's name at the import site is whatever the importer types, so `import Foo from "./bar"` and `import Baz from "./bar"` reference the same thing. Named exports keep names honest across the codebase.

Import paths that start with `./` or `../` are *relative* — they point to another file in your project. Paths that start with anything else are

bare specifiers and refer to installed packages (`@angular/core`, `rxjs`, and so on).

Watch out. Do not put the `.ts` extension in your imports (from `"./tasks.ts"`). Angular's build tooling handles the extension for you. In some other JavaScript environments — notably Node.js in ESM mode — you *do* need the extension. Angular is the first case, not the second.

Arrow functions

An arrow function is a shorter way to write a function, with one important semantic difference from a normal function: it does not have its own `this`.

```
// Long form
const isDone = function(task) { return task.done; };

// Arrow, block body
const isDone = (task) => { return task.done; };

// Arrow, expression body – no braces, no return keyword
const isDone = task => task.done;

// Multiple parameters need parentheses
const compare = (a, b) => a.priority - b.priority;
```

The lack of its own `this` matters when you pass an arrow function as a callback:

```

class TaskService {
  tasks: Task[] = [];

  loadTasks() {
    fetch("/api/tasks")
      .then(res => res.json())
      .then(data => {
        // `this` here still refers to the TaskService instance,
        // because the arrow function inherits it from loadTasks.
        this.tasks = data;
      });
  }
}

```

If the inner callback had been a normal `function`, `this` inside it would be `undefined` (in strict mode) or the global object — a bug source that plagued JavaScript for a decade before arrows arrived. In Angular code, you will almost always want arrows for callbacks.

Template literals

Backtick strings can contain interpolated expressions and span multiple lines:

```

const name = "Anna";
const greeting = `Hello, ${name}. Today is ${new Date().toLocaleStrin

```

They are strictly better than string concatenation for anything longer than trivial. Angular templates themselves are template literals when you write inline templates:

```
@Component({
  template: `
    <h1>Welcome, {{ user.name }}</h1>
    <p>You have {{ taskCount() }} tasks today.</p>
  `,
})
```

The `{{ }}` inside is Angular's template syntax, not JavaScript's `${}`. They coexist because they never overlap: the outside literal is JavaScript, evaluated once when the component class is defined; the `{{ }}` is Angular's, re-evaluated whenever the data changes.

Destructuring

Destructuring pulls fields out of an object (or elements out of an array) into named variables in a single expression.

```
const task = { id: "t1", title: "Buy milk", done: false };

// Instead of:
const id = task.id;
const title = task.title;

// You can write:
const { id, title } = task;
```

You can rename on the way out and provide defaults:

```
const { id: taskId, title, done = false } = task;
```

Destructuring works in function parameters, too:

```
function label({ title, done }: Task): string {  
  return done ? `[x] ${title}` : `[ ] ${title}`;  
}  
  
label({ id: "t1", title: "Buy milk", done: false, /* ... */ });
```

Array destructuring uses the same idea with square brackets:

```
const [first, second, ...rest] = [1, 2, 3, 4, 5];  
// first = 1, second = 2, rest = [3, 4, 5]
```

The `...rest` on the last line is the spread/rest operator, which we look at next.

Spread and rest

The three-dot operator `...` has two related uses.

Spread expands the elements of an array (or the properties of an object) inline:

```
const priorities = [1, 2, 3];  
const withZero = [0, ...priorities];           // [0, 1, 2, 3]  
  
const base = { title: "Untitled", done: false };  
const withId = { ...base, id: "t1" };          // { title, done, id }
```

Object spread is how you make a shallow copy of an object, and how you produce a modified copy without mutating the original:

```
const task: Task = { /* ... */ };
const updated = { ...task, done: true };
// task.done is still whatever it was; updated.done is true.
```

This "don't mutate; produce a new object" pattern is *the* pattern of modern reactive code in Angular. When we get to signals in Chapter 7, you will see it constantly.

Rest collects the remaining elements of an array or the remaining parameters of a function into a single variable:

```
function sum(first: number, ...others: number[]): number {
  return others.reduce((acc, n) => acc + n, first);
}

sum(1, 2, 3, 4); // 10
```

Same three dots, opposite direction: spread expands, rest collects.

Default parameters

Parameters can have defaults, and the defaults can reference earlier parameters:

```
function paginate(items: Task[], page: number, size: number = 20)
  return items.slice(page * size, (page + 1) * size);
}

paginate(all, 0); // uses size = 20
paginate(all, 0, 50); // uses size = 50
```

Default parameters kick in whenever the argument is `undefined` — passing `null` does *not* trigger the default. This is worth remembering the first time it surprises you.

Optional chaining and nullish coalescing

Two operators that quietly replaced a lot of defensive `if` chains.

Optional chaining — `?.` — returns `undefined` short-circuit if the left side is `null` or `undefined`, instead of throwing:

```
const city = user?.address?.city;  
// Equivalent to:  
const city2 = user && user.address ? user.address.city : undefined;
```

Nullish coalescing — `??` — returns the right side if the left side is `null` or `undefined`, otherwise the left side:

```
const display = user.nickname ?? user.name ?? "Anonymous";
```

The important distinction from `||` (which you may have seen used similarly) is that `??` only falls through on `null` and `undefined`, not on other falsy values. `0 || 10` gives you 10; `0 ?? 10` gives you 0. In Compass, `task.priority ?? 3` is a safe "default priority to 3 if unset," while `task.priority || 3` would incorrectly rewrite an intentional priority of 0.

Promises

A **Promise** is a value that will exist in the future. Any operation that takes time — a network request, a file read, a timer — returns a Promise. A Promise is always in one of three states: *pending* (still waiting), *fulfilled* (produced a value), or *rejected* (produced an error).

You attach callbacks with `.then()` and `.catch()`:

```
fetch("/api/tasks")
  .then(res => res.json())
  .then(tasks => console.log(tasks))
  .catch(err => console.error("Something went wrong:", err));
```

Each `.then` receives the value the previous step returned. If any step throws or returns a rejected Promise, control jumps to the nearest `.catch`.

This "chain of thens" style used to be the standard way to write asynchronous code. It works, and Angular's underlying HTTP machinery still uses Promises in places, but the ergonomics are awkward as soon as the chain grows. Modern code uses `async` / `await` instead.

`async` / `await`

`async` marks a function as returning a Promise, and lets you use `await` inside it. `await` pauses the function at that expression until the awaited Promise resolves, and then resumes with its value.

```

async function loadTasks(): Promise<Task[]> {
  const res = await fetch("/api/tasks");
  const tasks: Task[] = await res.json();
  return tasks;
}

// Later:
const tasks = await loadTasks();

```

Errors in `await` ed Promises come out as thrown exceptions, so you can use plain `try / catch`:

```

try {
  const tasks = await loadTasks();
} catch (err) {
  console.error("Could not load tasks:", err);
}

```

`async / await` is a nicer way to write the same computation a chain of `.then()` s expresses — but it is *the same computation*. Nothing runs synchronously. `await` still yields control back to the browser between steps. It is syntactic sugar, and it is delicious.

Note. Angular sometimes uses Promises, sometimes uses *Observables* (an RxJS type), and increasingly uses signals for reactive state. Chapter 11 unpacks the difference between Promises and Observables — for now, the important thing is that you understand Promises, because everything else in the `async` world builds on them or replaces them.

Array methods you will use constantly

Modern JavaScript's array methods are the toolkit of everyday application code. You will lean on them daily.

```
const tasks: Task[] = /* ... */;

// map: transform each element
const titles = tasks.map(t => t.title);

// filter: keep the elements that match
const active = tasks.filter(t => !t.done);

// find: get the first element that matches, or undefined
const buyMilk = tasks.find(t => t.title === "Buy milk");

// some / every: test the array as a whole
const anyOverdue = tasks.some(isOverdue);
const allDone = tasks.every(t => t.done);

// reduce: fold the array into a single value
const total = tasks.reduce((sum, t) => sum + (t.done ? 1 : 0), 0);

// includes: does the array contain this value?
const hasWork = tags.includes("work");

// sort: mutating! usually you want to copy first
const byPriority = [...tasks].sort((a, b) => a.priority - b.priority);
```

Three warnings about `sort` and its friends:

1. `sort`, `reverse`, and `splice` mutate the array in place. If you want a new sorted array without disturbing the original, spread it first: `[...tasks].sort(...)`.

2. `sort` with no comparator function coerces elements to strings, which is a legendary source of bugs. Numeric sorts *need* a comparator: `.sort((a, b) => a - b)`.
3. `reduce` is powerful and overused. If you find yourself writing a `reduce` that could be a `for` loop with two variables, the `for` loop is usually clearer.

`for` loops

You have three main ways to iterate:

```
// for..of iterates values in an array-like or iterable
for (const task of tasks) {
  console.log(task.title);
}

// forEach is fine for side effects; can't break out of it
tasks.forEach(task => console.log(task.title));

// classical for is still useful when the index matters
for (let i = 0; i < tasks.length; i++) {
  console.log(`${i}: ${tasks[i].title}`);
}
```

`for..in` also exists but iterates *keys of an object*, not elements of an array. Because arrays have string keys ("0", "1", ...), `for..in` will do surprising things on arrays. Prefer `for..of` for arrays and `Object.keys(obj)` / `Object.entries(obj)` for objects.

Objects and shallow copies

Copying objects is a place beginners get burned. Assignment does not copy; it creates a new reference to the same object.

```
const original = { title: "Buy milk", done: false };
const alias = original;
alias.done = true;
console.log(original.done); // true – surprise
```

To copy, use spread (or `Object.assign`):

```
const copy = { ...original };
copy.done = true;
console.log(original.done); // still false
```

This is a *shallow* copy: nested objects are still shared. If your object has nested objects and you need a deep copy, use `structured-Clone(value)` (built into modern browsers).

Classes, briefly

Angular is class-forward: components, services, directives, and pipes are all classes. You do not need to learn everything about classes today — Chapter 5 will do that — but the syntax is unsurprising:

```
class Counter {  
  private count = 0;  
  
  increment(): void {  
    this.count += 1;  
  }  
  
  get value(): number {  
    return this.count;  
  }  
}  
  
const c = new Counter();  
c.increment();  
console.log(c.value); // 1
```

The `private` keyword is TypeScript's, not JavaScript's — at runtime the field is accessible, but the compiler will refuse to let external code touch it. Modern JavaScript also has real `#` private fields; Angular's codebase uses TypeScript `private` for convention, so we will too.

What we deliberately skipped

Generators (`function*`), the `Symbol` type, `Proxy`, `Reflect`, `Map` and `Set` (you will meet these when they help), regular expressions in detail, and the many subtleties of `this`. All are useful; none are required to build Compass. When you meet one of them, look it up on MDN — the Mozilla Developer Network is the canonical JavaScript reference.

What comes next

Chapter 4 gets you a running Angular application. You will install Node.js, install the Angular CLI, run `ng new compass`, and take a slow tour of every file the CLI produces. By the end of Chapter 4, you will have Angular's default starter screen open in your browser and you will understand what every part of it is doing.

Exercises

1. Given `const nums = [3, 1, 4, 1, 5, 9, 2, 6]`, use array methods to produce (a) an array of squares, (b) the sum of the squares, (c) the largest number, and (d) whether every number is less than 10. Write each as a single expression.
2. Write a function `updateTask(tasks: Task[], id: string, changes: Partial<Task>): Task[]` that returns a new array where the task with matching `id` has `changes` merged into it, and every other task is untouched. Use `map` and spread; do not mutate the input. (The type `Partial<Task>` means "all fields of `Task`, but every one is optional.")
3. Convert this Promise chain to `async / await`:

```
function loadWithAuth(): Promise<Task[]> {  
  return fetch("/api/session")  
    .then(res => res.json())  
    .then(session => fetch("/api/tasks", { headers: { Authorization: session.token } })  
    .then(res => res.json());  
}
```

Then think about what happens if the first `fetch` fails. Where does the error surface in each version?

Chapter 4. The CLI, your first app, and the anatomy of an Angular project

This chapter is different from the previous three. It is the first chapter you should read *with your laptop open*. By the end of it, Compass will exist as a project on your disk, its dev server will be running, and its starter page will be open in your browser. Then we will go through every file the Angular CLI produced, so nothing in your new project looks mysterious the first time you open it in an editor.

If you get stuck installing Node.js, jump to Appendix A for platform-specific notes. Come back when you have a working `node --version` and `npm --version` in your terminal.

What we're installing, and why

You need three things to work on Angular:

1. **Node.js** — a runtime that lets JavaScript run outside a browser. Angular's development tools (the compiler, the dev server, the test runner) are JavaScript programs; they run on Node.js. You will barely interact with Node.js directly, but everything depends on it.
2. **npm** — Node.js's package manager. It comes bundled with Node.js. It downloads libraries — including Angular itself

— into a folder called `node_modules` inside your project.

3. **The Angular CLI** — the `ng` command. This is a tool that scaffolds new projects, generates new files, runs your app, runs your tests, and builds for production. Everything you do with Angular for the rest of this book goes through `ng`.

Installing Node.js

Go to `nodejs.org` and download the **LTS** (long-term support) version. At the time of writing this is Node.js 20; when you read this it will likely be Node.js 22 or later. Either is fine as long as it is 20 or above; Angular refuses to install on older Node versions.

Follow the installer for your platform. On macOS and Windows, the installer places `node` and `npm` on your `PATH` automatically; on Linux, use your distribution's package manager or the tarball on the Node.js website (see Appendix A).

Open a terminal — the *Terminal* app on macOS, *Windows Terminal* or *PowerShell* on Windows, your favorite terminal emulator on Linux — and verify:

```
node --version    # v20.x.x or higher
npm --version     # 10.x.x or higher
```

Both should print a version number. If either says "command not found," see Appendix A.

Installing the Angular CLI

With Node.js in place, install the CLI globally with npm:

```
npm install --global @angular/cli
```

The `--global` flag makes `ng` available anywhere on your machine, rather than only in one project's folder. Confirm:

```
ng version
```

You should see the Angular CLI version, plus a header listing the packages it will install. If you get "command not found," open a new terminal window — the shell needs to notice the newly-installed binary.

Note. Some people prefer never to install anything globally. If that is you, you can use `npx @angular/cli` in place of `ng` anywhere in this book. It downloads a temporary copy each time. It is slower but keeps your global environment clean.

Creating Compass

Pick a folder to keep your projects in. In this book I will use `~/dev`. Substitute your own if you prefer.

```
cd ~/dev  
ng new compass
```

The CLI asks you a few questions. Answer them as follows:

- **Which stylesheet format would you like to use?** → `CSS` . We will not use Sass in this book. If you already know Sass, you can pick it; the code will still work.
- **Do you want to enable Server-Side Rendering (SSR)...**? → `No` . We will add SSR in Chapter 18, once the app is worth prerendering.
- **Which type of routes would you like to configure?** — if this appears, choose the default (server routes with data prefetching if offered; otherwise leave routing enabled). Newer CLIs default this without asking.

The CLI creates a `compass` folder next to wherever you ran the command, downloads roughly two hundred packages into `node_modules` (this will take a minute or two the first time), and stops. It then leaves you at your prompt with a small hint about `ng serve` .

Take that hint:

```
cd compass
ng serve --open
```

A few seconds later, your browser will open at `http://localhost:4200` and you will see Angular's default starter page. Congratulations — you are running an Angular application.

`ng serve` also stays open in your terminal, watching your source files. Save a change, and the browser reloads automatically. Leave

`ng serve` running in one terminal window; open a second terminal for the rest of this chapter.

A tour of every file the CLI produced

Open `~/dev/compass` in your editor of choice. If you don't have a preferred editor, install **Visual Studio Code** from `code.visualstudio.com` and open the folder with `File → Open Folder`. VS Code has the best out-of-the-box Angular support of any free editor; when we start writing components in Chapter 5, its inline error messages will save you a lot of time.

We are going to walk through every top-level file and folder now. It is a long walk. Do not try to memorize; just get a mental map of what each thing is for. You will meet each of these files again, in context, later in the book.

`package.json`

This is the description of your project as far as npm is concerned. Two sections are worth inspecting.

`scripts` lists the commands you can run via `npm run <name>`:

```
"scripts": {  
  "ng": "ng",  
  "start": "ng serve",  
  "build": "ng build",  
  "watch": "ng build --watch --configuration development",  
  "test": "ng test"  
}
```

`npm run start` and `ng serve` are equivalent. Some conventions prefer `npm run` because it keeps the same commands working in projects that use different underlying tools; in an Angular project you can use either.

`dependencies` lists the libraries your app needs at runtime; `devDependencies` lists the tools it needs during development. When a new Angular version comes out and you want to update, you edit these version numbers or run `ng update` — a topic for another day.

angular.json

The CLI's configuration file. It tells Angular how to build, serve, and test the project. You will rarely edit it by hand in the early chapters; we will visit it in Chapter 16 (performance) and Chapter 19 (deployment). For now, know that it exists.

tsconfig.json and its friends

Three TypeScript configuration files:

- `tsconfig.json` — the base configuration.

- `tsconfig.app.json` — the configuration used when building the app itself.
- `tsconfig.spec.json` — the configuration used when running tests.

You will not touch these until you start writing tests in Chapter 17. The main file enables *strict mode* by default in modern Angular projects, which means TypeScript will be strict about `null` checks, implicit `any`, and other footguns. Do not turn strict mode off. It is the reason your app will remain refactorable a year from now.

`.editorconfig`

A tiny file that tells your editor about indentation, line endings, and other basics. If your editor understands it (most modern ones do, sometimes with a plugin), your formatting will match the CLI's defaults automatically.

`.gitignore`

A file listing what git should not track. Notably it excludes `node_modules/` (which is huge and reproducible from `package.json`) and `dist/` (build output). We will initialize a git repository shortly.

`node_modules/`

The installed packages. Angular itself lives here, along with everything Angular depends on and everything you install later. Do not check this folder into git; it is often gigabytes in size and is fully

reproducible with `npm install`. If you delete it, you can recreate it with `npm install`.

`public/`

Static files served as-is at the root of your app. If you drop `robots.txt` or `favicon.ico` here, they appear at `/robots.txt` and `/favicon.ico` in the running app. The CLI drops a default `favicon.ico` here.

`src/index.html`

Open it. It is very short:

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>Compass</title>
  <base href="/">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <link rel="icon" type="image/x-icon" href="favicon.ico">
</head>
<body>
  <app-root></app-root>
</body>
</html>
```

That `<app-root>` element is the single anchor point for the entire application. When the browser loads the page, Angular boots up, finds this element, and takes it over. Everything you see on `http://localhost:4200` is rendered inside `<app-root>`.

You will rarely edit `index.html`. You edit its title, occasionally add a `<link>` for a font, and otherwise leave it alone.

`src/styles.css`

Global CSS. Anything in here applies to the whole application. Font imports, CSS resets, and root-level custom properties (CSS variables) live here. Per-component styles — the styles for a task row, say — live in the component file, which we meet in Chapter 5.

`src/main.ts`

The bootstrap file. Very short:

```
import { bootstrapApplication } from '@angular/platform-browser';
import { appConfig } from './app/app.config';
import { App } from './app/app';

bootstrapApplication(App, appConfig)
  .catch(err => console.error(err));
```

`bootstrapApplication` says: "start the Angular application, whose root component is `App` and whose configuration is `appConfig`, in whatever element in the HTML has the tag defined by `App`'s selector." The result is that `<app-root>` in `index.html` gets populated with the rendered root component.

You will edit `main.ts` almost never — perhaps once in Chapter 18 to enable server-side rendering.

`src/app/`

This is where your application lives. Everything you write from Chapter 5 onward goes into `src/app/` or its subfolders. The CLI has already put three files here:

`app.ts` (or `app.component.ts` in some CLI versions) — the root component. Roughly:

```
import { Component, signal } from '@angular/core';
import { RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-root',
  imports: [RouterOutlet],
  templateUrl: './app.html',
  styleUrls: ['./app.css'],
})
export class App {
  protected readonly title = signal('compass');
}
```

The four things to notice, none of which you need to *understand* yet:

- The `@Component` decorator. It tells Angular that this class is a component, and it configures the component with a selector (`<app-root>`), a template file, a style file, and any other components or directives this component uses in its template.
- The class body. Anything the component wants to expose to its template goes here — fields, methods, signals.

- The `signal(...)` call. That is Angular's modern reactive state primitive. The template can read `title()` and it will re-render if the signal ever changes.
- `imports: [RouterOutlet]`. Standalone components declare what other components and directives they use. `RouterOutlet` is the placeholder where the router will render pages.

Every chapter from Chapter 5 onward spends time inside components like this one. Do not worry about parsing it now.

app.html — the template for the root component. The CLI fills this with a Angular-branded starter page. You will delete most of it in Chapter 5.

app.css — styles scoped to the root component. Anything you write here applies to `<app-root>` and nothing else — even if you have a `.container` class in this file and another `.container` class somewhere else, they will not collide. This is called *view encapsulation*, and it is one of the quietly good things about writing Angular.

app.config.ts — the app-level configuration.

```
import { ApplicationConfig, provideBrowserGlobalErrorListeners, p
import { provideRouter } from '@angular/router';
import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    provideZonelessChangeDetection(),
    provideRouter(routes),
  ],
};
```

(Older CLI versions emit `provideZoneChangeDetection({ eventCoalescing: true })` instead of `provideZonelessChangeDetection()` and don't include `provideBrowserGlobalErrorListeners`. Both work; the modern defaults shown here are what current `ng new` produces.)

`providers` is the list of services and configuration your app makes available everywhere. In modern Angular, most cross-cutting concerns are set up by calling `provideX()` functions here — routing, HTTP, animations, and so on. We will add several of them in later chapters.

`app.routes.ts` — an initially-empty list of route definitions:

```
import { Routes } from '@angular/router';

export const routes: Routes = [];
```

Chapter 13 fills this in and makes Compass a multi-page app.

What the CLI can do besides `new` and `serve`

Get a habit of using `ng generate` for scaffolding. It saves typing and, more importantly, it puts new files in the right place with the right imports.

```
ng generate component task-row      # creates src/app/task-row/*
ng generate service tasks           # creates src/app/tasks.service.ts
ng generate pipe date-friendly     # creates src/app/date-friendly.pipe.ts
```

You can abbreviate to `ng g c task-row`. In this book we will always type the long form so you know what you are doing; in daily practice you will abbreviate.

Other commands worth knowing:

```
ng serve      # dev server; watches files, reloads on save
ng build      # production build into dist/
ng test       # runs the unit tests
ng lint       # runs the linter, if configured
ng update     # updates Angular and related packages
```

Git, right now

Before you write a single line of code, initialize a git repository. Your future self will thank you the first time you break something and want to see what you had five minutes ago.

```
git init
git add .
git commit -m "Initial commit from ng new"
```

If you have never used git, spend twenty minutes with the first two chapters of any git primer (`git-scm.com/book` is free). You will use `git status`, `git add`, `git commit`, and `git diff` about fifty times each in this book.

Making sure it all works

With `ng serve` still running from earlier, open `src/app/app.html` in your editor. Find the first `<h1>` (or a place with visible text) and change some text to `Hello, Compass`. . Save the file.

Your browser tab should refresh within a second, and the new text should appear. If it does, you have a working Angular development environment.

If it does not — the page does not update, or the terminal prints an error — see Appendix A's troubleshooting section, or Appendix B, which is a guide to reading Angular error messages.

What comes next

Part II. In Chapter 5 we will delete most of the CLI's starter content and build Compass's first real component: a list of tasks. You will meet templates, data binding, and the new control flow syntax. By the end of Chapter 5, Compass will be a to-do list — a very simple one, running entirely in the browser, without any of the persistence, routing, or authentication of the finished product. Every subsequent chapter adds one more real capability.

Exercises

1. Run `ng generate component about --skip-tests --dry-run` and read the output. The `--dry-run` flag tells the CLI to print what it would do without actually doing it. What files would be created? Do the same for `ng generate service auth --dry-run`. This is a useful habit for previewing what the CLI wants to do before it does it.
2. Open the browser's developer tools (F12 in most browsers) and click the "Elements" or "Inspector" tab. Find the `<app-root>` element in the DOM. Note that it is a real element in the page, and the app's content is rendered inside it. This is worth seeing once so you know Angular is not doing anything magical to the HTML.
3. In `src/app/app.html`, put `{{ title() }}` somewhere the text will be visible, and save. Notice the parentheses after `title` — that is not a typo. Signals are functions you call to get their current value. In Chapter 7 we will explore why. For now, just observe that it works.

Chapter 5. Components: the atom of an Angular app

We have set up the workshop; now we start building. This is the first chapter where you write real Angular code. By the end you will have deleted the CLI's starter template, replaced it with your own component, and displayed a hardcoded list of tasks in the browser. That is a small accomplishment on its own, and it is also the moment where the shape of an Angular application clicks into place.

Keep `ng serve` running in one terminal, and your editor open on the `compass` folder. Every code block in this chapter is meant to be typed in and observed.

What a component is, revisited

In Chapter 1 I described a component as a *self-contained piece of the screen*: some HTML, some CSS, and some behavior, packaged together. Now that you have a project on your disk, we can be more concrete. A component is:

- **A TypeScript class** that holds the component's state (fields) and behavior (methods).
- **Decorated with `@Component`** — a piece of metadata that tells Angular "this class is a component, and here is its selector and its template."

- **Associated with a template**, which is a piece of HTML augmented with Angular's binding syntax.
- **Optionally associated with styles**, scoped so they do not leak to the rest of the app.
- **Used in another component's template by name**, via its selector.

That last point is the mechanism by which a component *tree* forms. Compass's root component (`App`) will eventually include a `<app-task-list>` in its template. That `<app-task-list>` is another component we are about to write. Inside `<app-task-list>` will be many `<app-task-row>` instances, one per task. Every element that starts with `app-` in your templates is a component you defined; every element that does not (like `<div>` or `<button>`) is regular HTML.

The `@Component` decorator, field by field

Open `src/app/app.ts` (or `app.component.ts` ; the filename depends on the CLI version, and the CLI defaults have varied). It looks something like:

```
import { Component, signal } from '@angular/core';
import { RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-root',
  imports: [RouterOutlet],
  templateUrl: './app.html',
  styleUrls: ['./app.css'],
})
export class App {
  protected readonly title = signal('compass');
}
```

Let's walk through the `@Component` fields one by one.

selector is the HTML tag name that represents this component. `'app-root'` means this component appears anywhere `<app-root>` is written. Selectors are how components find each other; they are also how the outside world (in this case, `index.html`) can drop your app into a page. Angular convention is a short, application-specific prefix (`app-` here) followed by the component's role. You will name components like `<app-task-row>`, `<app-add-task-form>`, `<app-habit-badge>`.

Why the prefix? To avoid collisions with real HTML elements and with third-party components. A tag like `<button>` is a native HTML element; a tag like `<mat-button>` is from Angular Material; a tag like `<app-button>` is yours. The prefix keeps them straight.

imports is a list of other components, directives, and pipes this component's template uses. In modern Angular, every standalone component declares its dependencies explicitly. If your template contains `<router-outlet>`, you must import `RouterOutlet`; if it

contains `<app-task-row>`, you must import `TaskRow`. This is one of the things that surprises newcomers ("do I really have to list everything?") and then quietly earns its keep ("I moved this component into another app and it still works").

`templateUrl` points to the HTML file that this component renders. Alternatively you can use `template`, which takes an inline string:

```
@Component({
  selector: 'app-hello',
  template: '<h1>Hello.</h1>',
})
export class Hello {}
```

Inline templates are convenient for tiny components (under about 10 lines). Anything larger belongs in a separate file. The book uses both, favoring separate files for real components and inline for one-liners.

`styleUrl` (or `styles`, or `styleUrls`) points to the component's scoped CSS. Everything in the file applies only inside instances of this component — a class called `.title` here will not collide with a `.title` class elsewhere in the app. This is done via *view encapsulation*, which Angular implements by adding a unique attribute to each element the component renders and rewriting your CSS to target that attribute. You do not need to think about the mechanism; you only need to know that CSS you write in a component stays inside that component.

Building Compass's first real component

Let's replace the CLI's starter with something we own. We'll build a `TaskList` component that shows a hardcoded list of tasks. In later chapters we will replace the hardcoded data with signals, then with a service, then with HTTP.

From your `compass` folder:

```
ng generate component task-list
```

This creates `src/app/task-list/` containing:

```
task-list/
├─ task-list.ts           (or task-list.component.ts)
├─ task-list.html
├─ task-list.css
└─ task-list.spec.ts
```

Open `task-list.ts`:

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-task-list',
  imports: [],
  templateUrl: './task-list.html',
  styleUrls: ['./task-list.css'],
})
export class TaskList {}
```

We are going to give this component a *type* for tasks and a hardcoded list. First, let's put the type in its own file so we can share it later. Create `src/app/task.ts`:

```
export interface Task {  
  id: string;  
  title: string;  
  done: boolean;  
  createdAt: string;  
  dueDate: string | null;  
  tags: string[];  
}
```

Now edit `task-list.ts`:

```
import { Component } from '@angular/core';
import { Task } from '../task';

@Component({
  selector: 'app-task-list',
  imports: [],
  templateUrl: './task-list.html',
  styleUrls: ['./task-list.css'],
})
export class TaskList {
  tasks: Task[] = [
    {
      id: 't1',
      title: 'Buy milk',
      done: false,
      createdAt: '2026-01-15T09:00:00Z',
      dueDate: null,
      tags: ['home'],
    },
    {
      id: 't2',
      title: 'Write chapter 5',
      done: true,
      createdAt: '2026-01-14T18:30:00Z',
      dueDate: '2026-01-16T00:00:00Z',
      tags: ['work'],
    },
    {
      id: 't3',
      title: 'Call the plumber',
      done: false,
      createdAt: '2026-01-15T11:00:00Z',
      dueDate: '2026-01-18T00:00:00Z',
      tags: ['home', 'urgent'],
    },
  ],
};
}
```

Now edit `task-list.html` to display them. Chapter 6 covers the syntax in depth; for now, copy it and observe:

```
<h1>Tasks</h1>
<ul class="task-list">
  @for (task of tasks; track task.id) {
    <li class="task" [class.done]="task.done">
      <span class="title">{{ task.title }}</span>
      @if (task.dueDate) {
        <span class="due">due {{ task.dueDate | date: 'shortDate' }}
      }
    </li>
  } @empty {
    <li class="empty">Nothing to do. Nice.</li>
  }
</ul>
```

And give it some scoped styling in `task-list.css` :

```
:host {  
  display: block;  
  max-width: 480px;  
  margin: 2em auto;  
  font-family: system-ui, sans-serif;  
}  
  
h1 {  
  font-size: 1.4rem;  
  margin: 0 0 1em 0;  
}  
  
.task-list {  
  list-style: none;  
  padding: 0;  
  margin: 0;  
}  
  
.task {  
  padding: 12px 14px;  
  border-bottom: 1px solid #eee;  
  display: flex;  
  gap: 12px;  
  align-items: center;  
}  
  
.task.done .title {  
  text-decoration: line-through;  
  color: #999;  
}  
  
.due {  
  margin-left: auto;  
  font-size: 0.85rem;  
  color: #666;  
}
```



```
.empty {  
  padding: 20px;  
  text-align: center;  
  color: #999;  
  font-style: italic;  
}
```

Using the component

Two more edits and we can see it in the browser.

First, tell the root component to render `<app-task-list>`. Edit `src/app/app.ts`:

```
import { Component } from '@angular/core';  
import { TaskList } from '../task-list/task-list';  
  
@Component({  
  selector: 'app-root',  
  imports: [TaskList],  
  templateUrl: './app.html',  
  styleUrls: ['./app.css'],  
})  
export class App {}
```

Two important changes: we removed the `RouterOutlet` import (we'll bring it back in Chapter 13), and we added `TaskList` to `imports`. Any component that appears in this component's template must be imported.

Second, edit `src/app/app.html`. Delete whatever the CLI put there and replace with:

```
<app-task-list></app-task-list>
```

Save. Look at your browser tab. You should see a heading "Tasks" and three items — one with a strikethrough (the completed one) and two with due dates.

You just wrote your first Angular component and mounted it in the app. Take a minute to appreciate that.

What `:host` is doing

You probably noticed the `:host` pseudo-class in `task-list.css`. That is how a component's stylesheet refers to the component's own root element — `<app-task-list>` itself, from outside. Inside the CSS file, `:host` is the wrapper; everything inside the component's template is descendant content of `:host`.

The distinction matters because Angular renders each component into its own element. When Angular sees `<app-task-list>` in App's template, it does not replace that tag with the task list's markup; it *inserts* the markup inside the tag. That preserves the encapsulation boundary and gives you a place to hang styles like `display: block` or `margin` on the component as a whole.

Component lifecycle, in one paragraph

Every component goes through a few moments in its life: it is *constructed* (the class is instantiated), then *initialized* (Angular has wired up its inputs), and eventually *destroyed* (the component is removed

from the DOM, and it should clean up any subscriptions). Angular provides lifecycle *hooks* — methods you can define with specific names — for these moments: `ngOnInit`, `ngOnDestroy`, and a handful of others. In older Angular code these were everywhere.

In modern Angular, most of the reasons you would have reached for a lifecycle hook are better served by other primitives — signals, `computed`, `effect`, and DI-based reactivity. The remaining valid uses are narrow: doing one-time setup that needs to see inputs (rare — `computed` is usually better), or cleanup of external resources not managed by Angular (rare — `takeUntilDestroyed` is usually better). We will meet the hooks properly when we need them; do not go looking for them.

Signal-friendly by default

Note that in this chapter, `tasks` is a plain field on the class — `tasks: Task[] = [...]`. That works: the template reads it and renders. But it is not *reactive*. If you later add a button that pushes a new task onto the list, Angular will re-render the list (thanks to zone-based change detection, which is the classic mechanism); but this behavior is on its way out. Modern Angular is moving toward signals as the reactive primitive, and Compass will make that shift starting in Chapter 7.

For now, plain fields are enough. Chapter 6 spends its time on the template syntax (`@for`, `@if`, `{{ }}`); Chapter 7 rewrites `tasks` as a signal. The pattern of the book is: introduce the plain version, feel the itch, learn the reactive version, sigh with relief.

Inline vs external templates

Two styles, both are fine:

```
// External (default from `ng generate`)
@Component({
  selector: 'app-task-list',
  templateUrl: './task-list.html',
  styleUrls: ['./task-list.css'],
})

// Inline
@Component({
  selector: 'app-empty-state',
  template: `
    <div class="empty">
      <p>{{ message }}</p>
    </div>
  `,
  styles: `
    .empty { padding: 20px; text-align: center; }
  `,
})
```

Rules of thumb: templates under about 10 lines of markup are fine inline; anything larger is easier to read in a separate file. If your inline template starts having `@for` and multiple `@if` branches, move it to a `.html` file — your editor will thank you (syntax highlighting, autocomplete, indentation all get better).

View encapsulation, briefly

Every component has a `ViewEncapsulation` mode. The default is `Emulated`, which is what we described above — Angular scopes your CSS to the component by adding attributes. `None` disables scoping (your CSS becomes global — usually not what you want). `ShadowDom` uses the browser's actual Shadow DOM API for scoping, which is stronger but comes with quirks around styling, fonts, and third-party libraries.

You will almost always leave encapsulation on the default. The one time you would change it is if you were building a component library and needed the strongest possible isolation.

What comes next

Chapter 6 covers everything that just happened in the template. You have seen `{{ }}`, `[class.done]`, `@for`, `@if`, `@empty`, and `| date: 'shortDate'`. In Chapter 6 we look at each of them properly, along with property bindings, event bindings, and two-way bindings. By the end of Chapter 6, Compass will respond to clicks — you will be able to toggle tasks between done and not done.

Exercises

1. Add a fourth task to the hardcoded array — one that is `done: true` and has a `dueDate` in the past. Save and confirm it renders with the strikethrough and its overdue date. Do not use any Angular features other than the ones this chapter covered.

2. Change the `selector` in `task-list.ts` from `'app-task-list'` to `'compass-task-list'`. Save and refresh the browser. What breaks? Fix it. Notice that selectors are a small contract between two files (the component and its user).
3. Run `ng generate component habit-badge`. Read the four files the CLI produces. Which of them are new information after this chapter? Which are boilerplate you can now write from memory?

Chapter 6. Data binding and the new control flow

Templates are how Angular describes a user interface. They are HTML plus a small set of extensions: interpolations to show data, bindings to connect data to attributes and events, and block syntax to render lists and conditionals. This chapter covers every extension you will use daily. By the end, Compass will let you click a task to toggle it between done and not done.

Keep the state of Compass from Chapter 5. We will add to `task-list.ts` and `task-list.html` throughout.

Interpolation: getting values onto the page

You met `{{ }}` in Chapter 5. It evaluates a TypeScript expression against the component's fields and inserts the result as text.

```
<h1>{{ heading }}</h1>
<p>You have {{ tasks.length }} tasks.</p>
<p>Active: {{ tasks.filter(t => !t.done).length }}</p>
```

Anything you put between the double braces is an expression, not a statement. That means:

- You can call methods and use operators:

```
{{ user.firstName + ' ' + user.lastName }}
```

- You cannot assign, throw, or use `new`. `{{ x = 5 }}` will not compile as a template expression.
- The expression must be *side-effect-free*, or Angular will call it many times per second and the app will misbehave.

Interpolation always produces text. To bind to *attributes* or *properties* of an element, you use one of the binding forms below.

Property binding: `[prop]="expression"`

Property binding sets an HTML element's *property* (not its attribute — those are different, more on that shortly) to the value of an expression.

```
<img [src]="user.avatarUrl" [alt]="user.name" />
<input [value]="task.title" [disabled]="task.done" />
<a [href]="'/tasks/' + task.id">Details</a>
```

The square brackets on the left name the property. The double quotes on the right hold a TypeScript expression evaluated in the component's context.

Property vs attribute

HTML has both attributes (what you write in the source) and properties (what the DOM object actually holds). For most elements they are aligned, but not always. `<input value="hello">` sets the *initial* attribute; the running DOM element has a `.value` property that changes as the user types. Property binding writes to the property.

In the rare case you actually need to set an attribute (usually for ARIA), Angular has a special syntax:

```
<button [attr.aria-label]=" 'Delete task ' + task.title">x</button>
```

`[attr.name]` sets the HTML attribute named `name`. If Angular ever gives you a "no such property" error on an element, this is often the fix.

Class and style shortcuts

Two special forms of property binding are so common they get short syntax:

```
<li [class.done]="task.done" [class.overdue]="isOverdue(task)">
  <p [style.color]="task.done ? '#999' : 'inherit'">{{ task.title }}
```

`[class.name]` toggles the class based on the truthiness of the expression. `[style.property]` sets the inline style property. Both forms accept multiple entries — you can have `[class.done]` and `[class.overdue]` on the same element.

For more elaborate class or style logic, bind an object or array:

```
<li [class]="{ done: task.done, overdue: isOverdue(task) }">
<li [class]="cssClasses()">
```

Event binding: `(event)="handler(...)"`

Event bindings run a component method when a DOM event fires.

```
<button (click)="markDone(task)">Done</button>
<input (input)="onSearch($event)" />
<form (submit)="save()">...</form>
```

The parentheses on the left name the event (any DOM event: `click`, `input`, `focus`, `keydown`, whatever). The right side is a *statement* — this is the one place in a template where you can call a method for its side effect. `$event` inside the statement refers to the event object (the same object you would receive in a plain JavaScript listener).

Let's use it in Compass. Open `task-list.ts` and add a method:

```
export class TaskList {
  tasks: Task[] = [ /* ... */ ];

  toggle(task: Task): void {
    task.done = !task.done;
  }
}
```

And in `task-list.html`, wire up the click:

```
<li class="task" [class.done]="task.done" (click)="toggle(task)">
  <span class="title">{{ task.title }}</span>
  @if (task.dueDate) {
    <span class="due">due {{ task.dueDate | date: 'shortDate' }}</span>
  }
</li>
```

Save. In the browser, click any task. It should toggle between done and not done, with the strikethrough appearing and disappearing.

Two things worth noting:

- The click handler mutates `task.done` in place. This works, but it works *only* because Angular's zone-based change detection re-checks templates after any DOM event. This is fragile: signals (Chapter 7) will make the same code more explicit and better-behaved.
- The whole `<li>` is now clickable. That is fine for now but bad accessibility practice — a proper implementation would use a `<button>` or add keyboard handling. We will fix that when we come back to accessibility.

Two-way binding: `[(ngModel)]="value"`

Two-way binding is Angular's syntax for the common pattern of "bind a property in *and* listen for changes going out." The canonical use is form inputs:

```
<input [(ngModel)]="task.title" />
```

`ngModel` is a directive from `@angular/forms` — using it requires adding `FormsModule` to the component's `imports:` array. If you paste this into a standalone component without that import, the template compiler will complain with "Can't bind to 'ngModel'."

Reading right to left, the parentheses inside the brackets are the "banana in a box" mnemonic. The syntax expands to a property binding on `ngModel` and an event binding on `ngModelChange`:

```
<!-- Equivalent to the above -->
```

```
<input [ngModel]="task.title" (ngModelChange)="task.title = $event" />
```

`ngModel` is one specific implementation of the two-way pattern. Signals-based two-way binding, which we will use in Compass, uses a *model input* — a concept we will meet in Chapter 8. For now, know only that `[(x)]` is Angular's syntax for two-way binding, and that its semantics are always "property in, event out." There is no magic — it is not a data binding library — just a syntactic bundle.

The new control flow

Templates need to make decisions ("show this if the user is logged in") and repeat ("render one row per task"). Angular's modern syntax for these is called *the new control flow*: `@if`, `@for` (with its `@empty` sub-block), `@switch`, and the local-variable form `@let`. They replace an older set of *structural directives* — `*ngIf`, `*ngFor`, `*ngSwitchCase` — that you will still see in existing codebases. Both work; the new syntax became stable in Angular 17 (late 2023) and is the recommended default in every project since.

`@if` and `@else`

```
@if (user.loggedIn) {
  <p>Welcome back, {{ user.name }}.</p>
} @else if (user.pending) {
  <p>Confirming your email...</p>
} @else {
  <a routerLink="/login">Sign in</a>
}
```

Braces are required around the branches. You can nest freely. There is no ambiguity about where a branch ends.

@for and track

```
@for (task of tasks; track task.id) {  
  <li>{{ task.title }}</li>  
}
```

The `track` clause is required. It tells Angular how to identify each item across renders. When the array changes — you add a task, remove one, reorder — Angular uses `track` to decide which existing DOM nodes correspond to which items in the new array. Correct tracking means Angular reuses DOM nodes (fast); wrong tracking means Angular re-creates every node every time (slow).

Track by a stable identifier — a database id, a unique string — whenever you have one. Track by index (`track $index`) only as a last resort, and only when items are truly interchangeable.

The `@for` block has other useful contextual variables:

```
@for (task of tasks; track task.id; let i = $index; let first = $  
  <li>  
    #{{ i + 1 }}: {{ task.title }}  
    @if (first) { <span>(first)</span> }  
    @if (last) { <span>(last)</span> }  
  </li>  
}
```

Available: `$index`, `$first`, `$last`, `$even`, `$odd`, `$count`.

And, importantly, an `@empty` block:

```
@for (task of tasks; track task.id) {
  <li>{{ task.title }}</li>
} @empty {
  <li>Nothing to do.</li>
}
```

`@empty` renders when the array is empty. This replaces an idiom you would have written with a separate `@if (tasks.length === 0)` — cleaner, and lets Angular optimize the empty case.

`@switch`, `@case`, `@default`

For multi-branch dispatch on a single value:

```
@switch (badgeFor(task)) {
  @case ('overdue') { <span class="badge overdue">Overdue</span> }
  @case ('due-today') { <span class="badge today">Today</span> }
  @case ('later') { <span class="badge later">Later</span> }
  @default { <span class="badge">—</span> }
}
```

(The `badgeFor(task)` method returns one of those literal strings based on the task's `dueDate` and `done` fields. Compass's `Task` interface has `done` and `dueDate`, not a `status` field.)

You can nest `@switch` inside `@for`, `@if` inside `@switch`, and so on. There are no restrictions.

`@let`

`@let` binds a local variable inside a template so you can reuse a computation:

```

@let activeCount = tasks.filter(t => !t.done).length;
@let hasActive = activeCount > 0;

@if (hasActive) {
  <p>{{ activeCount }} tasks remaining.</p>
} @else {
  <p>All done for today.</p>
}

```

Two things to know: `@let` bindings are scoped to their surrounding block; and the expression is re-evaluated on every change-detection cycle (like any template binding), so the binding stays fresh whenever any of its inputs change.

Pipes: transform in-place

You have already used a pipe: `{{ task.dueDate | date: 'shortDate' }}`. A *pipe* is a function you can apply inside a template expression, using `|`. Its job is one-way, side-effect-free formatting.

Built-in pipes worth knowing:

```

{{ task.title | uppercase }}
{{ task.title | lowercase }}
{{ task.title | titlecase }}
{{ task.createdAt | date: 'medium' }}
{{ percentage | percent: '1.0-2' }}
{{ price | currency: 'USD' }}
{{ someObject | json }}      <!-- great for debugging -->
{{ items | slice: 0 : 5 }}

```

Pipes can chain:

```
{{ task.title | slice: 0 : 40 | titlecase }}
```

You will write your own pipes in Chapter 15. Until then, the built-ins cover most needs.

Attribute bindings you will need

Beyond `class` and `style`, a few attribute bindings come up often:

```
<button [disabled]="loading">Save</button>
<input [placeholder]="hint" [autofocus]="isNew" />
<a [href]="url" [target]="external ? '_blank' : '_self'">Link</a>
<img [src]="avatarUrl" [width]="size" [height]="size" [alt]="user
```

Most HTML attributes have a corresponding property, and Angular is smart enough to bind to the property. When it isn't (the `aria-*` case above), use `[attr.name]`.

Bringing it together: an "Add task" input

Let's add a small form that appends a task to the list. Edit `task-list.ts`:


```

import { Component } from '@angular/core';
import { Task } from '../task';

@Component({
  selector: 'app-task-list',
  imports: [],
  templateUrl: './task-list.html',
  styleUrls: ['./task-list.css'],
})
export class TaskList {
  tasks: Task[] = [ /* ... existing three tasks ... */ ];

  draft = '';

  toggle(task: Task): void {
    task.done = !task.done;
  }

  add(): void {
    const title = this.draft.trim();
    if (title === '') return;
    this.tasks = [
      ...this.tasks,
      {
        id: crypto.randomUUID(),
        title,
        done: false,
        createdAt: new Date().toISOString(),
        dueDate: null,
        tags: [],
      },
    ];
    this.draft = '';
  }
}

```

Two details worth naming:

- `this.tasks = [...this.tasks, newTask]` creates a *new array* rather than mutating the old one. In signal-based Angular (Chapter 7), this pattern is required; here it is optional but a good habit to build.
- `crypto.randomUUID()` is a browser built-in that produces a fresh id. In real apps the id often comes from the server; for now this suffices.

And in `task-list.html`, add a form at the top:

```

<h1>Tasks</h1>

<form (submit)="add(); $event.preventDefault()">
  <input
    type="text"
    placeholder="What needs doing?"
    [value]="draft"
    (input)="draft = $any($event.target).value" />
  <button type="submit" [disabled]="draft.trim() === ''">Add</button>
</form>

<ul class="task-list">
  @for (task of tasks; track task.id) {
    <li class="task" [class.done]="task.done" (click)="toggle(task)">
      <span class="title">{{ task.title }}</span>
      @if (task.dueDate) {
        <span class="due">due {{ task.dueDate | date: 'shortDate' }}</span>
      }
    </li>
  } @empty {
    <li class="empty">Nothing to do. Nice.</li>
  }
</ul>

```

Save. Type a task name. Press Add or hit Enter. It appears at the end of the list, cleared from the input, and the button re-disables until you type again.

Two things to notice about the form:

- `(submit)="add(); $event.preventDefault()"` handles form submission and stops the browser from re-loading the page. `$event.preventDefault()` is the same call you would write in vanilla JavaScript.

- `(input)="draft = $any($event.target).value"` is *manual* two-way binding. It reads the input's `value` on every keystroke and copies it into `draft`. In Chapter 12 we will replace this with proper reactive forms, and in Chapter 8 you will meet `model()` — an even better version. For now, we do it by hand.

The `$any(...)` is a workaround: `$event.target` has type `EventTarget | null`, which does not know about a `.value` property. `$any` tells the template checker "trust me, don't fuss." In a component method with a properly typed handler, we would use an actual TypeScript cast; in a template expression, `$any` is the escape hatch.

What you should not do in templates

Templates should be declarative descriptions of what to render given some data. There are a handful of things that *work* in templates but that you should not do:

- **Long expressions.** Anything more complex than a small ternary or method call belongs in the component class. Templates full of `?:` chains and nested method calls become read-only artifacts.
- **Method calls that do real work.** Every binding is evaluated many times as the app runs. If a template says `{{ expensiveComputation(item) }}`, that call happens on every change detection. Use a computed signal (Chapter 7) or move the work into the class.

- **Mutations.** No template expression should modify state. Event *statements* (`(click)="..."`) can, but interpolations and property bindings must be pure reads.

Following these rules makes templates fast, predictable, and debuggable. Break them, and you will spend afternoons wondering why something re-renders forty times per second.

What comes next

Chapter 7 introduces signals — Angular's modern reactive state primitive. We will rewrite Compass's `tasks` field as a signal, and the app will behave the same but for entirely different reasons. Under the covers we will move from "Angular checks everything after every event" to "Angular updates exactly the parts of the screen whose signals changed."

Exercises

1. Add a "Clear completed" button above the list that removes every done task. Wire it up with an event binding; do not mutate the array, produce a new one with `filter`. The button should be disabled when there is nothing to clear.
2. Show a count under the list: "3 tasks, 1 done, 2 remaining." Use `@let` to compute the counts once, and interpolation to display them.
3. The clickable `<li>` is bad accessibility. Move the click handler onto an inner `<button>` element, and use CSS to

make the button visually match the row. Bonus: add a `(keydown.enter)` binding to the button so pressing Enter also toggles.

Chapter 7. Signals: reactive state that feels normal

The Compass code you have now works. Click a task, it toggles; add a task, it appears; clear the input, the button re-disables. It works because Angular runs *change detection* — a check of every binding in every visible component — after every DOM event. In small apps this is fine. In large apps it becomes a bottleneck, and it makes it hard to reason about *why* something re-rendered.

Signals are Angular's answer to both problems. A signal is a piece of reactive state that knows exactly who is looking at it. When a signal changes, only the parts of the screen that read that signal are marked to re-render — nothing else. Once your components read signals rather than plain fields, Angular can update the screen precisely rather than sweepingly. The mental model is *values that push*.

This chapter introduces signals, rewrites Compass's Chapter 6 code to use them, and covers when to reach for computed and effect. By the end you will have a mental model for reactive state that will guide every remaining chapter.

The problem signals solve

Imagine a Compass screen with a hundred tasks and three components on the page: the task list, a "remaining count" chip in the header, and a progress bar at the bottom. All three depend on the same data.

In the pre-signal model, the flow is: you click a task, its `done` flag flips, and Angular re-checks every binding on the page — a hundred `[class.done]`s, the header's `{{ remaining }}`, the progress bar's `[style.width]`, and everything else. Angular is fast enough that this is usually invisible, but the mechanism is: *when in doubt, re-check everything*.

With signals, the flow is: you click a task; the task's `done` signal changes; Angular knows exactly which template bindings read that signal (one `[class.done]`, one item in the `remaining` computation, one item in the `progress` computation) and updates just those. Twenty updates instead of ten thousand.

The performance win is real but not the reason to reach for signals. The reason is that *your code* becomes precise. "Where does this value come from?" always has an answer — a chain of signals and computations you can trace. "Why did this re-render?" always has an answer — a signal it reads changed.

Creating a signal

The simplest signal wraps a single value:

```
import { signal } from '@angular/core';  
  
const count = signal(0);
```

To read the signal, call it as a function:

```
console.log(count()); // 0
```


That is deliberate. Signals are functions, and reading is an act. Every read is tracked — Angular records who read the signal so it can notify them later.

To write, use `set` or `update`:

```
count.set(10);  
count.update(n => n + 1);
```

`set` replaces the value; `update` computes the next value from the current one. For most state, both are interchangeable. `update` is useful when the new value depends on the old — a counter, a toggle, an array append.

That's it. Read with `()`, write with `set` or `update`, and Angular does the rest.

Reading in templates

The same rule applies in templates: to read a signal, call it as a function.

```
<p>Count: {{ count() }}</p>
```

The parentheses are not optional. `{{ count }}` prints the function object itself (something like `[object Function]`). The compiler will not catch this — it looks like valid TypeScript — so getting the parentheses wrong is a common early mistake. If your interpolation prints `function(){...}` in the browser, you forgot the `()`.

Signals of arrays and objects

A signal can hold any value, including arrays and objects. But signals track *references*, not deep state. If you mutate an array in place, the signal has no idea anything changed and will not notify its readers.

```
const tasks = signal<Task[]>([]);

// Wrong: mutates in place, no notification
tasks().push(newTask);

// Right: produce a new array
tasks.update(current => [...current, newTask]);
```

This is the same "produce a new object" pattern we met in Chapter 3, now with a name. Immutable updates are how signals work. It sounds inconvenient the first time; it becomes second nature quickly, and it makes state changes visible and testable.

computed : derived values

A computed signal is a signal whose value is derived from other signals. It re-computes when its inputs change, and caches its result until they do.

```
import { computed } from '@angular/core';

const tasks = signal<Task[]>([]);
const remaining = computed(() => tasks().filter(t => !t.done).length);
const done = computed(() => tasks().length - remaining());
```

You read a computed the same way you read any signal: `remaining()`. You never write to a computed — its value is a function of its inputs and cannot be set directly.

Two things make `computed` powerful:

- **Automatic dependency tracking.** The framework watches which signals your function reads. If those signals change, the computed re-runs. If they do not, it doesn't. You never register dependencies by hand.
- **Lazy evaluation and caching.** A computed does not run until something reads it. Once it has run, its result is cached; subsequent reads return the cache. The cache is invalidated when any dependency changes. If your `progress` computed depends on `tasks`, and `tasks` doesn't change, `progress()` costs nothing to call fifty times.

effect : side effects at the edge

Some things need to happen *when signals change*, but do not produce a value. Saving to `localStorage`, updating the page title, logging analytics, calling a non-Angular library — these are *effects*.

```
import { effect } from '@angular/core';

class TaskList {
  tasks = signal<Task[]>([]);

  constructor() {
    effect(() => {
      const current = this.tasks();
      localStorage.setItem('compass:tasks', JSON.stringify(current));
    });
  }
}
```

The effect function runs once immediately (to establish dependencies) and again every time any signal it reads changes. Here it saves the current task list to `localStorage` on every change.

Rules of the road for effects:

- **Register effects in a DI context.** In a component, that means the constructor or a class field initializer. Outside those, use `runInInjectionContext`.
- **Be careful writing to signals from inside an effect.** Writing to a signal that the same effect *reads* creates a cycle: the write re-triggers the effect, which writes again. Angular will throw or warn on obvious cycles. Writing to *unrelated* signals is allowed and sometimes necessary, but reach for it sparingly — most of the time, a `computed` is what you actually want.
- **Reach for `computed` first.** If you find yourself writing an effect that reads signals and computes a value, that is a `computed`. Effects are for outputs to the outside world.

Effects are the closest thing signals have to a footgun. Used well, they replace lifecycle hooks and observable subscriptions. Used badly, they create spooky action at a distance. Chapter 14 revisits them when we build a signal-based state layer.

Rewriting Compass with signals

Time to convert `TaskList` from Chapter 6. Open `src/app/task-list/task-list.ts` and change it to:

```
import { Component, signal, computed } from '@angular/core';
import { Task } from '../task';

@Component({
  selector: 'app-task-list',
  imports: [],
  templateUrl: './task-list.html',
  styleUrls: ['./task-list.css'],
})
export class TaskList {
  tasks = signal<Task[]>([
    { id: 't1', title: 'Buy milk', done: false, createdAt: '2026-01-01' },
    { id: 't2', title: 'Write chapter 7', done: true, createdAt: '2026-01-01' },
    { id: 't3', title: 'Call the plumber', done: false, createdAt: '2026-01-01' },
  ]);

  draft = signal('');

  remaining = computed(() =>
    this.tasks().filter(t => !t.done).length
  );

  toggle(taskId: string): void {
    this.tasks.update(current =>
      current.map(t =>
        t.id === taskId ? { ...t, done: !t.done } : t
      )
    );
  }

  add(): void {
    const title = this.draft().trim();
    if (title === '') return;
    this.tasks.update(current => [
      ...current,
      {
        id: crypto.randomUUID(),

```

```
        title,
        done: false,
        createdAt: new Date().toISOString(),
        dueDate: null,
        tags: [],
    },
    ]);
    this.draft.set('');
}

onInput(event: Event): void {
    this.draft.set((event.target as HTMLInputElement).value);
}
}
```

And `task-list.html`:

```
<h1>Tasks – {{ remaining() }} left</h1>

<form (submit)="add(); $event.preventDefault()">
  <input
    type="text"
    placeholder="What needs doing?"
    [value]="draft()"
    (input)="onInput($event)" />
  <button type="submit" [disabled]="draft().trim() === ''">Add</button>
</form>

<ul class="task-list">
  @for (task of tasks(); track task.id) {
    <li class="task" [class.done]="task.done" (click)="toggle(task)">
      <span class="title">{{ task.title }}</span>
      @if (task.dueDate) {
        <span class="due">due {{ task.dueDate | date: 'shortDate' }}</span>
      }
    </li>
  } @empty {
    <li class="empty">Nothing to do. Nice.</li>
  }
</ul>
```

Save, refresh. Compass looks and behaves the same as before. Every important line of code, though, is different:

- `tasks` and `draft` are signals, read with `()`.
- `toggle` and `add` use `update` to produce a new array; nothing mutates in place.
- `remaining` is computed once, and re-computed only when `tasks` changes.
- The template calls signals: `remaining()`, `draft()`, `tasks()`.

Under the covers, Angular now knows exactly which parts of the template depend on which signals. If we made this component `changeDetection: ChangeDetectionStrategy.OnPush` — which Chapter 16 explains and which we will apply globally — Angular would refuse to re-check anything that could not possibly have changed. That is the performance model of a signal-driven app.

Signal inputs

Component *inputs* — values passed from a parent — have historically been declared with the `@Input()` decorator:

```
export class TaskRow {  
  @Input() task!: Task;  
}
```

Modern Angular offers a signal-based alternative: `input()` (and `input.required()`), which produces a signal.

```
import { Component, input } from '@angular/core';  
  
@Component({ /* ... */ })  
export class TaskRow {  
  task = input.required<Task>();  
  
  // In methods and computed, read it like any other signal:  
  isOverdue = computed(() => {  
    const t = this.task();  
    return t.dueDate !== null && !t.done && new Date(t.dueDate) <  
  });  
}
```

Two shapes are useful:

- `input<Type>()` — optional; the default is `undefined`, and the signal returns `Type | undefined` unless you supply a default: `input<number>(0)`.
- `input.required<Type>()` — required; the compiler enforces that every parent supplies a value.

Signal inputs are the modern default. They compose naturally with `computed` (as above) and with `effect`. When Chapter 8 refactors Compass into a tree of components, we will use `input()` throughout.

Model inputs: two-way binding, properly

`ngModel` is one way to get two-way binding on native inputs; another is `[(value)]` on your own components, powered by *model inputs*.

```
import { Component, model } from '@angular/core';

@Component({
  selector: 'app-quantity-picker',
  template: `
    <button (click)="value.set(value() - 1)">-</button>
    <span>{{ value() }}</span>
    <button (click)="value.set(value() + 1)">+</button>
  `,
})
export class QuantityPicker {
  value = model.required<number>();
}
```

A parent uses it with the `[(name)]` syntax:

```
<app-quantity-picker [(value)]="count" />
```

`model` gives you a writable signal that automatically propagates changes to the parent via a matched output. It is the modern replacement for the "input + output" pair that older Angular code used. Chapter 8 revisits model inputs alongside plain inputs.

When to use signals — and when not

The rule of thumb: **signals for state that lives inside your app; RxJS for streams from the outside world.**

- **Use signals for:** UI state (open/closed, selected id, form draft), derived state (filtered list, computed counts), reactive computations, and cross-cutting state you would previously have kept in a service (Chapter 14 covers signal stores).
- **Use RxJS for:** HTTP responses (which are naturally observables), DOM event streams that need debounce or `switchMap`, WebSocket connections, long-running data streams.
- **Use plain fields for:** values that never change over the life of a component, or that don't affect the template. If nothing reads it reactively, it does not need to be a signal.

Reaching for signals everywhere is a beginner mistake. Reaching for RxJS everywhere is an early-career-Angular mistake. Both cost sim-

plicity. Chapter 11 covers the interop between them (there is a `toSignal` and a `toObservable`, and both are two lines of code).

Change detection in a signal world

Change detection in Angular used to be *zone-based*: a library called Zone.js patched every async API in the browser (timers, Promises, XHR, event listeners) so that Angular could learn when any of them fired and re-check the app. It works, but it is a broad brush and it costs measurable CPU on every event.

Modern Angular is moving toward *zoneless* apps. In a zoneless app, Angular only re-checks when a signal changes. There is no ambient re-check on every `setTimeout`; there is no monkey-patched Promise. Bindings that read signals are precise; nothing else runs.

The default `ng new` still enables Zone.js for backward compatibility, but the trajectory is clear. Every design decision in this book — signals for state, `input()` over `@Input()`, `model()` for two-way — is on the zoneless path. If you follow this book's patterns, migrating to zoneless later will be a configuration change, not a rewrite.

What comes next

Chapter 8 breaks Compass's monolithic `TaskList` into a tree of components: a list, individual rows, and an add-task form. You will meet component composition properly — `input()`, `output()`, and content projection — and by the end you will have a Compass that is structured like a real Angular app, not a proof of concept.

Exercises

1. Add a computed signal `overdueCount` that counts tasks whose `dueDate` is in the past and are not `done`. Display it under the heading: "3 overdue." Only show the line when there is at least one overdue task.
2. Add an `effect` that logs to the console every time the tasks list changes, in the format `"tasks: 5 (2 done)"`. Then click around and read the console. When does the effect fire? Does it fire when you type in the input? Why or why not?
3. Add a "clear completed" button. It should be disabled when nothing is completed. Wire it up to update the `tasks` signal by filtering out done tasks. Use `update`, not `set`; do not mutate the array.

Chapter 8. Composing components

A real Angular app is not one big component; it is a tree of small ones. Each has a narrow job, and together they compose into a screen. Composition is what makes an app maintainable — you can rename, replace, or reuse any component without touching the rest — and it is what lets a codebase grow past a few thousand lines without becoming a swamp.

In this chapter we split Compass's `TaskList` component into three: a list, individual task rows, and an add-task form. Along the way we cover the tools of composition: `input()` for passing data down, `output()` for sending events up, `model()` for two-way binding, and content projection for slotting arbitrary content inside a container component.

By the end, Compass will look and behave exactly as it does now — but it will be structured like Angular apps that survive contact with real teams.

What composition looks like

Right now, `TaskList` owns everything: the array of tasks, the draft input state, the add and toggle logic, and the entire template that renders form and rows. That is fine for a hundred lines. It fails at ten thousand.

We are going to break it up:

- **TaskRow** — renders one task. It takes a `task` as input and emits an event when the user toggles it.
- **AddTaskForm** — renders the input and Add button. It emits an event when the user submits a new title.
- **TaskList** — owns the tasks array and coordinates the two. It becomes short and readable.

This "container / presentation" division is the most common way to structure Angular components. The container knows about state and orchestrates the parts; the presentational components know only about what they render and what events they emit. Presentational components tend to be small, pure, easy to test, and easy to reuse.

Passing data down: `input()`

We met `input()` briefly in Chapter 7. Here we use it in earnest.

Create `TaskRow` with the CLI:

```
ng generate component task-row
```

Edit `src/app/task-row/task-row.ts`:

```

import { Component, input, output, computed } from '@angular/core';
import { Task } from '../task';

@Component({
  selector: 'app-task-row',
  imports: [],
  templateUrl: './task-row.html',
  styleUrls: ['./task-row.css'],
})
export class TaskRow {
  task = input.required<Task>();
  toggled = output<string>();

  isOverdue = computed(() => {
    const t = this.task();
    return t.dueDate !== null && !t.done && new Date(t.dueDate) <
  });

  onToggle(): void {
    this.toggled.emit(this.task().id);
  }
}

```

Three things to notice:

- **task = input.required<Task>()** declares that this component takes a required input of type `Task`. If a parent forgets to pass one, the template compiler complains.
- **toggled = output<string>()** declares an event this component can emit. The type parameter is the payload type — we emit the task's id.
- **isOverdue is a computed signal** that depends on `task()`. Because `task` is itself a signal, every time the parent passes a new value, `isOverdue` recomputes.

The template, in `task-row.html` :

```
<li class="task" [class.done]="task().done" [class.overdue]="isOverdue()">
  <button class="toggle" (click)="onToggle()">
    <span class="title">{{ task().title }}</span>
    @if (task().dueDate) {
      <span class="due">due {{ task().dueDate | date: 'shortDate' }}</span>
    }
    @if (isOverdue()) {
      <span class="overdue-badge">overdue</span>
    }
  </button>
</li>
```

Notice `task().title` — you read a signal input just like any other signal, with `()`. The template repeats `task()` a few times; it is a cheap call (signals cache), and cleaner than assigning to a local first.

And `task-row.css` :

```

:host { display: block; }

.task { padding: 0; margin: 0; border-bottom: 1px solid #eee; }
.task.done .title { text-decoration: line-through; color: #999; }
.task.overdue .title { color: #c73a52; }

.toggle {
  width: 100%; text-align: left; background: none; border: 0;
  padding: 12px 14px; display: flex; gap: 12px; align-items: center;
  cursor: pointer; font: inherit; color: inherit;
}

.due { margin-left: auto; font-size: 0.85rem; color: #666; }
.overdue-badge {
  font-size: 0.75rem; padding: 2px 6px; border-radius: 4px;
  background: #ffe3e3; color: #c73a52; margin-left: 8px;
}

```

The row is now a self-contained thing you can drop anywhere.

Sending events up: `output()`

`output()` produces something with an `emit(value)` method. Calling

it fires the event on the component; the parent listens with the `(name)="handler($event)"` syntax.

We already wrote `this.toggled.emit(this.task().id)` inside `TaskRow`. A parent will subscribe with:

```
<app-task-row [task]="t" (toggled)="toggle($event)" />
```

`$event` in the handler is whatever was passed to `emit(...)` — here, the task id string. TypeScript flows the type through, so the parent's `toggle` method will get a typed `string`.

`output()` replaces the older `@Output() EventEmitter = new EventEmitter<T>()` pattern. Both still work; new code uses `output()`.

Adding the form: `AddTaskForm`

Create it:

```
ng generate component add-task-form
```

Edit `src/app/add-task-form/add-task-form.ts`:

```

import { Component, signal, output } from '@angular/core';

@Component({
  selector: 'app-add-task-form',
  imports: [],
  templateUrl: './add-task-form.html',
  styleUrls: ['./add-task-form.css'],
})
export class AddTaskForm {
  draft = signal('');
  add = output<string>();

  submit(): void {
    const title = this.draft().trim();
    if (title === '') return;
    this.add.emit(title);
    this.draft.set('');
  }

  onInput(event: Event): void {
    this.draft.set((event.target as HTMLInputElement).value);
  }
}

```

Template `add-task-form.html` :

```

<form (submit)="submit(); $event.preventDefault()">
  <input
    type="text"
    placeholder="What needs doing?"
    [value]="draft()"
    (input)="onInput($event)" />
  <button type="submit" [disabled]="draft().trim() === ''">Add</button>
</form>

```

Style, `add-task-form.css` :

```

:host { display: block; margin-bottom: 1em; }
form { display: flex; gap: 8px; }
input { flex: 1; padding: 10px; font: inherit; border: 1px solid #ccc; }
button {
  padding: 10px 16px; font: inherit; border: 0; border-radius: 6px;
  background: #7a3fbf; color: white; cursor: pointer;
}
button:disabled { background: #ccc; cursor: not-allowed; }

```

`AddTaskForm` owns its own draft state (the user's in-progress typing). It emits an `add` event with the final title when the user submits. The parent does not need to know or care how the form works — it plugs into two contracts: the `add` output, and the tag `<app-add-task-form>`.

The refactored `TaskList`

Now `TaskList` becomes short. Replace its class with:

```

import { Component, signal, computed } from '@angular/core';
import { Task } from '../task';
import { TaskRow } from '../task-row/task-row';
import { AddTaskForm } from '../add-task-form/add-task-form';

@Component({
  selector: 'app-task-list',
  imports: [TaskRow, AddTaskForm],
  templateUrl: './task-list.html',
  styleUrls: ['./task-list.css'],
})
export class TaskList {
  tasks = signal<Task[]>([
    { id: 't1', title: 'Buy milk', done: false, createdAt: '2026-01-01', dueDate: null, tags: [] },
    { id: 't2', title: 'Write chapter 8', done: true, createdAt: '2026-01-01', dueDate: null, tags: [] },
    { id: 't3', title: 'Call the plumber', done: false, createdAt: '2026-01-01', dueDate: null, tags: [] },
  ]);

  remaining = computed(() => this.tasks().filter(t => !t.done).length);

  onAdd(title: string): void {
    this.tasks.update(current => [
      ...current,
      {
        id: crypto.randomUUID(),
        title,
        done: false,
        createdAt: new Date().toISOString(),
        dueDate: null,
        tags: [],
      },
    ]);
  }

  onToggle(id: string): void {
    this.tasks.update(current =>
      current.map(t => (t.id === id ? { ...t, done: !t.done } : t)
    );
  }
}

```

```
);
}
}
```

And the template `task-list.html`:

```
<h1>Tasks – {{ remaining() }} left</h1>

<app-add-task-form (add)="onAdd($event)" />

<ul class="task-list">
  @for (task of tasks(); track task.id) {
    <app-task-row [task]="task" (toggled)="onToggle($event)" />
  } @empty {
    <li class="empty">Nothing to do. Nice.</li>
  }
</ul>
```

Save. The app looks identical. Every piece of behavior is unchanged. But now:

- `TaskRow` can be reused wherever a task needs to render.
- `AddTaskForm` can be reused wherever you need to gather a title.
- `TaskList` is fifteen lines of orchestration.

This is the shape of an Angular app.

Two-way with `model()`

`model()` gives you an input-output pair with `[(name)]` binding, packaged as one signal. It is useful when a child owns *some* of a

shared piece of state — the classic case is a text input like the one we just built.

Suppose we wanted `AddTaskForm` to expose its `draft` to the parent for external display. We could do it by hand with an output; `model()` does it in one line.

Rewrite `add-task-form.ts`:

```
import { Component, model, output } from '@angular/core';

@Component({ /* ... */ })
export class AddTaskForm {
  draft = model('');
  add = output<string>();

  submit(): void {
    const title = this.draft().trim();
    if (title === '') return;
    this.add.emit(title);
    this.draft.set('');
  }

  onInput(event: Event): void {
    this.draft.set((event.target as HTMLInputElement).value);
  }
}
```

Now a parent can do:

```
<app-add-task-form [(draft)]="currentDraft" (add)="onAdd($event)"
```

Any change from either side is reflected in the other. If the parent doesn't care about the draft, it can just pass an `(add)` handler and

`skip [(draft)] . model()` gives you optionality that a hand-rolled input/output pair does not.

For now, `TaskList` doesn't need to see the draft — the current template omits `[(draft)]` and everything works. Consider `model()` when you find yourself writing paired inputs and outputs whose names would just add "Change" to each other.

Content projection: `<ng-content>`

Some components are not defined by the data they receive, but by the *content they wrap*. A modal dialog, a card, a collapsible section — each is a container whose interior is filled in by its caller. Angular's tool for this is *content projection*, via the `<ng-content>` element.

Here is a `Card` component:

```

import { Component, input } from '@angular/core';

@Component({
  selector: 'app-card',
  imports: [],
  template: `
    <div class="card">
      <h2 class="card-title">{{ title() }}</h2>
      <div class="card-body">
        <ng-content></ng-content>
      </div>
    </div>
  `,
  styles: `
    .card { border: 1px solid #ddd; border-radius: 8px; padding: 10px; }
    .card-title { margin: 0 0 8px 0; font-size: 1.1rem; }
  `,
})
export class Card {
  title = input.required<string>();
}

```

A parent uses it like this:

```

<app-card title="Today">
  <p>Three tasks remaining.</p>
  <button>Refresh</button>
</app-card>

```

Whatever appears between `<app-card>` and `</app-card>` in the parent's template ends up wherever `<ng-content>` sits in the child's template. This lets one component define layout while callers fill in content.

Named slots

If a container has multiple insertion points, use the `select` attribute:

```
<!-- inside SplitCard -->
<div class="left">
  <ng-content select="[left]"></ng-content>
</div>
<div class="right">
  <ng-content select="[right]"></ng-content>
</div>
<div class="footer">
  <ng-content select="footer"></ng-content>
</div>
```

The parent tags each piece of content to route it to a slot:

```
<app-split-card>
  <p left>Left column content.</p>
  <p right>Right column content.</p>
  <footer>Footer content.</footer>
</app-split-card>
```

Content that doesn't match any `select` goes to a plain `<ng-content>` if there is one, or is dropped otherwise.

Content projection is how you build design systems: cards, dialogs, tabs, table wrappers. Compass will use it in Chapter 15 when we introduce a shared "empty state" component.

Accessing children: `viewChild` and `viewChildren`

Occasionally a parent needs to reach *into* a child — to focus an input, to trigger a method, to read a piece of state that isn't exposed via an output. Angular offers `viewChild()` and `viewChildren()` for this.

```
import { Component, viewChild, ElementRef, AfterViewInit } from '@angular/core';

@Component({
  selector: 'app-search',
  template: `<input #searchInput type="text" />`,
})
export class Search implements AfterViewInit {
  input = viewChild<ElementRef<HTMLInputElement>>('searchInput');

  ngAfterViewInit() {
    this.input()?.nativeElement.focus();
  }
}
```

`viewChild('searchInput')` returns a signal whose value is whatever element or component matched the template reference `#searchInput`. You read it with `()` like any signal; it may be `undefined` if the element hasn't rendered yet.

`viewChild` is a last resort. If you find yourself reaching for it, first ask: could this be done with an input, an output, or a signal? Nine times out of ten, it can. The one-in-ten case usually involves imperative browser APIs (focus, scroll, `<video>` playback).

Guidelines for splitting components

When to break one component into two? Signs:

- The template is longer than a screen. If you scroll to read it, you'll have trouble maintaining it.
- Two visually distinct sections have nothing to do with each other. A "header" and a "list" in the same component should almost always be separate components.
- You are about to copy-paste a chunk of markup for a second use. Extract it first.
- The class has more than about ten fields. Some of them are probably about a sub-concern.

When *not* to split? When the parts are meaningless on their own. A "task row title" is not a component; it is a `<span>`. A "task row that also has a delete button and an overdue badge" is a component. The test: could you imagine using this component somewhere else, or testing it in isolation? If yes, extract. If no, leave it.

What comes next

Part III begins with Chapter 9 on *dependency injection* — the mechanism that will let us pull the task data out of `TaskList` and into a service that other components can share. Chapter 10 hooks that service up to an HTTP backend; Chapter 11 introduces RxJS for the parts of the app where signals are not enough; Chapter 12 replaces our hand-rolled form with reactive forms.

By the end of Part III, Compass will talk to a real server, save the tasks you add, and stop losing your work when you refresh.

Exercises

1. Extract a `TaskCount` component that displays "3 tasks left" and takes `total` and `done` as inputs. Compute the "left" number inside the component with a `computed`. Use it in `TaskList` in place of the current heading text.
2. Add a delete button to `TaskRow`. It should emit a `deleted` output with the task id. Wire up `TaskList` to remove tasks on this event. Do not let clicking the delete button also fire the toggle; use `$event.stopPropagation()` in the click handler.
3. Build a `Panel` component with content projection: a titled container with a body slot and an optional actions slot. Its template should be roughly `<div class="panel"> <header>{{ title() }} <ng-content select="[actions]" /></header> <div class="body"><ng-content /></div> </div>`. Use it to wrap the task list — pass "Today" as the title and a "Clear completed" button in the actions slot.

Chapter 9. Dependency injection, Angular's superpower

We have hit a limit. Compass keeps its tasks inside `TaskList`. That means the tasks exist only while `TaskList` is on screen; when we start adding a *stats page*, a *sidebar count*, or *routing* between screens, the state will disappear on every navigation. We need to move the tasks out of the component and into something the whole app can share.

That "something" is a *service*. Services in Angular are ordinary TypeScript classes with one special property: they can be injected. A component asks for a service, and Angular provides an instance — the same instance to every asker in the same scope — without the component knowing how the service was created or who else uses it. This mechanism is called *dependency injection* (DI), and it is one of the reasons Angular apps stay maintainable at scale.

This chapter introduces DI, refactors Compass's task state into a `TaskStore` service, and gives you the vocabulary you need for every subsequent chapter. Every part of Angular you meet from now on — HTTP, router, forms, tests, guards, resolvers — arrives at your component through DI.

Why "injection"?

Imagine `TaskList` needing to talk to a server. Without DI, it would look like:

```
class TaskList {  
    taskService = new TaskService(new HttpClient(new HttpBackend(new
```

Every component that uses `TaskService` would have to construct the whole chain. Worse, in tests you would have to reconstruct the chain with mocks. It gets baroque fast.

DI inverts the arrangement. The framework maintains a registry (called an *injector*) of services. When your component says "I need a `TaskService`," Angular looks in the registry, constructs one if needed (using its own DI to satisfy the service's dependencies), caches it, and hands it back. Your component gets the finished product without knowing what went into it.

This has three consequences that matter to us:

- **Components stay small.** They don't build the world; they ask for what they need.
- **Instances are shared appropriately.** By default, a service registered at the root has one instance shared across the entire app. Two components asking for `TaskService` receive the *same* `TaskService`, and any state on it is visible to both.

- **Tests get easy.** In tests you can register a fake `TaskService` before creating the component; the component doesn't know or care.

Creating a service

The Angular CLI has a generator:

```
ng generate service task-store
```

That produces `src/app/task-store.ts` (or `task-store.service.ts` in older CLI versions) roughly like:

```
import { Injectable } from '@angular/core';

@Injectable({ providedIn: 'root' })
export class TaskStore {}
```

Two ingredients:

- **@Injectable** is a decorator that marks the class as available for DI. It also lets the class receive dependencies via DI itself.
- **providedIn: 'root'** tells Angular to register the service at the root injector — the top level of the app. That means one shared instance for the entire application. Every component that injects `TaskStore` gets the same instance.

For most services, `providedIn: 'root'` is exactly what you want. It is the modern default, it "tree-shakes" (the service is dropped from

the bundle if nothing uses it), and it has no downsides for the vast majority of cases. Reach for the alternatives (Chapter 13 covers a route-scoped variant) only when you have a specific reason.

Moving Compass's tasks into `TaskStore`

Edit `src/app/task-store.ts` :

```
import { Injectable, signal, computed } from '@angular/core';
import { Task } from './task';

@Injectable({ providedIn: 'root' })
export class TaskStore {
  private readonly _tasks = signal<Task[]>([
    { id: 't1', title: 'Buy milk', done: false, createdAt: '2026-01-01' },
    { id: 't2', title: 'Write chapter 9', done: true, createdAt: '2025-12-31' },
    { id: 't3', title: 'Call the plumber', done: false, createdAt: '2025-12-30' }
  ]);

  readonly tasks = this._tasks.asReadOnly();
  readonly remaining = computed(() => this._tasks().filter(t => !t.done));
  readonly done = computed(() => this._tasks().filter(t => t.done));

  add(title: string): void {
    this._tasks.update(current => [
      ...current,
      {
        id: crypto.randomUUID(),
        title,
        done: false,
        createdAt: new Date().toISOString(),
        dueDate: null,
        tags: [],
      },
    ]);
  }

  toggle(id: string): void {
    this._tasks.update(current =>
      current.map(t => (t.id === id ? { ...t, done: !t.done } : t)
    );
  }

  remove(id: string): void {
    this._tasks.update(current => current.filter(t => t.id !== id));
  }
}
```

```
}  
  
clearCompleted(): void {  
  this._tasks.update(current => current.filter(t => !t.done));  
}  
}
```

Three patterns worth naming:

- **Private writable signal, public readonly view.** `_tasks` is private and mutable; `tasks` is the public, readonly signal that consumers see. This prevents callers from doing `this.store.tasks.set(...)` and bypassing the API.
- **Behavior over data.** The service exposes methods that describe *intent* — add, toggle, remove, `clearCompleted`. Consumers do not know how tasks are stored, only what they can do with them.
- **Computed derived state.** `remaining` and `done` live here, not in components. Any component can read them; when tasks change, all readers update.

Injecting the service into a component

Two forms of injection work in modern Angular. The `inject()` function is the newer, preferred form; constructor injection is the older, still-supported form. We use `inject()` throughout the book.

Edit `src/app/task-list/task-list.ts`:

```
import { Component, inject } from '@angular/core';
import { TaskStore } from '../task-store';
import { TaskRow } from '../task-row/task-row';
import { AddTaskForm } from '../add-task-form/add-task-form';

@Component({
  selector: 'app-task-list',
  imports: [TaskRow, AddTaskForm],
  templateUrl: './task-list.html',
  styleUrls: ['./task-list.css'],
})
export class TaskList {
  private store = inject(TaskStore);

  tasks = this.store.tasks;
  remaining = this.store.remaining;

  onAdd(title: string): void {
    this.store.add(title);
  }

  onToggle(id: string): void {
    this.store.toggle(id);
  }
}
```

The component is now essentially a translator between the DOM and the service. All of its state and logic have moved to `TaskStore`. If we

want the same data in a different component — a sidebar showing `remaining`, say — we just inject `TaskStore` there too and read `store.remaining`. Nothing to plumb through the tree.

The template does not need to change; `tasks` and `remaining` are still signals with the same interface.

Save. The app behaves identically. The tasks now live outside the component, and other components could share them if they existed.

The `inject()` function

`inject(Class)` returns an instance of `Class` from the current injector. You can call it:

- In a component's field initializer or constructor (as above).
- In a service's field initializer or constructor.
- Inside functions called from those contexts, transitively.

You cannot call `inject()` from a random function that isn't in a DI context. Angular throws a runtime error. If you need to inject outside a natural context, wrap the call in `runInInjectionContext(injector, () => { ... })`, but this is rare in application code — you will encounter it maybe once a year.

`inject()` also has generic forms for optionality and multi-token cases:

```
private optionalThing = inject(TokenClass, { optional: true });  
private allOfThem = inject(TokenClass, { self: true });
```

We will not need these for a while. Plain `inject(Something)` covers 95% of uses.

Providers: how services get into the injector

`providedIn: 'root'` is one way to register a service. There are others, and you will meet them one at a time.

App-level providers via `provideX(...)`

Some services are configured with data; you cannot just register them with a decorator. Angular's convention is a `provide<Feature>(config)` function you place in `providers` in `app.config.ts`:

```
export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(routes),
    provideHttpClient(),
    provideAnimations(),
  ],
};
```

`provideRouter`, `provideHttpClient`, and their friends are how Angular's opt-in features arrive in your app. Each function returns a set of provider records; Angular reads them at bootstrap.

Component-level providers

A component can register a service scoped to itself and its children:

```
@Component({
  selector: 'app-editor',
  providers: [EditorState],
})
export class Editor { }
```

Now every `EditorState` reference *inside* `Editor` (in `Editor` itself and any child component) resolves to the same instance; a `EditorState` reference *outside* `Editor` resolves to a different instance (or fails if there is no root provider).

This is useful for state that is local to a screen: a wizard's step state, a modal's form draft, a page's filter selections. We will use it in Chapter 13.

Value and factory providers

Sometimes what you want to inject is not a class, but a value or a function's return. Angular has syntax for these too:

```
providers: [  
  { provide: 'API_BASE_URL', useValue: 'https://api.compass.example' },  
  { provide: TaskLoader, useFactory: () => new TaskLoader(navigate) },  
]
```

An `InjectionToken<T>` is the type-safe way to identify a non-class provider:

```
import { InjectionToken } from '@angular/core';  
  
export const API_BASE_URL = new InjectionToken<string>('API_BASE_URL');  
  
// providers:  
{ provide: API_BASE_URL, useValue: 'https://api.compass.example' },  
  
// injection:  
private apiUrl = inject(API_BASE_URL);
```


You will see `InjectionToken` in Angular's own code and in libraries. In application code, plain classes with `providedIn: 'root'` cover most needs.

Hierarchical injectors

Angular's injectors form a tree that follows the component tree. When you `inject(X)` in a component, Angular walks *up* from that component's injector looking for a provider for `X`. It uses the first one it finds.

That is why component-level `providers: [EditorState]` gives every descendant a shared instance, while the same code in a sibling component's `providers` array gives that sibling a different instance. The lookup finds the nearest provider.

You will rarely think about this explicitly. It matters when:

- You have a service you want scoped to a specific feature area (a wizard, a modal), not the whole app.
- You need to override a service in tests (Chapter 17).
- You want two instances of the same service class for two different purposes.

For everything else, `providedIn: 'root'` and forget about it.

Circular dependencies

Two services that inject each other create a cycle. Angular refuses to construct them and throws at runtime. If you hit this, it usually means

the services have a design problem: they should be merged, or one should hold the other but not vice versa, or the shared state should be extracted into a third service they both depend on.

There is a workaround — inject `Injector` and look up lazily — but you should not need it. When you feel the pull toward it, take that as a signal to redesign.

Testing benefits, previewed

Because everything a component uses comes in through DI, tests can *replace* those dependencies:

```
TestBed.configureTestingModule({
  providers: [
    { provide: TaskStore, useValue: fakeStore },
  ],
});
```

The component's `inject(TaskStore)` now returns `fakeStore`. No monkey-patching, no imports to redirect. Chapter 17 unpacks this in detail; for now, know that the reason your Compass tests will be easy later is because you designed it around DI now.

What comes next

Chapter 10 hooks `TaskStore` up to a real HTTP backend. We will introduce Angular's `HttpClient`, write functions to fetch, create, update, and delete tasks, and wire them into the store. By the end of

Chapter 10, refreshing the browser will preserve your tasks — because they live on a server, not in memory.

Exercises

1. Create a second component `TaskCountBadge` that shows just the remaining count as a small pill. Inject `TaskStore` inside it and render `{{ store.remaining() }}`. Place it near the heading in `TaskList`. Notice that no data flows through props — the two components share state via the service.
2. Add a `HabitStore` service scaffolded the same way as `TaskStore`.
It should hold a signal of habits (make up an interface `Habit`), expose read-only `habits`, and provide `add`, `remove`, and `check`. Don't wire it into any component yet; just make sure it compiles.
3. Register `TaskStore` at the component level instead of `providedIn: 'root'`. What breaks? Reset it, and think about why: what property of shared state depends on where the service is registered?

Chapter 10. HTTP: talking to real APIs

Compass has been running entirely in the browser's memory. That is enough to demo the app, and not nearly enough for real use. In this chapter we introduce Angular's `HttpClient` service, connect Compass to a small backend, and make it possible to add a task on one device and see it on another. By the end, the tasks you create will survive a browser refresh — because they live on a server, not in `TaskStore`'s in-memory signal.

The chapter has two halves. The first half spins up a minimal backend so we have something to talk to. The second half wires `HttpClient` into `TaskStore` and covers interceptors, error handling, and the modern `httpResource`.

A tiny backend for Compass

We need a server. To keep this book self-contained, we will use `json-server` — a zero-config REST server that reads a JSON file. It exposes GET/POST/PUT/PATCH/DELETE endpoints automatically. In production Compass would talk to a real backend (Node, Python, Go, whatever); the Angular side of the code is identical.

Install it globally:

```
npm install --global json-server
```

Create a file `compass/db.json` (at the top level of your Angular project, next to `package.json`):

```
{
  "tasks": [
    { "id": "t1", "title": "Buy milk", "done": false, "createdAt": "2016-01-01T12:00:00Z" },
    { "id": "t2", "title": "Write chapter 10", "done": true, "createdAt": "2016-01-01T12:00:00Z" }
  ]
}
```

Run it in a second terminal:

```
json-server --watch db.json --port 3000
```

Now `http://localhost:3000/tasks` returns the array of tasks. Curl it, or open the URL in a browser, to confirm.

`json-server` is a fake server, and its persistence is trivial — it writes changes back to `db.json` — but its API matches every JSON REST server you will ever call. That is the point.

Configuring CORS is not our problem

Because we're running Compass on `http://localhost:4200` and the API on `http://localhost:3000`, browsers would normally block the requests as cross-origin. `json-server` allows them by default; a real production API would send `Access-Control-Allow-Origin` headers. In your own future backends, remember this: the browser is strict about cross-origin requests, and CORS misconfiguration is the reason things "just don't work" ninety percent of the time when hooking a new frontend to a new backend.

Enabling HttpClient in Angular

`HttpClient` is Angular's built-in HTTP service. It arrives via DI, and it needs to be provided at the app level. Edit `src/app/app.config.ts`:

```
import { ApplicationConfig, provideBrowserGlobalErrorListeners, provideZonelessChangeDetection } from '@angular/core';
import { provideRouter } from '@angular/router';
import { provideHttpClient, withFetch } from '@angular/common/http';
import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    provideZonelessChangeDetection(),
    provideRouter(routes),
    provideHttpClient(withFetch()),
  ],
};
```

`provideHttpClient()` registers the service. `withFetch()` tells Angular to use the browser's native `fetch` API under the hood; the default older XHR-based implementation still works, but `fetch` is the direction of travel and integrates better with SSR (Chapter 18).

The API contract

Define a small module that describes the URLs and does the raw HTTP. Create `src/app/tasks-api.ts`:

```

import { Injectable, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { firstValueFrom } from 'rxjs';
import { Task } from './task';

const BASE_URL = 'http://localhost:3000';

@Injectable({ providedIn: 'root' })
export class TasksApi {
  private http = inject(HttpClient);

  list(): Promise<Task[]> {
    return firstValueFrom(this.http.get<Task[]>(`${BASE_URL}/tasks`));
  }

  create(task: Omit<Task, 'id'>): Promise<Task> {
    return firstValueFrom(this.http.post<Task>(`${BASE_URL}/tasks`, task));
  }

  update(id: string, changes: Partial<Task>): Promise<Task> {
    return firstValueFrom(this.http.patch<Task>(`${BASE_URL}/tasks/${id}`, changes));
  }

  remove(id: string): Promise<void> {
    return firstValueFrom(this.http.delete<void>(`${BASE_URL}/tasks/${id}`));
  }
}

```

Three details you will meet again:

- **Type parameters on the HTTP methods.**

`this.http.get<Task[]>(url)` tells the compiler the response body is an array of `Task`. Angular does not validate that the server actually returned that shape; it is a

contract you assert. In Chapter 17 we will add tests that check the server's shape matches this assertion.

- **firstValueFrom** from **RxJS**. **HttpClient** methods return *Observables*, not Promises. An Observable is a stream of values over time; **firstValueFrom** awaits the first one and returns a Promise. For request/response HTTP, this is what you want. Chapter 11 unpacks Observables properly.
- **Omit<Task, 'id'>** and **Partial<Task>**. These are TypeScript utility types: **Omit<T, 'K'>** is **T** without field **K**; **Partial<T>** is **T** with every field optional. Both are common in API method signatures.

Wiring HTTP into **TaskStore**

Now update **TaskStore** to use the API. The store still exposes the same reactive signals; it just fetches and syncs to the server.

Edit `src/app/task-store.ts` :


```

import { Injectable, signal, computed, inject } from '@angular/core';
import { Task } from './task';
import { TasksApi } from './tasks-api';

@Injectable({ providedIn: 'root' })
export class TaskStore {
  private api = inject(TasksApi);
  private readonly _tasks = signal<Task[]>([]);
  private readonly _loading = signal(false);
  private readonly _error = signal<string | null>(null);

  readonly tasks = this._tasks.asReadonly();
  readonly loading = this._loading.asReadonly();
  readonly error = this._error.asReadonly();
  readonly remaining = computed(() => this._tasks().filter(t => !t.done));

  async load(): Promise<void> {
    this._loading.set(true);
    this._error.set(null);
    try {
      const tasks = await this.api.list();
      this._tasks.set(tasks);
    } catch (err) {
      this._error.set('Could not load tasks. Try again in a moment');
    } finally {
      this._loading.set(false);
    }
  }

  async add(title: string): Promise<void> {
    const draft: Omit<Task, 'id'> = {
      title,
      done: false,
      createdAt: new Date().toISOString(),
      dueDate: null,
      tags: [],
    };
  }

```

```

// Optimistic add: put a temporary row in the UI immediately.
const tempId = `temp-${crypto.randomUUID()}`;
const optimistic: Task = { ...draft, id: tempId };
this._tasks.update(current => [...current, optimistic]);
try {
  const saved = await this.api.create(draft);
  this._tasks.update(current =>
    current.map(t => (t.id === tempId ? saved : t))
  );
} catch {
  this._tasks.update(current => current.filter(t => t.id !== tempId));
  this._error.set('Could not save the new task.');
```

```

}
```

```

async toggle(id: string): Promise<void> {
  const current = this._tasks().find(t => t.id === id);
  if (!current) return;
  const next: Task = { ...current, done: !current.done };
  this._tasks.update(all => all.map(t => (t.id === id ? next : t)));
  try {
    await this.api.update(id, { done: next.done });
  } catch {
    // Revert on failure
    this._tasks.update(all => all.map(t => (t.id === id ? current : t)));
    this._error.set('Could not save that change.');
```

```

}
```

```

async remove(id: string): Promise<void> {
  const previous = this._tasks();
  this._tasks.update(all => all.filter(t => t.id !== id));
  try {
    await this.api.remove(id);
  } catch {
    this._tasks.set(previous);
    this._error.set('Could not delete the task.');
```

```

    }
  }
}

```

The store now:

- **Loads asynchronously.** On `load()`, it fetches from the server, updates its signals, and exposes `loading` and `error` signals for the UI.
- **Applies changes optimistically.** `add`, `toggle`, and `remove` update the UI *before* the server responds; if the server fails, the store reverts. This is what makes web apps feel snappy.
- **Exposes only signals to the UI.** The UI never awaits anything; it reads reactive state.

Calling `load()` on startup

`TaskStore` has a `load()` method, but nothing calls it. Add a call from `TaskList`'s constructor:

```

export class TaskList {
  private store = inject(TaskStore);
  tasks = this.store.tasks;
  remaining = this.store.remaining;
  loading = this.store.loading;
  error = this.store.error;

  constructor() {
    this.store.load();
  }

  onAdd(title: string): void { this.store.add(title); }
  onToggle(id: string): void { this.store.toggle(id); }
}

```

Update the template to show loading and error state:

```

<h1>Tasks - {{ remaining() }} left</h1>

@if (loading()) {
  <p class="status">Loading...</p>
}
@if (error()) {
  <p class="status error">{{ error() }}</p>
}

<app-add-task-form (add)="onAdd($event)" />

<ul class="task-list">
  @for (task of tasks(); track task.id) {
    <app-task-row [task]="task" (toggled)="onToggle($event)" />
  } @empty {
    <li class="empty">Nothing to do. Nice.</li>
  }
</ul>

```

Save. Refresh. You should see a brief "Loading..." flash and then your two tasks from `db.json`. Add a task; open `db.json` in your editor to confirm the file got a third entry. Toggle a task; the change should persist across refreshes.

Interceptors: cross-cutting request behavior

Suppose every request needs an `Authorization` header, or every response needs to be logged, or every 401 needs to redirect to a login page. You do not want to put that code in every method of `TasksApi`. Angular's answer is *interceptors*.

An interceptor is a function that runs on every HTTP request, can modify the request, and can modify the response.

```
// src/app/auth-interceptor.ts
import { HttpInterceptorFn } from '@angular/common/http';

export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const token = localStorage.getItem('compass:token');
  if (token) {
    req = req.clone({ setHeaders: { Authorization: `Bearer ${token}` } });
  }
  return next(req);
};
```

Register it in `app.config.ts`:

```
import { provideHttpClient, withFetch, withInterceptors } from '@angular/common/http';
import { authInterceptor } from './auth-interceptor';

// ...providers:
provideHttpClient(withFetch(), withInterceptors([authInterceptor])
```

Now every request sent by `HttpClient` picks up the header, without `TasksApi` having to think about it.

Interceptors also make error handling uniform. A `logout` on `401` interceptor:

```
import { HttpInterceptorFn } from '@angular/common/http';
import { catchError, throwError } from 'rxjs';

export const authGuardInterceptor: HttpInterceptorFn = (req, next) => {
  return next(req).pipe(
    catchError(err => {
      if (err.status === 401) {
        localStorage.removeItem('compass:token');
        window.location.href = '/login';
      }
      return throwError(() => err);
    })
  );
};
```

We will use interceptors more in Part V; for now, know they exist and are the right hammer for "every request needs X."

httpResource : the modern signal-integrated fetch

The `firstValueFrom(this.http.get(...))` pattern is fine, but modern Angular offers something nicer for the common case of "fetch and hold this data." `httpResource<T>` creates a signal-backed HTTP resource that tracks loading, error, and value state for you.

```
import { httpResource } from '@angular/common/http';

@Injectable({ providedIn: 'root' })
export class TaskStore {
  private baseUrl = 'http://localhost:3000';

  taskList = httpResource<Task[]>(() => `${this.baseUrl}/tasks`);

  // In a component: taskList.value(), taskList.isLoading(), taskList.error()
}
```

`httpResource` returns a resource with `.value()`, `.isLoading()`, `.error()`, `.reload()`, and a few more. Its request URL is a function; when any signal read inside the function changes, it re-fetches. So `httpResource(() => `${baseUrl}/tasks?tag=${tag()})` re-fetches whenever `tag` changes.

For Compass we will keep our hand-rolled `TaskStore` because it also handles mutations, but for read-only data (statistics, dashboards, look-ups), `httpResource` cuts a lot of code. Chapter 14 revisits it when we add the stats page.

Errors, timeouts, retries

Every HTTP call can fail. `HttpClient` reports failures as errors on the Observable, which become rejections on the Promise, which become thrown exceptions on `await`. In the store above we caught those and revert the optimistic update.

For transient failures, retry logic can help. RxJS has a `retry` operator:

```
import { retry } from 'rxjs';

list(): Promise<Task[]> {
  return firstValueFrom(
    this.http.get<Task[]>(`${BASE_URL}/tasks`).pipe(retry({ count
  }));
}
```

This retries the request up to twice, with a half-second delay between attempts. Chapter 11 covers RxJS operators like this properly.

For timeouts, `HttpContext` and interceptors can cap request durations, or you can wrap the request in `Promise.race` with a timer. In Compass we will not add timeouts by hand; if a request hangs, the user's browser will surface the failure eventually, and our optimistic updates keep the UI responsive.

Cache and stale data

`HttpClient` does not cache. Every call goes to the network unless a browser or Service Worker layer catches it. For most application data,

this is what you want — you do not want stale data cached inside your JavaScript.

`httpResource` re-fetches when its request signal changes; it does not cache across pages either. If you need caching (a settings endpoint that never changes during a session, say), Chapter 14 shows a pattern that combines `httpResource` with a signal-based cache.

What comes next

Chapter 11 covers RxJS — the library beneath `HttpClient` and the way Angular represents streams that unfold over time. Then Chapter 12 replaces our hand-rolled "Add task" input with proper reactive forms, complete with validation and type safety.

Exercises

1. Add a "delete" button to `TaskRow` that emits a `deleted` event. Wire `TaskList` to call `store.remove(id)`. Verify that the deletion persists across refreshes.
2. Trigger a failure: stop `json-server`, then click a task. What does the UI do? Is the optimistic update reverted? Is the error message shown? Once you have verified, restart `json-server` and click again — everything should recover.
3. Rewrite `TasksApi.list()` to use `httpResource` instead of a plain `http.get`. What signature does the method have now? What does `TaskStore` need to change to consume it? (You can leave `TaskStore` using the prom-

ise-based version if you prefer; the point of the exercise is to see the difference.)

Chapter 11. RxJS essentials

You have seen RxJS twice already: `HttpClient` returned `Observables`, and we unwrapped them with `firstValueFrom`. That is enough for request-response HTTP, but not for the parts of Compass yet to come. Search-as-you-type, live streams, coordinated form updates, and route parameters — all of them are naturally streams of values over time, and RxJS is Angular's tool for streams.

This chapter is not a full tour of RxJS. That library has a hundred operators and a subculture of its own. What you need for Compass, and for most Angular application code, is a specific subset: what an `Observable` is, how it differs from a `Promise` and from a signal, a dozen operators, and the interop points between `Observables` and signals. That is what this chapter covers.

What an Observable is

An **Observable** is a producer of values over time. It might produce zero values (and then complete), one value (like an HTTP response), a fixed sequence (like the digits of pi), or an unbounded stream (like mouse events until the window closes). It might complete normally or error out. And it does nothing until someone *subscribes* to it — `Observables` are lazy.

Contrast with the two things you already know:

- **A Promise** produces one value at most. Once created, a Promise starts working immediately (it is eager) and eventually resolves or rejects. It cannot be cancelled and cannot deliver more than one value.
- **A signal** holds a current value at all times. Reading it always returns the value. Writing it notifies anyone tracking it.

An Observable is neither. It is a *plan* for producing values, activated by subscription, capable of producing many, and cancellable.

Concretely:

```
import { Observable } from 'rxjs';

const numbers = new Observable<number>(subscriber => {
  subscriber.next(1);
  subscriber.next(2);
  subscriber.next(3);
  subscriber.complete();
});

numbers.subscribe({
  next: n => console.log('got', n),
  complete: () => console.log('done'),
});
// Prints: got 1, got 2, got 3, done
```

You will almost never construct Observables with `new Observable`. In application code, they come from *creation functions* and from Angular APIs.

Creating Observables

The functions that turn something into an Observable:

```
import { of, from, fromEvent, interval, timer } from 'rxjs';

const three = of(1, 2, 3); // emits 1, 2, 3
const fromArray = from([1, 2, 3]); // emits array elements
const fromPromise = from(fetch('/api/tasks')); // wraps a Promise

const clicks = fromEvent<MouseEvent>(document, 'click'); // emits clicks
const ticks = interval(1000); // emits ticks
const delayed = timer(500); // emits after delay
```

Two of these — `fromEvent` and `interval` — produce unbounded streams. `subscribe` to them, and they emit forever, or until you unsubscribe.

Angular gives you Observables for many built-in things:

```
this.http.get<Task[]>('/api/tasks'); // one emission, then done
this.route.params; // emits on every route change
this.route.queryParams; // emits on every route change
formControl.valueChanges; // emits on every form change
```

Each of these is a well-defined stream, and each pairs naturally with a small pipeline of operators to produce the value you actually want.

Operators and `pipe`

An operator is a function that takes an Observable and returns a new Observable. `pipe` composes operators into a chain.

```
import { of, map, filter } from 'rxjs';

of(1, 2, 3, 4, 5)
  .pipe(
    filter(n => n % 2 === 0),
    map(n => n * 10)
  )
  .subscribe(n => console.log(n)); // 20, 40
```

There are dozens of operators. You will use maybe fifteen of them regularly. Here is that shortlist, roughly in order of how often you will reach for them.

map(fn) transforms each emission through **fn**. If **fn** returns a Promise or Observable, it is *not* awaited — for that, use **switchMap** or **forkJoin**.

filter(fn) drops emissions for which **fn** returns false.

tap(fn) runs **fn** on every emission for its side effects, without changing the stream. Useful for logging and debugging. **tap({ error })** also runs on errors.

take(n) takes the first **n** emissions, then completes. **take(1)** is common when you want an Observable to behave like a Promise.

takeUntil(otherObservable\$) completes the source stream when **otherObservable\$** emits. Angular has a specialized **takeUntilDestroyed()** that ties an Observable's lifetime to a component's — we'll use it below.

debounceTime(ms) waits **ms** after the last emission before passing it through. Perfect for search-as-you-type: the user pauses, the request goes.

`distinctUntilChanged()` suppresses consecutive duplicate values. Also very useful for search: don't refetch if the query hasn't actually changed.

`switchMap(fn)` flattens the source through an inner Observable, cancelling any in-flight inner Observable when a new one arrives. This is the operator that makes search-as-you-type work: each key-stroke triggers a new HTTP request, and the previous request is cancelled.

`mergeMap(fn)` flattens, but lets all inner Observables run concurrently. Rarely what you want for user-driven work; useful when you want parallel processing.

`exhaustMap(fn)` flattens, but ignores new source emissions while an inner Observable is still running. The right operator for "save" buttons: if the user clicks twice, don't fire two saves.

`concatMap(fn)` flattens, queueing inner Observables so they run one after another. Useful for ordered writes.

`catchError(fn)` intercepts errors and either returns a fallback Observable or re-throws. This is how you provide a default value on a failed HTTP call.

`retry({ count, delay })` re-subscribes on error. We used it in Chapter 10.

`combineLatest([a$, b$])` takes multiple Observables and emits an array of their latest values whenever any of them emits. Angular's route params + query params combination is a common use.

startWith(value) prepends a value to the stream, so subscribers get an immediate emission.

The rule of learning: don't try to memorize all of them. Learn `map`, `filter`, `tap`, `take`, `debounceTime`, `distinctUntilChanged`, and `switchMap`. Look up the rest when a problem calls for them.

A worked example: search-as-you-type

Suppose Compass gets a search box. The user types "buy m," and Compass fetches `/tasks?q=buy%20m`. If the user is still typing when a response arrives, the response should be thrown away — the query is stale.

```
import { fromEvent } from 'rxjs';
import { debounceTime, distinctUntilChanged, filter, map, switchMap } from 'rxjs/operators';

const input = document.querySelector('input')!;

fromEvent<Event>(input, 'input').pipe(
  map(e => (e.target as HTMLInputElement).value.trim()),
  debounceTime(200),
  distinctUntilChanged(),
  filter(q => q.length >= 2),
  switchMap(q => http.get<Task[]>(`/api/tasks?q=${encodeURIComponent(q)}`)),
).subscribe(results => {
  render(results);
});
```

Read it top to bottom:

1. Every input event, get the trimmed value.
2. Wait 200ms after the last keystroke.

3. Only proceed if the query changed.
4. Only proceed if there are at least two characters.
5. Fire the HTTP request. If a new keystroke arrives while this request is in flight, cancel it and start a new one.

That six-line pipeline replaces what would otherwise be a manually-managed timer, an in-flight request tracker, a debounce ref, and a comparison for change. This is what RxJS is for.

Signals and Observables: interop

Most of Compass's state lives in signals. Most of Angular's built-in APIs emit Observables. The interop is two functions:

toSignal(observable, options) turns an Observable into a signal.

```
import { toSignal } from '@angular/core/rxjs-interop';

class TaskList {
  private route = inject(ActivatedRoute);
  filter = toSignal(this.route.queryParamMap.pipe(
    map(params => params.get('tag') ?? 'all')
  ), { initialValue: 'all' });
}
```

Now **filter** is a signal you can read in the template as **filter()**, and it updates whenever the route's query params change. **toSignal** also cleans up the subscription when the component is destroyed.

toObservable(signal) turns a signal into an Observable.

```
import { toObservable } from '@angular/core/rxjs-interop';

class TaskStore {
  filter = signal('all');
  filter$ = toObservable(this.filter);

  filteredTasks$ = this.filter$.pipe(
    switchMap(f => this.api.list(f))
  );
}
```

`toObservable` emits every time the signal changes, and the emission is the signal's new value. Combined with operators, this lets you build "signal-driven fetch" pipelines without leaving reactive land.

Subscribing safely

Every subscription is a resource. If your component subscribes and never unsubscribes, the callback keeps running after the component leaves the screen — a leak. Angular gives you three good options and one bad one.

Best (usually): the `async` pipe. Bind an Observable directly in a template, and Angular subscribes and unsubscribes for you.

```
<p>{{ user$ | async }}</p>
```

Second best: `toSignal`. For non-template consumers of an Observable. Angular tears down the subscription on component destroy.

Third: `takeUntilDestroyed()`. For explicit subscriptions in a component context.

```
import { takeUntilDestroyed } from '@angular/core/rxjs-interop';

class TaskList {
  constructor() {
    this.route.params.pipe(takeUntilDestroyed()).subscribe(params)
    // ...
  };
}
}
```

`takeUntilDestroyed()` reads the current `DestroyRef` (from DI) and completes the Observable when the component is destroyed. No explicit `ngOnDestroy` needed.

Bad: bare `.subscribe()` without any teardown. This is the leak. If your linter is decent, it will yell at you.

Cold vs hot, briefly

An Observable is *cold* if each subscriber gets its own run of the producer, and *hot* if all subscribers share one run.

`http.get()` is cold: each subscription fires a new request. `fromEvent(document, 'click')` is *also* cold, though it fools people: each subscribe attaches its own `addEventListener`, so multiple subscribers get their own listeners rather than a shared stream. If you want DOM events shared across subscribers, wrap with `share()` or convert to a signal.

You will rarely have to think about this explicitly. Where it matters: if you `subscribe` to an `http.get` result twice, you fire two

requests. Use `shareReplay(1)` to share one request among many subscribers, or convert to a signal.

When not to use RxJS

RxJS is powerful. It is also overused. Signs you have overreached:

- Your components have five nested `pipe` calls to compute something a `computed` signal would express in one line.
- You have a `Subject` field on a service being used as a stateful bag. Use a signal.
- You are `.subscribe()`-ing to something and setting a field with the result. Use `toSignal`.
- You are debouncing a signal by piping through `toObservable` and `debounceTime`. Fine — but check whether a `linkedSignal` or a small effect is not clearer for your case.

The rule of thumb from Chapter 7 stands: signals for state, RxJS for streams. When you find yourself using RxJS for state, migrate; when you find yourself using signals for streams, ask why.

What comes next

Chapter 12 wraps up Part III with reactive forms. Compass currently has a hand-rolled "Add task" input. In Chapter 12 we replace it with `FormGroup`, add validation, and build an edit dialog for tasks with due dates and tags. After Chapter 12, Compass has a real backend,

real forms, and real reactive behavior — everything a small app needs before it grows into multiple pages, which is Part IV's job.

Exercises

1. Write an Observable pipeline that takes clicks on a "load more" button and, for each click, appends the next page of tasks. Use `exhaustMap` so multiple rapid clicks don't fire multiple requests. Verify by clicking quickly.
2. Convert the search-as-you-type pipeline in this chapter to a Compass component. Create a `SearchBox` with an input, wire the pipeline in its constructor, and emit an `results` output. Use `takeUntilDestroyed` on the subscription.
3. `HttpClient` returns cold Observables — every subscribe re-fires the request. Prove this: subscribe to the same `http.get(...)` Observable twice and check the network tab. Then add `shareReplay(1)` and confirm it fires once. Why is `shareReplay(1)` and not `shareReplay()` ?

Chapter 12. Forms: capturing user input reliably

Compass has one input right now: the "Add task" field. It works because it is trivial. Real applications have forms with multiple fields, validation rules, cross-field constraints, async checks, and complicated submit behavior. Trying to hand-roll all of that with `(input)` bindings the way we did in Chapter 6 is possible; it also loses more of an afternoon per form than it should.

Angular has two form systems: *template-driven forms* and *reactive forms*. They are not equivalent. Reactive forms are more powerful, more testable, and better-typed. This book uses reactive forms exclusively. Template-driven forms exist and have their fans; if you inherit an older codebase full of `ngModel`, know that it works and that Chapter 20's ecosystem tour will point you at resources for the older style.

In this chapter we replace the Chapter 6 "Add task" input with a reactive form, then build an edit dialog for tasks that lets the user change the title, set a due date, and tag it.

Setting up

Reactive forms come from `@angular/forms`. Register the necessary providers by importing the `ReactiveFormsModule` directives

in any component that uses them. Each standalone component that hosts a form imports what it needs:

```
import { FormBuilder, FormControl, FormGroup, ReactiveFormsModule } from '@angular/forms';

@Component({
  imports: [ReactiveFormsModule],
  // ...
})
```

`ReactiveFormsModule` is a bundle of directives — `formGroup`, `formControl`, `formControlName`, and so on. Templates that reference these directives must import the module.

`FormControl`, `FormGroup`, `FormArray`

The three primitives of reactive forms.

`FormControl<T>` wraps a single value. It knows its current value, its validation state, whether the user has touched or blurred it, and it emits changes.

```
const title = new FormControl('', { nullable: true, validators: [Validators.required] });

title.value;           // ''
title.setValue('Buy milk');
title.valueChanges;    // Observable<string>
title.valid;           // boolean
title.errors;          // { required: true } or null
```

`nullable: true` is important. Without it, `title.value` has type `string | null`, and calling `.reset()` sets the control back

to `null`. With `nonNullable: true`, the value is always `string`, and `reset` restores the initial value (" here). Use it on almost every control.

FormGroup is a collection of named controls.

```
const form = new FormGroup({
  title: new FormControl('', { nonNullable: true, validators: [Va
  dueDate: new FormControl<string | null>(null),
  tag: new FormControl('', { nonNullable: true }),
});

form.value;           // { title: string, dueDate: string | null
form.valid;
form.get('title')?.setValue('...');
```

The form is *typed*: `form.value` has an inferred type derived from the shape of the controls. This is the reactive-forms feature that most changed the game in Angular 14 — before, form values were all `any`, and typos in template `formControlName` would silently break.

FormArray is a dynamic list of controls of the same shape. Useful for repeating rows: "add another tag," "add another author." We will use it later in this chapter for tags.

FormBuilder is nicer

Constructing forms with `new FormControl(...)` is verbose. **FormBuilder** is a small helper service injected via DI:


```
private fb = inject(FormBuilder);

form = this.fb.nonNullable.group({
  title: ['', Validators.required],
  dueDate: [null as string | null],
  tag: [''],
});
```

`fb.nonNullable.group(...)` makes every control non-nullable by default. `fb.group(...)` (without `nonNullable`) uses the older nullable defaults. Prefer `fb.nonNullable`.

Each entry is `[initialValue, validators]`. Validators can be a single function or an array.

Rebuilding "Add task" with reactive forms

Open `src/app/add-task-form/add-task-form.ts`. Replace it with:

```
import { Component, inject, output } from '@angular/core';
import { FormBuilder, ReactiveFormsModule, Validators } from '@angular/forms';

@Component({
  selector: 'app-add-task-form',
  imports: [ReactiveFormsModule],
  templateUrl: './add-task-form.html',
  styleUrls: ['./add-task-form.css'],
})
export class AddTaskForm {
  private fb = inject(FormBuilder);
  add = output<string>();

  form = this.fb.nonNullable.group({
    title: ['', [Validators.required, Validators.minLength(2), Validators.maxLength(20)]]
  });

  submit(): void {
    if (this.form.invalid) return;
    this.add.emit(this.form.getRawValue().title.trim());
    this.form.reset();
  }
}
```

Template:

```

<form [formGroup]="form" (ngSubmit)="submit()">
  <input
    type="text"
    FormControlName="title"
    placeholder="What needs doing?" />
  <button type="submit" [disabled]="form.invalid">Add</button>
</form>

@if (form.controls.title.touched && form.controls.title.invalid) {
  <p class="error">
    @if (form.controls.title.hasError('required')) { Title is required }
    @if (form.controls.title.hasError('minlength')) { At least two characters }
    @if (form.controls.title.hasError('maxlength')) { Please keep it short }
  </p>
}

```

Save. It works exactly like before, but:

- **The form's shape is enforced by TypeScript.** If you mistype `FormControlName="titel"`, the template compiler tells you.
- **Validation is centralized.** No more `disabled` computation in the template — it reads `form.invalid`.
- **Errors have names.** Different messages for different constraints, without special-casing lengths in the template.
- **`.reset()` is one call.** No more manually clearing a signal.

`ngSubmit` (with no dollar sign in front) is a directive that catches form submissions and calls `submit()`, and it prevents the default browser reload. You no longer need `$event.preventDefault()`.

Built-in validators

The `Validators` object exports the common cases:

```
Validators.required
Validators.requiredTrue           // for checkboxes
Validators.min(0)
Validators.max(100)
Validators.minLength(3)
Validators.maxLength(200)
Validators.pattern(/^[a-z0-9-]+$/i)
Validators.email
```

For anything else, write a validator by hand. A validator is a function
`(control) => errors | null`.

```
function noWhitespace(control: AbstractControl): ValidationErrors {
  const v = (control.value ?? '').trim();
  return v.length === 0 ? { whitespace: true } : null;
}

new FormControl('', [Validators.required, noWhitespace]);
```

The error object's keys become entries in `control.errors`, which the template can check with `hasError('whitespace')`.

An edit dialog: `FormGroup` and `FormArray` together

Compass will need to let the user edit a task: change the title, set or clear the due date, and manage the list of tags. This is a real form: a `FormGroup` with a nested `FormArray`.

Create it:

```
ng generate component edit-task
```

```
edit-task.ts:
```

```

import { Component, effect, inject, input, output } from '@angular/core';
import {
  FormArray, FormBuilder, FormControl, FormGroup,
  ReactiveFormsModule, Validators
} from '@angular/forms';
import { Task } from '../task';

@Component({
  selector: 'app-edit-task',
  imports: [ReactiveFormsModule],
  templateUrl: './edit-task.html',
  styleUrls: ['./edit-task.css'],
})
export class EditTask {
  task = input.required<Task>();
  saved = output<Partial<Task>>>();
  cancelled = output<void>();

  private fb = inject(FormBuilder);

  form = this.fb.nonNullable.group({
    title: ['', [Validators.required, Validators.minLength(2)]],
    dueDate: [null as string | null],
    tags: this.fb.array<FormControl<string>>([]),
  });

  constructor() {
    // Populate the form when the task input arrives – required input
    // set *after* construction, so we use `effect` to react once it's
    // available. Effects run on registration, but `task()` will only be
    // set here – so guard with a try/catch and let the second effect handle
    effect(() => {
      // Read task() lazily so the effect only actually populates the form
      // has bound the input.
      const t = this.task();
      this.form.patchValue({ title: t.title, dueDate: t.dueDate });
      const tagsArray = this.form.controls.tags;
    });
  }
}

```

```
        tagsArray.clear();
        for (const tag of t.tags) {
            tagsArray.push(this.fb.nonNullable.control(tag, Validators.required));
        }
    });
}

addTag(): void {
    this.form.controls.tags.push(
        this.fb.nonNullable.control('', Validators.required)
    );
}

removeTag(index: number): void {
    this.form.controls.tags.removeAt(index);
}

submit(): void {
    if (this.form.invalid) return;
    const raw = this.form.getRawValue();
    this.saved.emit({
        title: raw.title.trim(),
        dueDate: raw.dueDate,
        tags: raw.tags,
    });
}

cancel(): void {
    this.cancelled.emit();
}
}
```

And a template that binds a `FormArray` :

```
<form [formGroup]="form" (ngSubmit)="submit()" class="edit-task">
  <label>
    Title
    <input type="text" formControlName="title" />
  </label>

  <label>
    Due date
    <input type="date" formControlName="dueDate" />
  </label>

  <fieldset>
    <legend>Tags</legend>
    <div formArrayName="tags" class="tags">
      @for (control of form.controls.tags.controls; track $index;
        <div class="tag-row">
          <input type="text" [formControlName]="i" />
          <button type="button" (click)="removeTag(i)">Remove</button>
        </div>
      }
    </div>
    <button type="button" (click)="addTag()">Add tag</button>
  </fieldset>

  <div class="actions">
    <button type="button" (click)="cancel()">Cancel</button>
    <button type="submit" [disabled]="form.invalid">Save</button>
  </div>
</form>
```

Two directives are new:

- `formArrayName="tags"` binds a container to a `FormArray`.

- Inside it, `formControlName="0"`, `"1"`, etc., bind each child to its slot. Using `[formControlName]="i"` with a template variable does this cleanly.

The form now supports adding and removing tags on the fly, with each tag validated individually. The typed form gives us `form.controls.tags.controls` as `FormControl<string>[]`, and TypeScript knows that `raw.tags` is `string[]`.

Async validators, briefly

Occasionally validation needs to hit the server: "is this username available?" Async validators are functions that return an Observable or Promise of `ValidationErrors | null`.

```
function uniqueTitle(api: TasksApi): AsyncValidatorFn {
  return control => {
    if (!control.value) return of(null);
    return timer(300).pipe(
      switchMap(() => api.check(control.value)),
      map(taken => (taken ? { unique: true } : null))
    );
  };
}

// In the form:
title: ['', {
  validators: [Validators.required],
  asyncValidators: [uniqueTitle(this.api)],
}],
```

The `timer(300)` is a manual debounce so we don't spam the server on every keystroke. `form.pending` is `true` while the async validator is running — bind it in the template to show a spinner.

We will not add unique-title validation to Compass — tasks with duplicate names are fine — but you now know where to reach.

`valueChanges` and reacting to form input

Every control and group has a `valueChanges` Observable:

```
this.form.controls.title.valueChanges
  .pipe(takeUntilDestroyed())
  .subscribe(v => console.log('title now:', v));
```

Or, converted to a signal:

```
titleValue = toSignal(this.form.controls.title.valueChanges, {
  initialValue: this.form.controls.title.value,
});
```

Both work. Reach for `toSignal` when you want the form's live state to drive `computed` derivations or effects.

For the common case of "when this control changes, run this side effect," use `valueChanges` with `takeUntilDestroyed` and don't reach for signals unless the effect happens to belong in a `computed`.

Cross-field validation

Sometimes validity depends on more than one control. A due date must be after "start date"; a confirm-password must match a password. Add a validator to the group, not the controls.

```
form = this.fb.nonNullable.group({
  password: ['', Validators.required],
  confirm: ['', Validators.required],
}, {
  validators: (group) => {
    const p = group.get('password')?.value;
    const c = group.get('confirm')?.value;
    return p === c ? null : { mismatch: true };
  },
});
```

Read it in the template as `form.errors?.['mismatch']`.

When to use template-driven forms

The short answer: on tiny forms where reactive forms are ceremony overkill. A single search input, a single toggle, a one-off checkbox: `[(ngModel)]` is fine. Anything with multiple fields, validation across them, or programmatic manipulation should be reactive.

The book uses reactive forms throughout because they scale from tiny to huge without changing paradigm. When you inherit an older Angular codebase and see `ngForm`, `ngModel`, `ngModelGroup`, you will be able to read it; you probably will not want to write new features in that style.

Wiring `EditTask` into `TaskList`

We won't build the full modal experience in this chapter — routing to a detail page is Chapter 13's job. For now, add a temporary "edit" mode to `TaskList` that shows the form under a selected row.

```
export class TaskList {
  private store = inject(TaskStore);
  tasks = this.store.tasks;
  editingId = signal<string | null>(null);
  editing = computed(() => this.tasks().find(t => t.id === this.e

  onEdit(id: string): void { this.editingId.set(id); }
  onCancel(): void { this.editingId.set(null); }
  onSave(changes: Partial<Task>): void {
    const id = this.editingId();
    if (id) this.store.applyChanges(id, changes);
    this.editingId.set(null);
  }
}
```

You will need to add `applyChanges` to `TaskStore`. It's very similar to `toggle` — apply an optimistic update, call the API, revert on failure:

```
async applyChanges(id: string, changes: Partial<Task>): Promise<v
  const previous = this._tasks();
  this._tasks.update(all => all.map(t => (t.id === id ? { ...t,
  try {
    await this.api.update(id, changes);
  } catch {
    this._tasks.set(previous);
    this._error.set('Could not save changes.');
```

Template:

```
@let e = editing();
@if (e) {
  <app-edit-task
    [task]="e"
    (saved)="onSaved($event)"
    (cancelled)="onCancel()" />
}
```

That is enough plumbing to prove the form works end-to-end. Chapter 13 turns this into a proper edit route.

What comes next

Part IV. Compass is now a working single-screen app: it fetches, adds, edits, and saves tasks against a real backend. Chapter 13 adds routing — a home screen, a stats page, a task detail view with the edit form — and Chapter 14 promotes `TaskStore` from a single service into a proper signal-based state layer that other stores can follow.

Chapter 15 refactors reusable UI into directives and pipes; Chapter 16 makes Compass fast on a phone.

Exercises

1. Add a `notes` field to `EditTask` — a `<textarea>` with `maxLength(1000)` validation. Show the remaining character count under the field, driven by `toSignal(form.controls.notes.valueChanges)`.
2. Convert the "Add task" form to include an optional due date.
When the user submits, the due date should be sent to `TaskStore.add` along with the title. Update `TaskStore.add` to accept and store it.
3. Wire an async validator into the "Add task" form that prevents duplicate titles: if the current tasks array already has a task with the same trimmed title, mark the control invalid with a `{ duplicate: true }` error. You don't need HTTP for this — read the store's `tasks()` signal directly inside the validator using `computed`.

Chapter 13. Routing: pages, params, guards, lazy loading

Compass has been a single screen. Real applications have several — a home, a settings, a detail view, sometimes dozens. Angular's *router* is the piece that turns one URL into one component tree, updates it as the user navigates, and lets pieces of the app react to which URL is active.

This chapter turns Compass into a multi-page app. We will add a home route that shows today's tasks, a stats route that will host our charts in Chapter 16, and a detail route that shows a single task with the edit form we built in Chapter 12. Along the way we cover route parameters, guards, resolvers, and lazy loading.

The three moving parts

Angular routing has three concepts you meet everywhere.

Route definitions — a list of `{ path, component }` records that tell the router what to render for each URL. You have seen the empty version in `src/app/app.routes.ts`.

<router-outlet> — a placeholder in a template that says "render the currently-active route's component here." You already had one, briefly, in the CLI-generated `App`; we removed it in Chapter 5.

Navigation — the user (or your code) changes the URL, and the router matches it against the route definitions, resolves any guards, and updates whichever `<router-outlet>` is showing the affected component.

Defining Compass's routes

Edit `src/app/app.routes.ts`:

```
import { Routes } from '@angular/router';
import { Home } from './home/home';
import { Stats } from './stats/stats';
import { TaskDetail } from './task-detail/task-detail';

export const routes: Routes = [
  { path: '', component: Home, title: 'Today · Compass' },
  { path: 'stats', component: Stats, title: 'Stats · Compass' },
  { path: 'task/:id', component: TaskDetail, title: 'Task · Compass' },
  { path: '**', redirectTo: '' },
];
```

Four routes:

- `''` is the empty path — the home route, matching `http://localhost:4200/`.
- `'stats'` matches `/stats`.
- `'task/:id'` matches `/task/anything`, capturing the segment after `task/` as `id`.
- `'**'` is the wildcard — anything unmatched redirects to home. Always define this last; the router walks the list top-to-bottom, and a `**` earlier would swallow everything.

The `title` property sets `document.title` when the route activates. It is a small feature that improves how tabs and bookmarks look in a browser.

We need `Home`, `Stats`, and `TaskDetail` components. Create them:

```
ng generate component home
ng generate component stats
ng generate component task-detail
```

For now, `Home` is where `TaskList` lives. Move the `<app-task-list>` markup and any orchestration from `App` into `Home`.

`src/app/home/home.ts`:

```
import { Component } from '@angular/core';
import { TaskList } from '../task-list/task-list';

@Component({
  selector: 'app-home',
  imports: [TaskList],
  template: `<app-task-list />`,
})
export class Home {}
```

`Stats` is a stub we will flesh out in Chapter 16:

```
@Component({
  selector: 'app-stats',
  template: `<h1>Stats</h1><p>Coming soon.</p>`,
})
export class Stats {}
```

`TaskDetail` we look at in a moment.

`App` becomes a shell

The root component's job is now to hold the `<router-outlet>` and any chrome that surrounds every page — a header, a footer, a sidebar.

Rewrite `src/app/app.ts` :

```
import { Component } from '@angular/core';
import { RouterOutlet, RouterLink, RouterLinkActive } from '@angular/router';

@Component({
  selector: 'app-root',
  imports: [RouterOutlet, RouterLink, RouterLinkActive],
  templateUrl: './app.html',
  styleUrls: ['./app.css'],
})
export class App {}
```

`src/app/app.html` :

```

<header class="app-header">
  <h1 class="brand">Compass</h1>
  <nav>
    <a routerLink="/" routerLinkActive="active" [routerLinkActiveOptions]="{ exact: true }">
      Today
    </a>
    <a routerLink="/stats" routerLinkActive="active">Stats</a>
  </nav>
</header>

<main class="app-main">
  <router-outlet />
</main>

```

`routerLink` renders an `<a>` with the correct `href` and, when clicked, tells the router to navigate — no full page reload. `routerLinkActive` adds a class when the link's route is active. `{ exact: true }` prevents `/` from being "active" every time — otherwise it would be highlighted for `/stats` too, because `/stats` starts with `/`.

Save. Refresh. You should see the header with two nav links. Clicking "Today" and "Stats" switches which view appears below. Notice the URL changes, and the browser back button works.

Route parameters and `:id`

The `task/:id` route captures a segment. In `TaskDetail`, we need to read that segment.

Modern Angular has an especially nice way: **component input binding** from the router. Turn it on in `app.config.ts`:

```
import { provideRouter, withComponentInputBinding } from '@angular/router';

// ...providers:
provideRouter(routes, withComponentInputBinding()),
```

Now any `input()` in a routed component receives matching route parameters as its input:

```
import { Component, inject, input, computed } from '@angular/core';
import { TaskStore } from '../task-store';
import { EditTask } from '../edit-task/edit-task';

@Component({
  selector: 'app-task-detail',
  imports: [EditTask],
  templateUrl: './task-detail.html',
})
export class TaskDetail {
  id = input.required<string>();
  private store = inject(TaskStore);
  task = computed(() => this.store.tasks().find(t => t.id === this.id));
}
```

And the template:

```
@if (task(); as t) {
  <app-edit-task [task]="t" />
} @else {
  <p>Task not found. <a routerLink="/">Back to today.</a></p>
}
```

`@if (task(); as t)` is a template alias — it binds the truthy value to `t` inside the block. Now `t` is the resolved `Task`, and

`EditTask` gets a real task to work with. The `computed` ensures that if the store updates, the detail view updates too.

Reading route params via `input()` is only available with `with-ComponentInputBinding()`. Without it, you would read them via `ActivatedRoute`:

```
private route = inject(ActivatedRoute);  
id = toSignal(this.route.paramMap.pipe(map(p => p.get('id') ?? '')))
```

Both work; input binding is cleaner and gets you free type inference on generated links.

Linking to a task

In `TaskRow`, add a link:

```
<a [routerLink]="['/task', task().id]" class="detail-link">Detail
```

The array form of `routerLink` builds the URL from segments. It handles URL encoding for you. Prefer it over `'/task/' + task().id`.

Notice that `TaskRow` now needs to import `RouterLink` in its `imports` array.

Query parameters

For state that should be reflected in the URL but is not part of the route match — filters, sorts, search queries — use *query parameters*.

Read them the same way you read route params, using `queryParamMap` on `ActivatedRoute`:

```
private route = inject(ActivatedRoute);
filter = toSignal(
  this.route.queryParamMap.pipe(map(p => p.get('tag') ?? 'all')),
  { initialValue: 'all' }
);
```

Write them via the `Router`:

```
private router = inject(Router);

setFilter(tag: string): void {
  this.router.navigate([], {
    relativeTo: this.route,
    queryParams: { tag: tag === 'all' ? null : tag },
    queryParamsHandling: 'merge',
  });
}
```

`queryParamsHandling: 'merge'` preserves other query params. Setting a value to `null` removes it.

Query params in the URL have two benefits: users can bookmark and share, and the browser's back button becomes meaningful (each filter change is a history entry, if you want it to be).

Guards: gating navigation

A guard is a function that decides whether a navigation is allowed. Three main types:

canActivate — can this user visit this route? Used for authentication.

canMatch — should this route even be considered for matching? Used to hide routes entirely from users who shouldn't reach them. Better than **canActivate** for auth because the route's code isn't downloaded if **canMatch** returns false.

canDeactivate — can the user leave this route? Used to warn about unsaved changes.

Here is a **canMatch** for an authenticated area:

```
// src/app/auth.guard.ts
import { CanMatchFn, Router } from '@angular/router';
import { inject } from '@angular/core';
import { AuthStore } from '../auth-store';

export const requireAuth: CanMatchFn = () => {
  const auth = inject(AuthStore);
  const router = inject(Router);
  if (auth.isSignedIn()) return true;
  router.navigateByUrl('/login');
  return false;
};
```

Attach it in **app.routes.ts** — note that the "code isn't downloaded when **canMatch** returns false" benefit only holds for **lazy-loaded routes**, so pair it with **LoadComponent** :

```
{
  path: 'stats',
  loadComponent: () => import('./stats/stats').then(m => m.Stats),
  canMatch: [requireAuth],
  title: 'Stats · Compass',
},
```

We won't add auth to Compass until Chapter 19 (deployment considers it), but you now know where auth guards go.

Resolvers: pre-fetching data

Some routes should not render until their data is loaded. A *resolver* is a function that runs before the route activates and blocks the navigation until it completes.

```
import { ResolveFn } from '@angular/router';
import { inject } from '@angular/core';
import { TaskStore } from './task-store';
import { Task } from './task';

export const taskResolver: ResolveFn<Task | undefined> = (route) => {
  const store = inject(TaskStore);
  const id = route.paramMap.get('id') ?? '';
  return store.tasks().find(t => t.id === id);
};
```

Attach it:

```
{ path: 'task/:id', component: TaskDetail, resolve: { task: taskR
```


And read the resolved data in the component via `Activated-Route.data`:

```
task = toSignal(this.route.data.pipe(map(d => d['task'] as Task |
```

Resolvers keep loading spinners out of your component code — but they also delay the first paint. Use them for data you cannot render meaningfully without; otherwise render the shell and let a loading state fill in.

Compass's `TaskDetail` uses the store's signal, not a resolver — the parent store has already loaded tasks, and we can find the task synchronously. Resolvers are more useful when the detail page's data is not already in a store.

Lazy loading

Every component in `imports` in your route file is included in the initial bundle. On a big app, this bloats the download.

Lazy loading defers a component's load until it is needed:

```
{
  path: 'stats',
  loadComponent: () => import('./stats/stats').then(m => m.Stats)
}
```

The router calls the arrow function only when the user navigates to `/stats`, downloads the resulting JavaScript chunk, and renders. There is a fraction-of-a-second delay the first time, which is usually acceptable.

For entire feature areas, `loadChildren` lazily loads a set of routes:

```
{
  path: 'settings',
  loadChildren: () => import('./settings/settings.routes').then(m
}
```

The child file exports its own `Routes` array. Everything under `/settings/*` is downloaded together.

We will convert Compass's `Stats` and `TaskDetail` routes to lazy loading in Chapter 16, once we have measurement to justify it.

Preloading

Lazy chunks download on demand, which introduces latency on first visit. Preloading downloads them in the background *after* the initial render:

```
import { withPreloading, PreloadAllModules } from '@angular/route
provideRouter(routes, withPreloading(PreloadAllModules)),
```

`PreloadAllModules` is the simple option: download everything lazy after the app has painted. A custom preloader can be smarter (only preload chunks likely to be visited next). For most apps, `PreloadAllModules` is a good default.

Programmatic navigation

You navigate imperatively with the `Router` :

```
private router = inject(Router);

goHome(): void { this.router.navigate(['/']); }
openTask(id: string): void { this.router.navigate(['/task', id]); }
back(): void { window.history.back(); }
```

`navigate` returns a Promise that resolves when navigation completes (or false if it was blocked). `navigateByUrl(url)` takes a string URL.

For most in-template navigation, prefer `[routerLink]` — it renders a real anchor, which is better for accessibility and for open-in-new-tab. Reach for `router.navigate` when navigating from a handler that already ran (after a form save, on a socket event, on a timer).

`RouterLinkActive` for nav highlighting

`routerLinkActive` adds a CSS class when the link's target is the active route. It can take a list of classes:

```
<a routerLink="/stats" routerLinkActive="active highlighted">Stats</a>
```

By default, a link is active when the URL *starts with* its target. `{ exact: true }` requires exact match. Use `exact` on the home link (/), which otherwise matches every URL.

titleStrategy : dynamic titles

Static `title: '...'` in a route works for pages whose title is known ahead of time. For "Task: X" where X depends on the task, define a `TitleStrategy`:

```
import { TitleStrategy, RouterStateSnapshot } from '@angular/router';
import { Title } from '@angular/platform-browser';

@Injectable({ providedIn: 'root' })
export class CompassTitleStrategy extends TitleStrategy {
  private title = inject(Title);
  override updateTitle(state: RouterStateSnapshot): void {
    const t = this.buildTitle(state);
    this.title.setTitle(t ? `${t} · Compass` : 'Compass');
  }
}

// providers:
{ provide: TitleStrategy, useClass: CompassTitleStrategy },
```

Compass's static titles are enough for now; know that the extension point exists.

What comes next

Chapter 14 promotes `TaskStore` into a general pattern for signal-based state and shows when reaching for a bigger state library (NgRx) is worth it. Chapter 15 covers directives and pipes — the primitives for reusable behavior and formatting that we haven't touched yet. Chapter 16 measures Compass's performance and fixes what needs fixing.

Exercises

1. Convert the "Stats" and "Task detail" routes to lazy loading. Then run `ng build` and look inside `dist/`. You should see multiple chunk files, one per lazy route.
2. Add a "canDeactivate" guard on the task detail route that prompts the user "You have unsaved changes; leave anyway?" when they try to navigate away from an edit with a dirty form. Hint: the guard function receives the component instance as its first argument.
3. Add a query parameter `filter` to the home route, valid values `all`, `open`, and `done`. Render only the matching tasks. Provide three buttons that update the query param via `router.navigate`, and highlight the active one using `routerLinkActive` on `routerLink` versions of the buttons.

Chapter 14. State beyond one component

`TaskStore` was a good first step. It works because Compass has one kind of data. Real applications have many kinds — tasks, habits, users, settings, drafts, filters, uploads, notifications — and they interact. Once several stores need to share values or react to each other, you need a *pattern*, not one clever service.

This chapter lays out that pattern. We convert Compass's ad-hoc state services into a small, consistent shape. We also cover when to reach for larger libraries — specifically NgRx `SignalStore` and the classic NgRx (Redux for Angular) — and how to tell.

The chapter is about design, so it has less code and more decisions than the last few. Read it slowly.

The problem statement

By the end of Compass, we will have at least these pieces of state:

- **Tasks** — an array, loaded from a server, mutated by the user.
- **Habits** — same shape.
- **User** — the currently signed-in user (Chapter 19).
- **UI state** — which task is selected, which filter is active, which panel is open, which theme is chosen.

- **Offline queue** — mutations that failed and need retry when the connection returns.

Some of these are server-backed. Some are purely local. Some are read by two components on screen; some are read by many across the app.

You want the state layer to have consistent shape so someone reading Compass six months from now doesn't have to relearn how each piece works. You also don't want to introduce so much machinery that a tiny piece of state costs fifty lines of ceremony.

The pattern: a **store** per concern

The pattern this book uses is simple and scales further than beginners expect. For each domain concern, you have one Injectable class — call it a Store — that owns a small number of private signals, exposes readonly signals for reads, and exposes methods for mutations.

The template is roughly:

```

@Injectables({ providedIn: 'root' })
export class SomeStore {
  private readonly _state = signal<State>(initialState);
  readonly state = this._state.asReadonly();

  readonly derivedA = computed(() => /* ... */);
  readonly derivedB = computed(() => /* ... */);

  method1(input): void { this._state.update(s => /* ... */); }
  async method2(input): Promise<void> {
    // possibly async work; optimistic updates; error handling
  }
}

```

That is `TaskStore` from Chapter 10, described generically. Two properties make this scale:

- **One writer per signal.** Only the store's own methods write to `_state`. Callers never bypass. This makes the mutations searchable and testable.
- **Readonly views.** The public `state` is a readonly signal. Consumers cannot mutate it accidentally. `computed` values are already readonly by construction.

Add a `HabitStore` with the same shape. Add a `UserStore`. Compass now has three stores, each about a hundred lines, each following the same template. New readers grok them immediately.

Local (UI) state vs shared state

Not every piece of state belongs in a Store. Two questions decide.

Does more than one component read this state? If no, keep it in the component. Signals inside a component are just as reactive as signals inside a service. Moving them into a service to be "clean" hurts readability without helping anything.

Does this state need to survive when this component is destroyed? If no, keep it in the component. A form draft, a modal's local UI, a component-local loading flag — all belong in the component.

The dividing line is usually clear. `TaskList`'s `editingId` in Chapter 12 is component-local — it does not survive the component going away. `TaskStore`'s `tasks` is shared and long-lived. Both are signals; only one is in a service.

Cross-store interaction

Sometimes one store needs to react to another. When the user signs out, all the task and habit state should clear. When a habit is deleted, the tasks that referenced it should have their tags updated.

The clean pattern for this: use `effect` in the depending store. It reads signals from the store it depends on, and reacts.

```

@Injectables({ providedIn: 'root' })
export class TaskStore {
  private userStore = inject(UserStore);

  constructor() {
    effect(() => {
      const user = this.userStore.currentUser();
      if (user === null) {
        this._tasks.set([]);
      }
    });
  }
}

```

Rules:

- **Depend downstream, never in a cycle.** `TaskStore` depending on `UserStore` is fine. `UserStore` depending on `TaskStore` is a bug waiting.
- **Never write to a signal you also read in the same effect.** Angular will refuse to run it and log a warning.
- **Never read one store's signals in a place another store's `computed` reads.** If `HabitStore.streakDays` reads `TaskStore.tasks`, refactor: put the computation where it belongs, or expose a purpose-built projection.

Effects are the sharp end of state management. Used sparingly they solve real problems. Overused they turn the state graph into an unlabeled maze.

Persistence: `localStorage` and beyond

Compass's tasks are backed by HTTP, but UI state — filters, theme, sidebar open/closed, drafts — is nicer to persist across sessions without a round trip. `localStorage` is the standard tool.

A tiny persistence utility:

```
export function persistedSignal<T>(key: string, initial: T): WritableSignal<T> {
  const raw = localStorage.getItem(key);
  let start: T = initial;
  if (raw !== null) {
    try { start = JSON.parse(raw); } catch { /* ignore */ }
  }
  const s = signal<T>(start);
  effect(() => {
    localStorage.setItem(key, JSON.stringify(s()));
  });
  return s;
}
```

Use it inside a store's constructor (or as a class-field with `runInInjectionContext` if you're clever):

```
@Injectable({ providedIn: 'root' })
export class UIState {
  filter = persistedSignal<'all' | 'open' | 'done'>('compass:filter');
  theme = persistedSignal<'light' | 'dark' | 'system'>('compass:theme');
}
```

Now the filter and theme survive refresh, with no additional wiring. Just remember: `localStorage` is synchronous and blocks the main thread. Do not store megabytes. Keep it for small UI state; keep server-backed data on the server.

Offline queue: a state pattern with real teeth

An offline queue is a good case study for a nontrivial store.

The idea: when the user makes a mutation (add a task, toggle done), Compass tries to send it to the server. If the network is down or the server returns 5xx, the mutation is queued locally with a timestamp and retried later — when the browser is online again, or after a delay.

Sketch of the store:

```

type PendingMutation =
  | { kind: 'add'; task: Omit<Task, 'id'>; localId: string }
  | { kind: 'toggle'; taskId: string; done: boolean }
  | { kind: 'remove'; taskId: string };

@Injectable({ providedIn: 'root' })
export class MutationQueue {
  private api = inject(TasksApi);
  private queue = persistedSignal<PendingMutation[]>('compass:queue');

  readonly pendingCount = computed(() => this.queue().length);
  readonly online = signal(navigator.onLine);

  constructor() {
    window.addEventListener('online', () => this.online.set(true));
    window.addEventListener('offline', () => this.online.set(false));
    effect(() => {
      if (this.online() && this.queue().length > 0) {
        this.drain();
      }
    });
  }

  enqueue(m: PendingMutation): void {
    this.queue.update(q => [...q, m]);
  }

  private async drain(): Promise<void> { /* ... */ }
}

```

The full drain implementation is longer than we need in this chapter (it has to handle ordering, partial failure, and reconciliation with server IDs). The key point is that the queue is *just another store*. It has private signals, exposes readonly views, exposes methods, uses effect for reactivity. It looks like every other store in Compass.

When you reach for something bigger — NgRx SignalStore or classic NgRx — this is usually the kind of code that motivates the switch. Not "we have more than one signal" but "we have coordinated state updates and cross-cutting behaviors."

NgRx SignalStore

`@ngrx/signals` is a lightweight library that codifies the pattern above and adds features like *entity management* (indexed arrays with fast lookups by id), *effect helpers*, and *reactive extensions*. If you find yourself writing a lot of `store.tasks().find(t => t.id === x)`, or building indexed lookups by hand, SignalStore's `withEntities` is worth a look.

A sketch:

```

import { signalStore, withState, withMethods } from '@ngrx/signalStore';
import { withEntities } from '@ngrx/signals/entities';
import { inject } from '@angular/core';

export const TaskStore = signalStore(
  { providedIn: 'root' },
  withEntities<Task>(),
  withState({ loading: false, error: null as string | null }),
  withMethods((store) => {
    const api = inject(TasksApi);
    return {
      async load() { /* ... */ },
      async add(title: string) { /* ... */ },
      // ...
    };
  }),
);

```

(Note that `withEntities` lives in `@ngrx/signals/entities`, not the main `@ngrx/signals` package. And `inject()` is called inside the `withMethods` factory body — not as a default parameter — because factories run in an injection context.)

`SignalStore` is opinionated and modestly sized. You do not need it for Compass; hand-rolled stores work fine. When you find that all your stores use similar entity-indexing code, `SignalStore` is the natural place to reach.

When to reach for classic NgRx (Redux)

Classic NgRx — actions, reducers, effects, selectors — is a much larger commitment. It brings:

- **Time-travel debugging.** Every state change is an action; you can step through them in DevTools.
- **Serializable state.** State is plain data, always. This makes it easy to save, replay, or ship over the network.
- **Predictable mutations.** A reducer is a pure function; the only way to change state is to dispatch an action.

It also brings:

- **Ceremony.** Every mutation becomes an action, a reducer case, an effect, a selector. A one-line change to state can touch five files.
- **A learning cliff.** New team members spend a week getting oriented.

Reach for classic NgRx when you have all of these:

- A team of five or more engineers who will touch state code daily.
- State mutations that need audit trails, undo/redo, or replay.
- Complex asynchronous flows that benefit from being described as pipelines of actions.

Do not reach for it because "the app is getting big." Bigness alone is not the criterion; complexity of the state graph is. Compass, even

fully-featured, does not need it. A large customer-support platform probably does.

Selectors, projections, and joins

When two stores need to combine their state, the answer is a *selector* — a `computed` that pulls from both.

```
@Injectable({ providedIn: 'root' })
export class TodayDashboard {
  private tasks = inject(TaskStore);
  private habits = inject(HabitStore);

  todayCount = computed(() =>
    this.tasks.tasks().filter(t => !t.done && this.isToday(t.dueDate))
    this.habits.habits().filter(h => !this.doneToday(h)).length
  );

  private isToday(d: string | null) { /* ... */ return false; }
  private doneToday(h: Habit) { /* ... */ return false; }
}
```

`TodayDashboard` is a *view store*: a store built on top of two others, providing a projected view. This composition pattern replaces most of the need for a global state container. Each store owns its concern; a view store combines what it needs.

Testing stores

Because stores are Injectables, they test with the same DI mechanics as components. Chapter 17 has the full story; the preview:

```
describe('TaskStore', () => {
  let store: TaskStore;
  let apiSpy: jasmine.SpyObj<TasksApi>;

  beforeEach(() => {
    apiSpy = jasmine.createSpyObj('TasksApi', ['list', 'create'])
    TestBed.configureTestingModule({
      providers: [{ provide: TasksApi, useValue: apiSpy }],
    });
    store = TestBed.inject(TaskStore);
  });

  it('exposes tasks after load', async () => {
    apiSpy.list.and.resolveTo([{ id: 't1', title: 'a', done: false }]);
    await store.load();
    expect(store.tasks().length).toBe(1);
  });
});
```

The store is Injectable, so `TestBed.inject` gets a fresh instance. Its dependency, `TasksApi`, is provided as a spy. The test drives the store's public API and asserts on its exposed signals.

What you should avoid

A few state-shaped anti-patterns catch beginners.

- **Reading from and writing to the same signal in one `computed`.** Computed values are for pure derivations. If you find yourself wanting to write, use an effect (and think hard about whether you have a state-graph cycle).
- **Making everything an observable.** Older Angular code centered on `BehaviorSubject` as the state primitive.

Signals are the right primitive now. Keep observables for genuine streams (HTTP, DOM events, WebSockets).

- **Storing derived state.** If A is $B + C$, don't store A . Store B and C ; derive A with `computed`. Then A cannot drift out of sync.
- **Deep object graphs.** Signals track references, not deep values. Big nested trees are hard to update immutably. Prefer flat, normalized state; use entity maps (`Record<Id, Entity>`) rather than nested arrays.

What comes next

Chapter 15 turns to directives and pipes — the reusable UI primitives that let you package behavior (auto-focus, long-press, tooltip) and formatting (relative time, currency, truncation) without wrapping everything in components. Chapter 16 profiles Compass and adds `@defer`, `OnPush`, and image optimization to make it fast on a phone.

Exercises

1. Add a `SelectionStore` that tracks which tasks are currently selected (say, for bulk actions). It should expose `selected: Signal<Set<string>>`, `isSelected(id)`, `toggle(id)`, `clear()`, and a `count = computed(...)`. Keep it small.
2. Wire the `MutationQueue` sketch above into `TaskStore`. When the network is offline, enqueue muta-

tions; when it comes back, drain them in order. Test by killing `json-server` mid-session and reading the queue's `pendingCount` signal.

3. Refactor Compass's `UIState` (filter, theme, sidebar) to use `persistedSignal`. Then open two browser tabs on Compass and see whether changes in one propagate to the other (they won't by default — `storage` events would, and that is a small extension to `persistedSignal`).

Chapter 15. Directives, pipes, and reusable UI

Components are Angular's biggest reuse unit. Pipes and directives are the smaller ones. A *pipe* is a function you can call inside a template expression — small, pure, transformation-shaped. A *directive* is behavior you attach to an existing element without wrapping it in another component. Both let you factor out repetition without introducing new tags or new component boundaries.

This chapter covers building both. We add a `TimeAgo` pipe that renders "2 hours ago" from a timestamp; an `Autofocus` directive that focuses an input on render; a `LongPress` directive that emits an event after a hold; and, briefly, a shared `EmptyState` component that uses content projection. By the end, Compass has a small toolkit of reusable UI primitives that the rest of the book (and any future feature) can reach for.

Pipes: template-only transformation

Compass currently uses `| date: 'shortDate'` in a few places. That is a built-in pipe. Angular ships several: `date`, `currency`, `percent`, `slice`, `keyvalue`, `json`, `titlecase`, `lowercase`, `uppercase`, `async`. You have used most of them.

Custom pipes are for transformations you find yourself repeating. The rules of a good pipe:

- **Pure.** Given the same inputs, always return the same output. No side effects. No global state.
- **Fast.** Pipes run every change detection. Anything expensive should be memoized or moved to a `computed`.
- **Small.** A pipe with 200 lines is a component or a service in disguise.

Building `TimeAgo`

Compass shows `createdAt` on hover, and we want the display to be "just now," "5 minutes ago," "2 hours ago," "yesterday," "3 days ago." Perfect pipe material.

Generate it:

```
ng generate pipe time-ago
```

```
src/app/time-ago.pipe.ts:
```

```

import { Pipe, PipeTransform } from '@angular/core';

@Pipe({ name: 'timeAgo' })
export class TimeAgoPipe implements PipeTransform {
  transform(value: string | Date | number | null | undefined): string {
    if (value === null || value === undefined || value === '') return '';
    const date = value instanceof Date ? value : new Date(value);
    if (isNaN(date.getTime())) return '';

    const seconds = Math.floor((Date.now() - date.getTime()) / 1000);
    if (seconds < 45) return 'just now';
    if (seconds < 90) return 'a minute ago';

    const minutes = Math.floor(seconds / 60);
    if (minutes < 45) return `${minutes} minutes ago`;
    if (minutes < 90) return 'an hour ago';

    const hours = Math.floor(minutes / 60);
    if (hours < 24) return `${hours} hours ago`;
    if (hours < 42) return 'yesterday';

    const days = Math.floor(hours / 24);
    if (days < 30) return `${days} days ago`;

    const months = Math.floor(days / 30);
    if (months < 12) return `${months} months ago`;
    return `${Math.floor(months / 12)} years ago`;
  }
}

```

Two rules of the road:

- **Handle bad input gracefully.** `null`, `undefined`, invalid strings — return `''` rather than throwing. Templates should not blow up because a value hasn't loaded yet.

- **Accept a union of reasonable types.** Consumers pass in different shapes; a pipe that only accepts one type causes ceremony at call sites.

Use it in a template:

```
<span title="{{ task.createdAt | date: 'medium' }}">
  Added {{ task.createdAt | timeAgo }}
</span>
```

Any component that uses the pipe in its template must import it:

```
@Component({
  imports: [TimeAgoPipe],
  // ...
})
```

Pipes with arguments

A pipe can take arguments: `{{ value | myPipe: arg1: arg2 }}`. Arguments become extra parameters to `transform`:

```
@Pipe({ name: 'truncate' })
export class TruncatePipe implements PipeTransform {
  transform(value: string, maxLength: number = 100, ellipsis: string) {
    if (!value) return '';
    return value.length <= maxLength ? value : value.slice(0, maxLength) + ellipsis;
  }
}
```

Use it: `{{ description | truncate: 80 }}`.

Pipes and change detection

Pipes are called on every change detection cycle by default (they are *pure* pipes, and Angular caches their output). Angular re-invokes a pipe only if any of its arguments has a new reference. That is why we `[...tasks]` before sorting: the new reference triggers re-evaluation.

An *impure* pipe (`@Pipe({ pure: false })`) is called every change detection whether inputs changed or not. Almost always the wrong choice. If you find yourself considering impure, ask why you aren't using a signal.

Attribute directives: behavior on existing elements

An attribute directive attaches behavior to whatever it's placed on. The classic use case is small pieces of imperative browser API — focus, tooltip, click-outside, resize observer — where wrapping the element in another component would be overkill.

Autofocus

Generate:

```
ng generate directive autofocus
```

```
src/app/autofocus.directive.ts:
```

```
import { Directive, ElementRef, inject, afterNextRender } from '@angular/core';

@Directive({
  selector: '[appAutofocus]',
})
export class AutofocusDirective {
  private el = inject<ElementRef<HTMLElement>>(ElementRef);

  constructor() {
    afterNextRender(() => this.el.nativeElement.focus());
  }
}
```

Use it in any template — for instance, on the edit-task input:

```
<input type="text" formControlName="title" appAutofocus />
```

Whenever the directive is created, the element receives focus after the next render (`afterNextRender` is a modern lifecycle hook that runs once, after DOM has been committed). No components involved. The element stays a native `<input>`.

Notice the selector: `[appAutofocus]`. Directives use attribute selectors, matched anywhere the attribute appears. The `app` prefix keeps them from colliding with existing attributes.

LongPress

A `LongPress` directive that emits an event after the user has been holding the element for a threshold of time.

```

import { Directive, ElementRef, inject, output, DestroyRef, input } from '@angular/core';

@Directive({
  selector: '[appLongPress]',
})
export class LongPressDirective {
  threshold = input<number>(500); // milliseconds
  longPress = output<void>();

  constructor() {
    const el = inject<ElementRef<HTMLElement>>(ElementRef).nativeElement;
    const destroyRef = inject(DestroyRef);

    let timeout: number | undefined;

    const start = () => {
      timeout = window.setTimeout(() => this.longPress.emit(), this.threshold);
    };
    const cancel = () => {
      if (timeout) window.clearTimeout(timeout);
    };

    el.addEventListener('mousedown', start);
    el.addEventListener('touchstart', start);
    el.addEventListener('mouseup', cancel);
    el.addEventListener('mouseleave', cancel);
    el.addEventListener('touchend', cancel);

    destroyRef.onDestroy(() => {
      el.removeEventListener('mousedown', start);
      el.removeEventListener('touchstart', start);
      el.removeEventListener('mouseup', cancel);
      el.removeEventListener('mouseleave', cancel);
      el.removeEventListener('touchend', cancel);
      cancel();
    });
  }
}

```

```
}
}
```

Use it:

```
<div appLongPress [threshold]="800" (longPress)="onLongPress()">
  Hold me
</div>
```

Directives can have inputs and outputs just like components. The whole reactive machinery works the same way. What's missing is the template — a directive has no view of its own; it augments a host element.

`DestroyRef` is Angular's DI-based tear-down mechanism. Injecting it and calling `onDestroy` is cleaner than an `ngOnDestroy` method. It also composes with `takeUntilDestroyed` under the hood.

Structural directives — mostly, don't

Structural directives are the ones prefixed with `*` (`*ngIf`, `*ngFor`, `*ngSwitchCase`). They add or remove elements from the DOM. Since the new control flow (`@if`, `@for`, `@switch`) covers every real case, you almost never write your own structural directive in modern Angular.

The one place they still appear is when you want to add a template-shaped API — take a piece of template content and render it under some condition or with some enrichment. If you find yourself needing this, look up the docs on `TemplateRef` and

`ViewContainerRef`. For an application-code book, we can leave it there.

`HostBinding` and `HostListener`, or their modern equivalents

An older pattern for directive behavior used decorators:

```
@HostBinding('class.active') isActive = false;  
@HostListener('click') onClick() { this.isActive = !this.isActive;
```

Modern Angular uses `host` in the metadata block:

```
@Directive({  
  selector: '[appToggleActive]',  
  host: {  
    '[class.active]': 'isActive',  
    '(click)': 'onClick()',  
  },  
})  
export class ToggleActiveDirective {  
  isActive = false;  
  onClick() { this.isActive = !this.isActive; }  
}
```

The two forms are equivalent; the `host` block is preferred going forward. In tiny directives, the imperative approach we used for `Long-Press` (adding event listeners directly on the element in the constructor) is also fine and sometimes clearer.

Content-projected components: `EmptyState`

Some reusable UI is a component with slots. Compass has several places that need an "empty state" — a friendly message when a list has no items. Rather than repeat the markup, extract a component.

```
@Component({
  selector: 'app-empty-state',
  template: `
    <div class="empty-state">
      <div class="icon"><ng-content select="[icon]"></ng-content>
      <h3>{{ title() }}</h3>
      <p class="body"><ng-content></ng-content></p>
      <div class="actions"><ng-content select="[actions]"></ng-content>
    </div>
  `,
  styles: `
    .empty-state { padding: 40px; text-align: center; color: #666; }
    .icon { font-size: 3rem; margin-bottom: 12px; }
    h3 { margin: 0 0 6px; font-size: 1.2rem; }
    .body { margin: 0 0 16px; }
  `,
})
export class EmptyState {
  title = input.required<string>();
}
```

Usage in `TaskList`:

```

@if (tasks().length === 0) {
  <app-empty-state title="Nothing to do today.">
    <span icon>☀️</span>
    Enjoy some free time, or add something below.
    <button actions (click)="focusInput()">Add a task</button>
  </app-empty-state>
}

```

Three named slots (`icon` , `default` , `actions`), all optional, filled by the caller. Content projection is what lets a design system's components stay declarative — the caller composes markup, the container styles.

When to reach for what

A checklist for "component, directive, or pipe."

Pipe — you have a value; you want a differently-shaped value in the template. No behavior, no DOM changes, no state.

Directive — you want to attach behavior to an element that already has a role. Focus, drag-and-drop, intersection observer, tooltip trigger. Zero or minimal DOM changes.

Component — the reusable thing is a piece of markup with its own visual identity, or has state, or needs to project content.

Getting this right saves a lot of code. Wrapping a `<div>` in an `<app-tooltip-host>` component when a `[appTooltip]` directive would do doubles the elements in the DOM. Pipe-ing a computation when a component-owned `computed` would do fragments the logic across the codebase.

Naming conventions

Convention across kinds of reusable UI:

- **Components** get an `app-` prefix on the tag: `<app-task-row>`, `<app-card>`.
- **Directives** get an `app` prefix on the attribute selector: `[appAutofocus]`, `[appLongPress]`.
- **Pipes** are usually unprefixed (`timeAgo`, `truncate`) because they don't collide with HTML the way tags and attributes do — the framework's built-in pipes are also unprefixed. A prefix is fine if you're publishing a library, but application code rarely needs it.

For application-code components and directives, the prefix keeps things unambiguous. For pipes, keep the name short and specific.

What comes next

Chapter 16 measures Compass's performance and applies the tools that matter. `OnPush` change detection, `@defer` blocks for parts that don't need to load immediately, image optimization, and bundle analysis. By the end of Chapter 16, Compass will feel snappy on a mid-range phone.

Exercises

1. Build a `Currency` pipe that formats a `number` as USD to two decimal places with commas ("1,234.56"). Then re-

- move it and use the built-in `currency` pipe. Compare — how many built-in pipes do you not need to write?
2. Build a `ClickOutside` directive that emits when the user clicks outside the element it is attached to. Use it to close a dropdown menu component. Hint: listen on `document.click`; check whether the target is a descendant of `el.nativeElement`.
 3. Convert Compass's "task row" from a component into a directive on `<li>` — `<li appTaskRow [task]="task" (toggled)="onToggle($event)">`. What can you keep? What has to move? At the end, decide whether the component or the directive is the better shape and undo whichever loses.

Chapter 16. Performance: change detection, `OnPush`, `@defer`, bundle analysis

Compass is fast on your laptop. It is probably not fast on a five-year-old phone on 4G. This chapter closes that gap. We measure Compass, apply the four techniques that matter most in an Angular app, and finish with a smaller, faster Compass that is ready for the real world.

The four techniques:

1. `OnPush` change detection (or, better, signals-driven change detection).
2. `@defer` blocks for parts of the UI that don't need to load immediately.
3. `NgOptimizedImage` for images that block time-to-interactive.
4. Bundle analysis, to see what your users actually download.

We will also touch on zoneless Angular — the direction of travel that ties several of these together.

Measure first

Never optimize without measuring. Angular gives you three good views into performance, and the browser gives you a fourth.

Angular DevTools is a browser extension that shows your component tree with change-detection metrics. Every time Angular checks a component's bindings, it records how long it took. Slow components are colored red. Install it from the Chrome or Edge extension store and open it via the browser's regular DevTools panel.

The Performance tab in the browser DevTools records the entire main thread. Open it, click record, click around in Compass, stop. You will see the flame chart of every function that ran. Look for long tasks (>50ms) and functions high in the flame chart.

Lighthouse (built into Chrome DevTools) runs a synthetic audit that includes Largest Contentful Paint, Total Blocking Time, and Cumulative Layout Shift — the metrics Google uses to score user-perceived performance.

`ng build --configuration production` builds an optimized bundle and prints the final file sizes. This is the fastest way to see whether a change made the download bigger or smaller.

Run all four before you touch anything. If Compass is already fast enough for your target devices, close this chapter.

Change detection: the classic story

Angular's default change detection is a *tree walk*. After every event (click, HTTP response, setTimeout, Promise resolution), Angular walks the entire component tree and checks every binding. If a binding's value has changed since the last check, it updates the DOM.

Zone.js is the library that intercepts every async API to trigger this walk. Without it, Angular has no idea when your code has changed state.

The walk is fast in small apps. It gets slow when the tree has thousands of bindings or when a single binding calls a method that does real work.

ChangeDetectionStrategy.OnPush

You can opt a component out of the default tree-walking. **OnPush** mode tells Angular: "Only check this component when one of its input references changes, or an event fires on it, or an Observable it subscribes to via **async** pipe emits, or one of the signals it reads changes."

```
import { Component, ChangeDetectionStrategy } from '@angular/core'

@Component({
  selector: 'app-task-row',
  changeDetection: ChangeDetectionStrategy.OnPush,
  // ...
})
```

OnPush reduces the amount of checking dramatically. In a signal-heavy app like ours, its side effects are barely visible — signals already know which components read them, and only those components get checked.

Best practice: **set OnPush on every component**. In a signal-driven codebase, you get precision for free. In a codebase mixing signals

and older patterns, `OnPush` forces you to be explicit about state, which is a good thing.

Zoneless: the direction of travel

Modern Angular can run without Zone.js entirely. Change detection triggers only from signal changes and from a few Angular-managed sources. Every `setTimeout` or `Promise.resolve()` no longer schedules a re-check.

Enabling it is one line in `app.config.ts`:

```
import { provideZonelessChangeDetection } from '@angular/core';

providers: [
  provideZonelessChangeDetection(),
  // Remove provideZoneChangeDetection() if it's there.
],
```

Then remove the `zone.js` polyfill from `angular.json` (in the `polyfills` array).

`provideZonelessChangeDetection` is the stable API (Angular v20+). Earlier versions used `provideExperimentalZonelessChangeDetection`, which was renamed once the feature stabilized — if you're on an older Angular, use that spelling. In a signal-first app — which Compass is — zoneless works. The parts that would break under zoneless are:

- Code that relies on `setTimeout` to cause a re-render. Rewrite to update a signal.
- `Promise.then` chains that update state without going through signals or `async` pipe. Same fix.

- Third-party libraries that dispatch DOM events outside Angular's awareness. Wrap the event source with a signal or an Observable that Angular can track.

Zoneless is stable as of Angular v20 (was called *experimental* zoneless in v18/v19). Every design decision in this book has been zoneless-ready — the switch is a single provider change plus removing the `zone.js` polyfill.

@defer blocks

Some parts of the UI aren't needed on first paint. Compass's "stats" page has a chart library it can lazy-load. A task detail page has an image gallery that only matters when the user scrolls to it. The "unsubscribe" section of a settings page is rarely visited.

`@defer` blocks defer the loading and rendering of a chunk of template until a condition is met. The syntax:

```
@defer (on viewport) {  
  <app-heavy-chart [data]="stats()" />  
} @placeholder {  
  <div class="chart-placeholder">Loading chart...</div>  
} @loading (after 100ms; minimum 300ms) {  
  <div class="spinner"></div>  
} @error {  
  <p>Could not load the chart.</p>  
}
```

The parts:

- **@defer (on viewport)** — start loading when the placeholder scrolls into view.
- **@placeholder** — what to render before the trigger fires. Must be simple markup; no heavy components allowed here (or the placeholder itself becomes heavy).
- **@loading** — what to render while the deferred chunk is downloading, if that takes long enough to be worth showing.
- **@error** — what to render if loading fails.

Triggers other than `on viewport` :

- `on idle` — when the browser is idle after page load (fires via `requestIdleCallback`).
- `on interaction` — when the user clicks or presses a key on a specified element.
- `on hover` — when the user hovers or focuses (`mouseenter` / `focusin`) — distinct from `interaction`.
- `on timer(30s)` — after a fixed delay.
- `on immediate` — as soon as non-deferred content on the page has finished rendering (useful for background chunks that don't need to block the initial paint).
- `when someSignal()` — programmatic trigger.

Deferred blocks are separate chunks in the build output. They download in the background, chunked by trigger. `@defer` is the single

largest lever most Angular apps have for reducing initial bundle size without lazy-loading whole routes.

For Compass, we'll defer the stats page's chart until the container scrolls into view. That saves the chart library (a few hundred KB) from the initial bundle.

NgOptimizedImage

Images are often the largest asset on a page. Angular's `NgOptimizedImage` directive handles size hints, lazy-loading below the fold, priority loading of the LCP image, and a few other tricks that light-house-audit checklists demand.

Import and use:

```
import { NgOptimizedImage } from '@angular/common';

@Component({
  imports: [NgOptimizedImage],
  // ...
})
```

```
<img ngSrc="/assets/hero.jpg" width="800" height="400" priority a
<img ngSrc="/assets/footer.jpg" width="1200" height="300" alt="fo
```

Notes:

- `ngSrc` (not `src`) enables the directive.
- `width` and `height` are required — *unless* you use `fill`, which sizes the image to fill its positioned parent

(useful for responsive layouts). Either way, the directive prevents layout shift.

- `priority` marks this image as the LCP candidate. Include it on at most one image per screen.
- Off-screen images without `priority` are lazy-loaded by default.

For Compass we do not have many images (it is a text-heavy app), but if you add a user avatar or a habit-icon, use `NgOptimizedImage`.

Bundle analysis

`ng build` prints file sizes. That is a start. For a real look at what's inside your bundle, use `source-map-explorer`:

```
npm install --save-dev source-map-explorer
ng build --configuration production --source-map
npx source-map-explorer dist/compass/browser/main-*.js
```

It opens an interactive treemap of your bundle: modules colored by size, grouped by source. You will usually find:

- **Angular itself:** 150–250 KB gzipped. Not much you can do about this.
- **Your own code:** usually tiny for a small app; the file you spent a week on is 10 KB.
- **Third-party libraries:** often the surprise. A charting library, a date library, or an icon set can dwarf everything else.

The two levers when third-party libraries dominate:

- **Tree-shaking.** Import only what you use. `import { format } from 'date-fns'` bundles just `format`; `import * as dateFns from 'date-fns'` bundles the whole thing.
- **Lazy or deferred loading.** Move the library behind an `@defer` or a lazy route. It does not need to be in the initial bundle.

Compass uses very few third-party libraries. If you extend it — with a chart library for the stats page, say — apply these levers early.

`trackBy` in `@for`, revisited

You met `track` in Chapter 6. It matters for performance too.

```
@for (task of tasks(); track task.id) {  
  <app-task-row [task]="task" />  
}
```

Without `track`, Angular would re-create every DOM node when the array reference changed. With `track task.id`, Angular reuses nodes by identity — moves them if positions changed, adds new ones for new ids, removes nodes for absent ids. Reused nodes preserve focus, animations, and any imperative state.

For long lists — hundreds of items — `track` by a stable id is the difference between "instant" and "sluggish."

For truly gigantic lists (thousands), reach for *virtual scrolling*: Angular Material's CDK includes a `cdk-virtual-scroll-viewport` that only renders the visible items. Compass will never need it, but it exists.

`computed` and memoization

`computed` caches. That is one of its virtues.

```
readonly expensiveDerivation = computed(() => {  
  // This body runs only when its dependencies change.  
  return tasks().sort(complexComparator).slice(0, 20);  
});
```

If your template calls `expensiveDerivation()` in ten places, the body runs once per change to `tasks`, and the ten reads return the cached result. Extracting expensive template expressions into `computed` is one of the most reliable wins in a signal-driven codebase.

The anti-pattern: an expensive method called from the template.

```
<span>{{      formatComplex(x)      }}</span>
```

runs `formatComplex` on every change detection. Extract it: `readonly formatted = computed(() => formatComplex(x()))` and read `formatted()`.

OnPush + signals in Compass

Turn on `OnPush` across Compass:

```
// Every component:  
@Component({  
  changeDetection: ChangeDetectionStrategy.OnPush,  
  // ...  
})
```

Because Compass reads state via signals, this changes nothing observable — but it lays the groundwork for zoneless. Angular's own scaffolding (via `ng generate component --change-detection=OnPush`) sets this for you if you configure it as the CLI default.

Add `@defer` to the stats page's chart section. Add `NgOptimizedImage` to any image you add. Add `computed` for any expensive derivation. That is 95% of the Angular-specific performance wisdom.

Not-Angular-specific things that matter

Framework performance can only help so much if the surroundings are broken. Two non-Angular checklists:

- **HTTP is slow.** Cache responses on the server, use CDN, use Brotli compression, use HTTP/2 or HTTP/3, use service workers if offline matters. Chapter 18's SSR helps time-to-first-paint; Chapter 19's deployment picks affect all of the above.
- **Fonts are heavy.** Every custom font is a network request that blocks text rendering. Subset your fonts. Use `font-display: swap`. Consider system fonts if you can.

If your Lighthouse audit is bad and you cannot find the Angular problem, look outside Angular.

What comes next

Part V begins with testing. Chapter 17 introduces `TestBed`, component harnesses, and Playwright for end-to-end tests. By the end of Part V, Compass has a test suite and a deployment plan, and Chapter 20 sends you off to the wider Angular world.

Exercises

1. Turn on `OnPush` on every component in Compass. Run the app; verify everything still works. Then break something on purpose (call `this._tasks().push(...)` in `TaskStore` — mutating instead of updating). What happens under `OnPush`? What is the fix? Write down the rule.
2. Wrap the (yet-to-be-built) `StatsChart` component in an `@defer` block on the stats page. Trigger on viewport with a placeholder that reads "Loading chart..." Verify in the network tab that the chart's chunk downloads only when you scroll.
3. Run `source-map-explorer` against Compass's production build. What are the three largest things in your bundle? For each, decide whether it can be lazy-loaded, deferred, or tree-shaken.

Chapter 17. Testing: TestBed, component harnesses, Playwright

Compass works. You have seen it work: you have clicked around, added tasks, and refreshed the page. But "it worked when I tried it" is not the same as "it will keep working when we change it next month." Tests are the difference. A good test suite lets you rename a signal, extract a service, or move a component with confidence — the tests will tell you what broke.

This chapter covers three layers of testing in Angular:

- **Unit tests** for stores, services, and pipes with `TestBed`.
- **Component tests** for individual components, using the CDK component harnesses.
- **End-to-end tests** for user flows across the whole app, with Playwright.

You will not test everything. Compass, like every real app, will always have testable code you haven't tested. The goal is not coverage — it is confidence. Test what would ruin your day if it broke, and what you keep breaking accidentally.

The testing setup Angular gives you

ng new scaffolds a testing setup out of the box: **Jasmine** as the assertion framework and **Karma** as the browser runner. Every generated file has a corresponding `.spec.ts` next to it.

Run the tests:

```
ng test
```

A browser window opens (Karma launches a real Chrome), runs the tests, and reports. On the second run, the browser stays open; changes to test files re-run the affected specs automatically.

If you want a modern, headless alternative, the Angular ecosystem also supports **Vitest** and **Web Test Runner**. Switching is a configuration change, not a rewrite — the test API is unchanged. Karma is being retired but is still the default at time of writing; Vitest is the direction of travel.

Regardless of runner, you write tests the same way: with Jasmine (`describe`, `it`, `expect`) and `TestBed`.

TestBed : DI for tests

`TestBed` is Angular's testing container. It configures an injector, lets you register providers (real or fake), and lets you instantiate components and services under test.

The pattern:

```

import { TestBed } from '@angular/core/testing';
import { TaskStore } from './task-store';
import { TasksApi } from './tasks-api';

describe('TaskStore', () => {
  let store: TaskStore;
  let apiSpy: jasmine.SpyObj<TasksApi>;

  beforeEach(() => {
    apiSpy = jasmine.createSpyObj<TasksApi>('TasksApi', ['list',
    TestBed.configureTestingModule({
      providers: [
        TaskStore,
        { provide: TasksApi, useValue: apiSpy },
      ],
    }));
    store = TestBed.inject(TaskStore);
  });

  it('starts with an empty tasks array', () => {
    expect(store.tasks()).toEqual([]);
  });

  it('loads tasks from the API', async () => {
    apiSpy.list.and.resolveTo([
      { id: 't1', title: 'a', done: false, createdAt: '', dueDate
    });
    await store.load();
    expect(store.tasks().length).toBe(1);
    expect(store.tasks()[0].title).toBe('a');
  });

  it('surfaces errors from the API', async () => {
    apiSpy.list.and.rejectWith(new Error('boom'));
    await store.load();
    expect(store.tasks()).toEqual([]);
    expect(store.error()).toContain('Could not load');
  });

```



```
});
});
```

The key move: `{ provide: TasksApi, useValue: apiSpy } .` Because `TaskStore` gets `TasksApi` via DI (`inject(TasksApi)`), the test replaces it with a spy. The store cannot tell — it just calls methods on whatever it was handed.

`jasmine.createSpyObj` builds an object whose named methods are pre-stubbed with `jasmine.Spy`. Each spy method has `and.resolveTo(value)`, `and.rejectWith(err)`, `and.returnValue(v)`, `and.callFake(fn)` for controlling behavior, and `.calls.count()`, `.calls.mostRecent()`, etc. for asserting on how it was called.

This is what makes Angular so testable: because everything comes in through DI, everything can be swapped in tests without touching the code under test.

Testing signals

Signals are values, not events. You test them by reading them at the right time.

```
it('remaining ignores done tasks', () => {
  store['_tasks'].set([
    { id: 't1', title: 'a', done: false, /* ... */ } as Task,
    { id: 't2', title: 'b', done: true, /* ... */ } as Task,
  ]);
  expect(store.remaining()).toBe(1);
});
```

The bracket access `store['_tasks']` is a slight cheat — it reaches into the private signal to set state directly. For most tests, prefer to drive state through the public API (call `store.add`, `store.toggle`). Reaching for the private signal is fine when a test is explicitly about a derived value and you don't want to set up the whole API dance.

For async signal changes, `await` the promise that drives the change, then read:

```
it('sets loading true then false around a load', async () => {
  const seen: boolean[] = [];
  const stop = TestBed.runInInjectionContext(() =>
    effect(() => seen.push(store.loading()))
  );
  apiSpy.list.and.resolveTo([]);
  await store.load();
  expect(seen).toEqual([false, true, false]);
});
```

`effect` inside a test needs to run in an injection context, which `TestBed.runInInjectionContext` provides.

Testing pipes

Pipes are functions. Test them like functions.

```
import { TimeAgoPipe } from './time-ago.pipe';

describe('TimeAgoPipe', () => {
  let pipe: TimeAgoPipe;

  beforeEach(() => {
    pipe = new TimeAgoPipe();
  });

  it('returns "just now" for recent times', () => {
    const twentySecondsAgo = new Date(Date.now() - 20_000).toISOString();
    expect(pipe.transform(twentySecondsAgo)).toBe('just now');
  });

  it('returns a minute count for < 45 minutes', () => {
    const tenMinutesAgo = new Date(Date.now() - 10 * 60_000).toISOString();
    expect(pipe.transform(tenMinutesAgo)).toBe('10 minutes ago');
  });

  it('handles null gracefully', () => {
    expect(pipe.transform(null)).toBe('');
  });
});
```

No `TestBed` needed — the pipe has no injected dependencies. This is the easiest kind of test, and pipes are often over-covered because of that. Test the interesting edge cases; skip the trivial ones.

Testing components with `ComponentFixture`

`TestBed.createComponent` instantiates a component and returns a `ComponentFixture` — a wrapper

with access to the component instance, its rendered DOM, and change detection controls.

```
import { TestBed } from '@angular/core/testing';
import { AddTaskForm } from './add-task-form';
import { By } from '@angular/platform-browser';

describe('AddTaskForm', () => {
  it('emits add with the trimmed title', async () => {
    await TestBed.configureTestingModule({
      imports: [AddTaskForm],
    }).compileComponents();

    const fixture = TestBed.createComponent(AddTaskForm);
    fixture.detectChanges();

    const emitted: string[] = [];
    fixture.componentInstance.add.subscribe(v => emitted.push(v))

    const input: HTMLInputElement = fixture.debugElement.query(By.c
    input.value = ' Buy milk ';
    input.dispatchEvent(new Event('input'));
    fixture.detectChanges();

    const form: HTMLFormElement = fixture.debugElement.query(By.c
    form.dispatchEvent(new Event('submit'));

    expect(emitted).toEqual(['Buy milk']);
  });
});
```

Two things to note:

- **fixture.detectChanges()** runs Angular's change detection for the fixture's tree. Call it after you mutate an input or a signal to see the rendered DOM update.

- `By.css(selector)` queries the rendered DOM. `fix-
ture.debugElement.query(...)` returns a `DebugEle-
ment`; `.nativeElement` is the underlying HTML ele-
ment.

This kind of test works but is verbose. For real component testing, the CDK component harnesses are much better.

Component harnesses

A **harness** is a class that wraps a component and exposes its behavior at a task level — "click the submit button," "read the error message" — rather than at the DOM level. Angular's CDK library provides a base class and Angular Material provides harnesses for every Material component out of the box. You can write your own harnesses for your own components.

```

import { ComponentHarness } from '@angular/cdk/testing';

export class AddTaskFormHarness extends ComponentHarness {
  static hostSelector = 'app-add-task-form';

  private input = this.locatorFor('input');
  private submitButton = this.locatorFor('button[type=submit]');

  async setTitle(v: string): Promise<void> {
    const el = await this.input();
    await el.sendKeys(v);
  }

  async submit(): Promise<void> {
    const btn = await this.submitButton();
    await btn.click();
  }

  async isSubmitDisabled(): Promise<boolean> {
    const btn = await this.submitButton();
    return (await btn.getProperty('disabled')) as boolean;
  }
}

```

Using it in a test:

```

import { TestbedHarnessEnvironment } from '@angular/cdk/testing/t

it('disables submit while empty', async () => {
  const fixture = TestBed.createComponent(AddTaskForm);
  const harness = await TestbedHarnessEnvironment.harnessForFixture(fixture);
  expect(await harness.isSubmitDisabled()).toBe(true);
  await harness.setTitle('New task');
  expect(await harness.isSubmitDisabled()).toBe(false);
});

```

The test reads like the user's actions. No `dispatchEvent`, no `detectChanges`, no `nativeElement`. Every method is `async` because harnesses work identically in unit tests (TestBed) and end-to-end tests (Protractor / Playwright) — the async-ness is what makes that portability possible.

Writing a harness for each of Compass's components is more code than the components themselves. In practice, teams write harnesses for components used by multiple tests — the "add task form" gets one because five tests exercise it; a one-off component doesn't. Match your investment to the reuse.

Testing components that inject services

The pattern is the same as testing stores: provide fakes.

```
it('shows tasks from the store', async () => {
  const fakeStore = {
    tasks: signal<Task[]>([{ id: 't1', title: 'a', done: false, /
    remaining: signal(1),
    loading: signal(false),
    error: signal(null),
    load: () => Promise.resolve(),
    add: () => Promise.resolve(),
    toggle: () => Promise.resolve(),
  };
  await TestBed.configureTestingModule({
    imports: [TaskList],
    providers: [{ provide: TaskStore, useValue: fakeStore }],
  }).compileComponents();

  const fixture = TestBed.createComponent(TaskList);
  fixture.detectChanges();

  expect(fixture.nativeElement.textContent).toContain('a');
});
```

Signals in the fake need to be actual signals — the component reads them with `()`. Plain values won't do.

End-to-end tests with Playwright

Unit tests exercise pieces. End-to-end (E2E) tests exercise the whole thing: a real browser, a real server, a real user flow.

Angular no longer scaffolds Protractor. Playwright is the modern choice; add it with:

```
ng add playwright-ng-schematics # or follow Playwright's own
```


A Playwright test:

```
import { test, expect } from '@playwright/test';

test('adds a task', async ({ page }) => {
  await page.goto('http://localhost:4200/');
  await page.fill('input[type=text]', 'Wash the car');
  await page.click('button[type=submit]');
  await expect(page.locator('li.task').filter({ hasText: 'Wash the car' }));
});
```

The test drives a real browser. It clicks, types, waits for elements, and asserts. Playwright handles auto-waiting so you rarely need `sleep` s.

E2E tests need `ng serve` and `json-server` running. In CI, you script them to boot before the tests run and shut down after. Playwright has a `webServer` config option for this.

E2E tests are slow and flakey by nature (real network, real timing). Do not try to test every path with E2E. Test the *critical journeys*: sign in and see your tasks, add a task and refresh, delete a task and confirm it's gone.

Testing routing and guards

The router has its own testing utilities. `RouterTestingModule` is the modern way:

```
import { provideRouter, withComponentInputBinding } from '@angular/router';
import { RouterTestingModule } from '@angular/router/testing';

it('navigates to task detail', async () => {
  await TestBed.configureTestingModule({
    providers: [provideRouter(routes, withComponentInputBinding())]
  }).compileComponents();

  const harness = await RouterTestingModule.create();
  const detail = await harness.navigateByUrl<TaskDetail>('/task/t1');
  expect(detail.id()).toBe('t1');
});
```

(The `withComponentInputBinding()` is essential — without it, `TaskDetail`'s `id = input.required<string>()` won't receive the route param and reading `detail.id()` will throw.)

Guards and resolvers are functions; test them with a `TestBed` that provides their dependencies, and call them directly:

```
it('redirects unauthenticated users', () => {
  const routerSpy = jasmine.createSpyObj('Router', ['navigateByUrl']);
  const authSpy = { isSignedIn: signal(false) };
  TestBed.configureTestingModule({
    providers: [
      { provide: Router, useValue: routerSpy },
      { provide: AuthStore, useValue: authSpy },
    ],
  });
  const allowed = TestBed.runInInjectionContext(() => requireAuth());
  expect(allowed).toBe(false);
  expect(routerSpy.navigateByUrl).toHaveBeenCalledWith('/login');
});
```

What to test, in priority order

You cannot test everything. Roughly, invest in this order:

1. **State transitions in stores.** These are pure and easy to test, and they encode most of the business logic. `toggle` before / after; `add` when API succeeds and when it fails.
2. **User-visible behavior of critical components.** "Clicking submit adds the task." Do not test private methods; test what a user or another component observes.
3. **Guards and resolvers.** They gate access; a bug here has security consequences.
4. **Pipes with non-trivial logic.** Test the branches.
5. **A handful of end-to-end journeys.** Sign in, do the main action, sign out. Not every feature, just the ones whose failure would page you at 3 AM.

Skip the trivial. Do not test that `TaskList` sets `title = 'Tasks'`. Do not test the CLI-generated boilerplate.

Continuous integration, briefly

Every test in `.spec.ts` runs with `ng test`. In CI, run it headless:

```
ng test --browsers=ChromeHeadless --watch=false
```

(Some CI environments need the no-sandbox flag on Chrome — if you hit that, define a `customLaunchers` entry in `karma.conf.js`)

extending `ChromeHeadless` with `flags: ['--no-sandbox']`, and reference it here.)

Playwright:

```
npx playwright test
```

Compass's CI pipeline (Chapter 19 revisits deployment):

- Install: `npm ci`
- Lint: `ng lint`
- Type-check + unit test: `ng test --watch=false`
- Build: `ng build --configuration production`
- E2E: `npx playwright test`

If all pass, deploy.

What comes next

Chapter 18 covers server-side rendering and hydration — the technique that gives your users their first paint before Angular has finished booting. Then Chapter 19 deploys Compass to the internet, and Chapter 20 sends you off with a plan for staying current.

Exercises

1. Write a spec for `TaskStore.toggle` that covers the optimistic-update happy path and the reverting-on-failure path. Verify with the API spy that the store called `update`

exactly once and, on failure, the tasks signal returned to its previous shape.

2. Write a component harness for `TaskRow`. Methods: `getTitle`, `isDone`, `clickToggle`, `clickDetails`. Use it in a test that verifies clicking the toggle emits the `toggled` output with the row's id.
3. Set up Playwright for Compass. Write one end-to-end test that starts on the home page, adds a new task, refreshes the browser, and asserts the task is still there. This confirms the full stack (frontend + json-server) is wired correctly.

Chapter 18. Server-side rendering and hydration

Compass, as it stands, is a *client-rendered* application. When a user first visits, the browser downloads a nearly-empty HTML shell, downloads a bundle of JavaScript, and only then does Angular boot up and render the page. Users on fast networks see this as a brief flash of nothing before the app appears. Users on slow networks see it as a long blank screen. Search-engine crawlers may see it as an empty page and index nothing.

Server-side rendering (SSR) fixes both. The server runs your Angular app, produces the fully-rendered HTML for the requested URL, and sends that back to the browser. The user sees content immediately. Angular then boots up on the client and *hydrates* the page — attaching event handlers to the existing DOM rather than re-rendering it from scratch.

This chapter turns Compass into an SSR app. It is one of the shortest chapters in the book because Angular has done most of the work. `ng add @angular/ssr` gets you 80% of the way.

SSR, client rendering, and static generation

Three modes, and the naming varies across ecosystems:

- **Client-side rendering (CSR)**: server sends a shell; the browser downloads JS and renders. Simple; slow first paint.
- **Server-side rendering (SSR)**: server renders each request. First paint is fast; the server does work per request.
- **Static site generation (SSG) / prerendering**: server renders every route once, at build time, and serves the static HTML. First paint is fast and the server does zero per-request work — but every URL must be known at build time.

Angular supports all three. You choose per route: some are prerendered (the marketing pages), some are SSR (personalized pages that need session data), some are CSR (heavy interactive tools that don't benefit from server rendering).

For Compass, the sensible mix is:

- Home page: CSR (personalized, no SEO need).
- Marketing / landing page (if we had one): SSG.
- Task detail: SSR when the URL is shared publicly; otherwise CSR is fine.

We will enable SSR globally in this chapter and let per-route configuration follow.

Adding SSR to Compass

Run:

```
ng add @angular/ssr
```

The schematic does several things:

1. Adds `@angular/ssr` and Express as dependencies.
2. Creates `server.ts` at the project root — an Express server that renders Angular for each request.
3. Adds a `server` build target to `angular.json`.
4. Adds `main.server.ts` — the server-side entry point (your browser-side `main.ts` stays as it was).
5. Adds `app.config.server.ts` for server-only providers.
6. Enables hydration via `provideClientHydration()` in `app.config.ts`.

After it finishes:

```
ng serve                                # dev server; SSR-enabled once the
npm run build                           # builds both browser and server bu
npm run serve:ssr:compass                # runs the built server on port 4000
```

Open `http://localhost:4000` (the default SSR port). View the source of the page (right-click → View Source). You will see the fully-rendered task list in the HTML — not just an empty `<app-root>`. Refresh and inspect the network tab; the first response arrives already populated.

Hydration: matching the client to the server

Once Angular boots on the client, it needs to *hydrate* the existing DOM instead of throwing it away and re-rendering. `provideClientHydration()` (added by the schematic) handles this. It matches the server-rendered nodes with the client-rendered component tree, attaches event listeners, and preserves any DOM state (like an input's value) that was already there.

Rules of the road:

- **Server and client must produce the same HTML.** Random content (`Math.random()` , `new Date()`) will differ, and hydration will warn about the mismatch. Wrap non-deterministic content in `@if (isBrowser())` (see below) so it renders only on the client.
- **DOM manipulation before hydration completes is dangerous.** Don't reach into `document` from a component constructor unless you know what you are doing.
- **Third-party libraries that manipulate the DOM directly (jQuery plugins, some chart libraries) don't work well with hydration.** Load them inside `afterNextRender` , or defer them entirely.

Detecting the platform

Some code should only run in the browser. Angular gives you `isPlatformBrowser` :

```
import { inject, PLATFORM_ID } from '@angular/core';
import { isPlatformBrowser } from '@angular/common';

@Injectable({ providedIn: 'root' })
export class SomeService {
  private isBrowser = isPlatformBrowser(inject(PLATFORM_ID));

  loadPreference(): string {
    if (this.isBrowser) {
      return localStorage.getItem('pref') ?? 'default';
    }
    return 'default';
  }
}
```

On the server, `PLATFORM_ID` is `'server'`; on the client, `'browser'`. This lets you avoid calling browser APIs on the server, which would throw because `window`, `document`, and `localStorage` don't exist there.

For components: `afterNextRender` fires only on the client. Use it to run browser-specific initialization:

```
constructor() {
  afterNextRender(() => {
    // Runs on the client, once, after the first render.
    window.scrollTo(0, 0);
  });
}
```

Transfer state: don't re-fetch what the server already had

When the server renders Compass's home page, it fetches tasks from the API. When the client then hydrates, it would re-fetch — the same data — from the same API, wasting a round trip.

Angular has *transfer state* to solve this. Wrap HTTP with `withHttpTransferCacheOptions` in `app.config.ts`:

```
import { provideHttpClient, withFetch } from '@angular/common/http';
import { provideClientHydration, withHttpTransferCacheOptions } from '@angular/platform-browser';

providers: [
  provideHttpClient(withFetch()),
  provideClientHydration(
    withHttpTransferCacheOptions({
      includeHeaders: [],
      includePostRequests: false,
    })
  ),
],
```

Angular serializes every HTTP GET's response into the initial HTML (inside a `<script>` tag). On the client, `HttpClient` checks that cache before hitting the network — a matching request returns the cached response immediately.

You can also transfer arbitrary state via `TransferState`:

```
import { makeStateKey, TransferState } from '@angular/core';

const TASKS_KEY = makeStateKey<Task[]>('tasks');

// On the server:
const transferState = inject(TransferState);
transferState.set(TASKS_KEY, tasks);

// On the client:
const initial = transferState.get(TASKS_KEY, []);
```

Compass will pick up transfer state automatically for its `HttpClient` calls, without extra plumbing.

Prerendering specific routes

For routes whose content doesn't depend on the user, prerender at build time. Add them to `app.routes.server.ts` (created by the SSR schematic):

```
import { RenderMode, ServerRoute } from '@angular/ssr';

export const serverRoutes: ServerRoute[] = [
  { path: '', renderMode: RenderMode.Prerender },
  { path: 'stats', renderMode: RenderMode.Server },
  { path: 'task/:id', renderMode: RenderMode.Server },
];
```

`Prerender` runs at build time; `Server` runs per request; `Client` disables SSR for that route.

For dynamic segments like `task/:id`, provide a `getPrerenderParams` function that returns the ids to prerender at build time:

```
{
  path: 'task/:id',
  renderMode: RenderMode.Prerender,
  getPrerenderParams: async () => {
    const tasks = await fetchAllPublicTasks();
    return tasks.map(t => ({ id: t.id }));
  },
}
```

Compass's task detail is user-specific, so prerendering doesn't help; `RenderMode.Server` (SSR per request) is the right choice if we want the page shareable, or `RenderMode.Client` (client-only) if not.

Testing SSR locally

`npm run build && npm run serve:ssr:compass` runs the production build against a local Node server. Open it, view source, confirm content is present.

For debugging server-only code, `console.log` in a component or service will appear in the terminal (server), not the browser (client). If you see nothing, remember the split.

SEO metadata

Server-rendered pages let search engines see your content — but they also need metadata. Angular's `Title` and `Meta` services set the `<title>` and `<meta>` tags:

```
import { Title, Meta } from '@angular/platform-browser';

constructor() {
  const title = inject(Title);
  const meta = inject(Meta);
  title.setTitle('Compass – plan your day');
  meta.updateTag({ name: 'description', content: 'A calm daily plan' });
  meta.updateTag({ property: 'og:image', content: 'https://compass...' });
}
```

For per-page metadata, drive these from route data or resolvers. The `TitleStrategy` we introduced in Chapter 13 is the systematic way.

When SSR is worth it

Not every Angular app benefits from SSR.

SSR pays off when:

- Your users often visit on slow networks.
- Search engines need to index the content.
- Social sharing (Open Graph, Twitter cards) is important.
- Your app has significant JS beyond what's on the visible portion of first paint.

SSR is not worth it when:

- The app is behind auth and never indexed.
- The heavy content is user-generated after first load (chat, editor, dashboard).

- Every page requires personalization that the server can't cheaply compute.
- You would need to duplicate significant server-side logic.

Compass benefits mildly from SSR — the app is small, and users typically bookmark it and load it from cache after the first visit. If you were building a public marketing site, SSR would be closer to essential.

What comes next

Chapter 19 deploys Compass to the internet. Static hosting for a client-rendered Compass, Node hosting for the SSR variant, and CI to make deployment safe. Chapter 20 hands you the map for what to learn next.

Exercises

1. Enable SSR on Compass, build it, and serve it. View source on the home page. Compare the size of the initial HTML response now versus before SSR (network tab, "Doc" filter, look at "Size"). Was there a meaningful change?
2. Add `provideClientHydration()` if it isn't there; then break hydration on purpose by exposing a `now = Date.now()` property on a component and rendering `{{ now }}` in its template. Open the console; you should see a hydration mismatch warning because the server and client compute different timestamps. Fix it by either (a)

guarding the value with `isPlatformBrowser` so it only exists on the client, or (b) marking the element with `ng-SkipHydration` so Angular skips matching it.

3. Add title and description meta tags to the task detail route, so a task's URL, when shared to Slack or a messaging app, unfurls with a useful preview. Test with the Facebook Sharing Debugger or the Twitter Card Validator on your deployed URL (Chapter 19 gets you deployed).

Chapter 19. Deployment

Compass is finished. It runs on your laptop against `json-server`. To have any value, it needs to live somewhere on the internet where you (and, if you want, others) can reach it. This chapter walks through the choices — static hosting, Node hosting, containers — and takes Compass live on one of them.

The two shapes of an Angular app in production

Whatever hosting you pick, an Angular app in production is one of two shapes.

Static bundle. `ng build` produces `dist/compass/browser/` — a folder of HTML, JS, and CSS. Serving that folder from any static file server is enough. This is the shape of a client-rendered Compass or a prerendered SSG build.

Node server. With SSR enabled, `ng build` also produces `dist/compass/server/` — a Node.js server. Running `node dist/compass/server/server.mjs` starts an Express server that renders each request and serves the built assets. This shape needs Node hosting.

The rest of the chapter is: which hosting for which shape.

What Compass needs from its backend

Before hosting the frontend, remember: Compass talks to a backend. `json-server` is a development toy; it is not something to run in production. For a real deployment, replace it with:

- A real REST or GraphQL API you own.
- A backend-as-a-service (Firebase, Supabase, PocketBase).
- A serverless API (Cloudflare Workers, Vercel Functions, AWS Lambda + API Gateway).

The Angular side of Compass doesn't care which. As long as the endpoints in `TasksApi` return the same shape, everything works.

For this chapter we assume you have chosen one and it exposes an API at `https://api.compass.example`. Update the `BASE_URL` constant in `tasks-api.ts`, or (better) turn it into a build-time environment variable — see the *Environment configuration* section below.

Hosting a static Angular build

If Compass is client-rendered (no SSR), any static host works. The mechanics differ; the principle is the same: upload `dist/compass/browser/` and configure the host to fall back to `index.html` for unmatched routes.

The last point matters. Angular's client-side router expects the browser to load `index.html` for any URL and let Angular figure out the route. If the host returns 404 for `/stats`, users cannot deep-

link. Every static host has a way to configure this fallback; it is often called "SPA mode" or "rewrite to index."

Three good static hosts:

Netlify. Drag-and-drop the folder onto `app.netlify.com`, or push to GitHub and connect the repo. Netlify auto-detects Angular, runs `ng build`, and serves `dist/compass/browser/`. Configure fallback in `netlify.toml`:

```
[[redirects]]
  from = "/*"
  to = "/index.html"
  status = 200
```

Vercel. Similar workflow. `vercel.json`:

```
{
  "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }]
}
```

Cloudflare Pages. Connect a Git repo; Cloudflare builds and serves globally on their edge network. Fallback via `_redirects`:

```
/*      /index.html    200
```

All three have free tiers generous enough for personal projects, custom domains, HTTPS, and CI-triggered deployments. The picks are close in features; pick one and try it.

Hosting SSR Compass

An SSR Compass ships a Node server. That excludes pure static hosts and requires one of:

Vercel, with the Node runtime. Angular's SSR output works out of the box on Vercel; deploy the whole repo, and Vercel will build the SSR bundle and run the server on demand.

Firebase Hosting + Cloud Functions. Angular has first-party integration; `ng deploy` from `@angular/fire` uploads the browser assets to Firebase Hosting and the server bundle as a Cloud Function.

A container on any cloud. Build a small Docker image, deploy it to Fly.io, Railway, Render, DigitalOcean's App Platform, AWS ECS, Google Cloud Run, or any equivalent. This is the most flexible; it is also the most work.

Here is a minimal Dockerfile that works for Angular SSR:

```
FROM node:20-slim AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM node:20-slim AS runtime
WORKDIR /app
COPY --from=build /app/dist ./dist
COPY --from=build /app/node_modules ./node_modules
COPY package*.json ./
EXPOSE 4000
CMD ["node", "dist/compass/server/server.mjs"]
```

Build and run locally:

```
docker build -t compass .  
docker run -p 4000:4000 compass
```

Push the image to a registry and deploy on the host of choice.

Environment configuration

Compass's `BASE_URL` should not be hardcoded. Angular projects handle this with environment files.

Create `src/environments/environment.ts`:

```
export const environment = {  
  production: false,  
  apiUrl: 'http://localhost:3000',  
};
```

And `src/environments/environment.prod.ts`:

```
export const environment = {  
  production: true,  
  apiUrl: 'https://api.compass.example',  
};
```

Configure `angular.json` to substitute the file at build time. In the `configurations.production.fileReplacements` array (which is usually already there):

```
"fileReplacements": [
  {
    "replace": "src/environments/environment.ts",
    "with": "src/environments/environment.prod.ts"
  }
]
```

Import and use:

```
import { environment } from '../environments/environment';
// ...
const BASE_URL = environment.apiUrl;
```

`ng build --configuration production` swaps in the prod values. `ng serve` uses the dev values.

For secrets — API keys, tokens — do not check them into environment files. Anything Angular ships to the browser is public. Put secrets on the backend, expose only the frontend-safe values.

HTTPS, CORS, and cookies

Once Compass is on a domain and its backend is on another, three configurations become load-bearing.

HTTPS everywhere. Every good host provides it automatically. Do not deploy over HTTP; browsers block or degrade too many APIs (service workers, geolocation, cookies) over HTTP.

CORS. The backend must send `Access-Control-Allow-Origin: https://compass.example` (or the specific origins that are

allowed). If Compass makes requests with cookies, `Access-Control-Allow-Credentials: true` and the origin must not be `*`.

Cookies. If auth uses cookies, they must be `Secure` (HTTPS-only), `HttpOnly` (JS-inaccessible), and `SameSite=Lax` or `Strict`. Cross-site cookies (`SameSite=None`) require a good reason and additional care.

Each of these has bitten teams once and taught them a lesson permanently. Configure them right the first time.

Continuous deployment

Set up CI so pushes to `main` deploy Compass. Every host has a template.

GitHub Actions to Netlify:

```

name: Deploy
on:
  push:
    branches: [main]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20 }
      - run: npm ci
      - run: npx playwright install --with-deps
      - run: npm run test -- --watch=false --browsers=ChromeHeadless
      - run: npx playwright test
      - run: npm run build
      - uses: nwtgck/actions-netlify@v3
        with:
          publish-dir: 'dist/compass/browser'
          production-deploy: true
    env:
      NETLIFY_AUTH_TOKEN: ${ secrets.NETLIFY_AUTH_TOKEN }
      NETLIFY_SITE_ID: ${ secrets.NETLIFY_SITE_ID }

```

Push a commit; a few minutes later, production is updated.

Preview environments

Ideally, every pull request gets its own preview URL. Netlify, Vercel, and Cloudflare Pages do this automatically for connected Git repos. Reviewers open the preview link, click around, and comment on real behavior rather than trying to imagine it from a diff.

If your host doesn't do this natively, GitHub Actions can build and push to a per-branch S3 bucket or Firebase channel. The setup is one afternoon and pays off forever.

Custom domains

Every host we mentioned supports custom domains. The mechanics:

1. Buy a domain from a registrar (Namecheap, Cloudflare Registrar, etc.).
2. In the host, add the domain.
3. In the registrar, add the DNS records the host provides (usually a CNAME or two A records).
4. Wait a few minutes for DNS to propagate.
5. HTTPS certificate is provisioned automatically via Let's Encrypt.

Point `compass.yourname.dev` at Netlify; five minutes later, `https://compass.yourname.dev` serves Compass.

Analytics and error reporting

Once Compass is public, you want to know when it breaks.

Error reporting: Sentry, Rollbar, Bugsnag, LogRocket. Each has an Angular SDK; you initialize it in `main.ts` and configure global error handling by providing `ErrorHandler`:

```
providers: [
  { provide: ErrorHandler, useClass: SentryErrorHandler },
],
```

Analytics: Plausible, Fathom, or Google Analytics. Load a script tag, or use their SDK. The Angular Router has a `Router.events` Observable that emits on every navigation; subscribe and record the URL if you want single-page tracking.

Both are optional; both are usually worth setting up before you tell anyone about your app.

Performance monitoring

Real-user monitoring (RUM) tracks how the app performs for actual users, on their actual devices. Web Vitals — LCP, FID, CLS — are standardized metrics you can measure with `web-vitals` (the library):

```
import { onCLS, onINP, onLCP } from 'web-vitals';

onCLS(metric => sendToAnalytics(metric));
onINP(metric => sendToAnalytics(metric));
onLCP(metric => sendToAnalytics(metric));
```

(Older code used `onFID`, but `web-vitals` v4 removed it in favor of `onINP` — Interaction to Next Paint, a more accurate responsiveness metric.)

Each host provides its own RUM if you prefer not to roll your own.

Rolling back and staging

Every deployment can go wrong. Two safety nets you want early:

One-click rollback. All the hosts we listed let you rewind to a previous deploy. Practice using it before you need it.

A staging environment. A separate deployment (compass-staging.example) that mirrors production but with a test backend. Deploy to staging first, verify, promote to production. This is the difference between a Friday-evening deploy and a Friday-evening incident.

Post-deploy: keeping Angular current

Angular ships a major version every six months. `ng update` handles most migrations:

```
ng update @angular/cli @angular/core
```

It runs migration schematics that adjust your code automatically for renamed APIs. Sometimes manual work is needed; the update output tells you what.

Stay one or two versions behind current if you value stability; stay on current if you want new features. Do not fall more than three versions behind — the upgrade path becomes painful.

What comes next

Chapter 20 is the last chapter. It maps the Angular ecosystem beyond what Compass needed — Material, Nx, NgRx, Universal, Router extensions — and points you at what to read, watch, and try next.

Exercises

1. Deploy Compass (client-rendered version) to Netlify, Vercel, or Cloudflare Pages. Include the SPA fallback so `/stats` doesn't 404. Share the URL with a friend and confirm it loads over their phone data.
2. Enable SSR (per Chapter 18), build the SSR bundle, and deploy to a host that supports Node (Vercel, Fly.io, or Cloud Run). Verify with `curl -sS https://your-url/task/t1 | grep -c task-detail` that the response contains rendered HTML.
3. Set up GitHub Actions to run tests, build, and deploy on every push to `main`. Break one test intentionally; verify that the deploy is skipped. Fix the test; watch it succeed.

Chapter 20. Where to go next

Compass exists. You built it. In the process you met every idea a working Angular developer relies on: components, templates, signals, dependency injection, HTTP, RxJS, forms, routing, state management, directives, pipes, performance, testing, SSR, deployment. That is a full skill set. Not a starter kit — a working set. If you understood what you typed in each chapter, you can walk into an Angular codebase and be productive.

This chapter is the map for what to do with that.

The ecosystem beyond Compass

Compass used a small subset of the Angular ecosystem. Here is what else is out there, in the order you are most likely to want it.

Angular Material and the CDK

Angular Material (`@angular/material`) is Google's Material Design component library for Angular. Buttons, dialogs, tables, tabs, snackbars, autocompletes, date pickers, drag-and-drop — all designed to a coherent visual system and tested against accessibility guidelines.

Angular CDK (`@angular/cdk`) is the primitives under Material. If you don't want the Material look but want the a11y-correct behaviors (a proper focus trap, an accessible dialog, keyboard navigation for

tables, drag-and-drop, virtual scrolling, overlays), CDK is your library. Even large custom design systems reach for CDK to avoid re-implementing the hard parts.

Compass would benefit from CDK for its edit dialog, keyboard navigation, and drag-and-drop between "today" and "later." Add it any time — CDK has no visual opinion.

PrimeNG, Nebular, and other UI kits

Not every project wants Material. **PrimeNG** is a large component library with a business-app aesthetic. **Nebular** is a comprehensive UI kit built around a theme system. **Taiga UI** is a modern Angular UI kit. **Spartan/ui** is a shadcn-inspired kit.

The pattern is the same in each: install, import, use. The right choice depends on how your app should look; if you're building an internal tool, PrimeNG's density is a match; if it should feel modern and light, Spartan is a match.

NgRx and the state ecosystem

Chapter 14 introduced state patterns. The libraries worth knowing beyond that:

- **@ngrx/signals** — signal-based state with `signalStore` and `withState / withMethods / withComputed` features. `withEntities` (from `@ngrx/signals/entities`) adds indexed entity management; `rxMethod` (from `@ngrx/signals/rxjs-interop`) is how you attach RxJS-driven side effects.

- **@ngrx/store** — the classic Redux-for-Angular. Actions, reducers, selectors, effects. Bigger commitment, bigger payoff on complex apps.
- **@ngrx/data** — reduces the "we need an entity store" boilerplate on top of Store.
- **@rx-angular/state** — a lighter, RxJS-first alternative to NgRx.

Pick the smallest one that solves your problem. Do not adopt NgRx Store on a small app "just in case"; the ceremony will drag every feature.

Nx: the monorepo tooling

Nx is a monorepo tool with deep Angular integration. If your team is building multiple Angular apps that share code — a customer app and an admin app that both use the same design system, say — Nx makes the shared library, code-generation, testing, and caching much better than what plain Angular CLI provides.

Nx also works for polyglot repos (Angular + Node + Nest + React). Its overhead pays off around 5+ people and 2+ apps.

Testing beyond what we covered

Chapter 17 covered TestBed, harnesses, and Playwright. Two more libraries worth knowing:

- **Cypress** — an older Playwright competitor. Solid, especially for teams already using it.

- **Storybook** — not a test runner, but adjacent: interactive documentation for your components, with visual regression testing available via addons. Perfect for design systems.
- **jest-preset-angular** or **Jest with @angular/build** — if you want to use Jest instead of Jasmine. Vitest is now the more forward-looking alternative.

Server frameworks that pair well

If you build a backend to go with Compass, three Angular-adjacent choices:

- **NestJS** — a Node backend framework whose design is unmistakably inspired by Angular (decorators, DI, modules). Angular devs learn it in a day.
- **AnalogJS** — a meta-framework built on Angular, similar in shape to Next.js for React. If you want file-based routing, server functions, and SSR/SSG baked in, AnalogJS is worth exploring.
- **Astro with Angular islands** — Astro is a content-first framework that can embed Angular components as interactive "islands." Useful for content-heavy sites that need small interactive pieces.

Real-time and offline

Compass in this book had a mutation queue but no real-time. Two libraries make it interactive:

- **@microsoft/signalr** (SignalR) or **socket.io-client** — WebSocket abstractions for pushing updates from the server to the client.
- **RxDB** — a client-side reactive database with server-sync built in. It replaces `MutationQueue` and much of `TaskStore` with a small, working library.

If Compass ever grows into a collaborative app, one of these will be in the stack.

Accessibility

Angular apps can be accessible; they can also be inaccessible. The tools:

- **@angular/cdk/a11y** — focus trap, focus monitor, live announcers.
- **axe-core** and **@axe-core/playwright** — automated a11y audits. Add to CI.
- The MDN pages on ARIA and semantic HTML — required reading if you're the person on the team who cares about a11y.

A study plan for the next month

Concrete things to do, in order.

Week 1: Build again. Pick a new small app — a reading list, a workout log, a bookmark manager — and build it from scratch, using only the ideas in this book. Give yourself a week, and don't peek back at Compass except when stuck. This is where the material becomes muscle memory.

Week 2: Read someone else's code. Clone a small, well-regarded Angular open-source project. Some starting points:

- **Angular Material's source** — good code, high standards, well-tested.
- **Nx's example apps** — mid-sized realistic monorepos.
- **Angular Realworld** — the "realworld" reference implementation for the Angular framework.

Read for a few hours, following one feature end-to-end. Note idioms you didn't know.

Week 3: Contribute. Find an issue tagged `good first issue` on any Angular-adjacent project — Angular itself, Angular Material, NgRx, RxJS. Fix it. The PR process teaches you as much as the code did.

Week 4: Answer questions. Open Stack Overflow or the Angular Discord and answer beginner Angular questions. The best way to consolidate what you know is to explain it to someone else who is stuck.

Staying current

Angular has a well-run release cadence and a reasonable public record. Where to keep up:

- **The official blog** at `blog.angular.dev` announces releases and RFCs. (`blog.angular.io` redirects there — the `.io` domain is legacy.)
- **The Angular Twitter/X account** (`@angular`) posts release videos.
- **This Week in Angular** (a newsletter, findable via web search) summarizes the ecosystem weekly.
- **YouTube channels:** Deborah Kurata, Joshua Morony, Angular Air, Angular Nation — good for feature deep-dives.
- **Conferences:** ng-conf (US), NG-DE (Germany), NG Poland — talks are usually on YouTube afterward.

You do not need to consume all of these. Pick one or two and check in monthly.

What Angular is becoming

At the time of writing, Angular is in the middle of a quiet transformation. Signals, standalone components, and the new control flow have shifted the mental model; zoneless is coming; SSR and hydration are mature. If you learned Angular five years ago, the framework today would surprise you.

That trajectory is worth knowing because the ideas you learned in this book are the ideas Angular is moving toward, not away from.

Signals will keep expanding — signal-based forms, resource, effect improvements. NgModules will keep receding. Zoneless will become the default. The gap between "this book" and "the future" is small; the gap between "an older Angular book" and "the future" is large.

A closing note

You started this book not knowing what a framework was. You are ending it having built and deployed a real Angular application, complete with authentication, offline support, tests, and CI. The steps between those two points — every chapter — were designed to be small enough to take one at a time and large enough that at the end you were doing real work.

Compass is yours. Extend it. Use it. Rewrite it in a year to see how you have grown. The best sign that a foundation is solid is that you can build past it, and this book was meant to leave you a foundation, not a ceiling.

Angular is a large framework in a large ecosystem. You will not know all of it. You do not need to. You know enough to look up what you don't, understand what you find, and add it to your working set. That is what "learning a framework" actually means, and you have done it.

Good luck. Ship things. Come back to the exercises in a few months and see how much easier they are.

Appendix A. Setting up your machine

This appendix collects the platform-specific setup Chapter 4 pointed you to. Read only the sections that apply to your operating system. Come back for troubleshooting.

What you need, at a glance

- **Node.js 20 or later.** Angular refuses to install on older versions.
- **npm 10 or later.** Ships with Node.js.
- **The Angular CLI.** Installed globally via npm.
- **A code editor.** Visual Studio Code is recommended and free.
- **Git.** For version control; often preinstalled.
- **A terminal.** Any modern terminal application.

macOS

Node.js. Two good options:

1. Download the LTS installer from `nodejs.org` and run it. This installs `node` and `npm` at `/usr/local/bin` (Intel) or `/opt/homebrew/bin` (Apple Silicon).

2. Use Homebrew: `brew install node`. Homebrew's install matches your architecture automatically.

Verify:

```
node --version
npm --version
```

Angular CLI.

```
npm install --global @angular/cli
ng version
```

Editor. Install Visual Studio Code from `code.visualstudio.com`. After install, open Terminal and run `code --version` — if it isn't found, open VS Code and use Command Palette → "Shell Command: Install 'code' command in PATH".

Git. `xcode-select --install` installs the Command Line Tools, which include git. Or `brew install git`.

Windows

Node.js. Download the LTS `.msi` installer from `nodejs.org` and run it. Accept the default options. It installs Node.js and npm and adds them to your PATH.

Open a new PowerShell or Windows Terminal window (not one from before the install — PATH updates only affect new sessions). Verify:

```
node --version  
npm --version
```

Angular CLI.

```
npm install --global @angular/cli  
ng version
```

If you get an execution policy error running `ng`, run PowerShell as administrator and set:

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

Editor. Install Visual Studio Code from code.visualstudio.com. The installer offers an "Add to PATH" checkbox — leave it checked.

Git. Install Git for Windows from git-scm.com. Accept the defaults, but on the "Adjusting your PATH environment" screen, choose "Git from the command line and also from 3rd-party software."

A terminal. Install Windows Terminal from the Microsoft Store if it isn't already there. It handles PowerShell, cmd, and Git Bash gracefully.

Linux (Ubuntu / Debian / similar)

Node.js. The distro-packaged Node.js is often too old. Use Node-Source:

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install nodejs
node --version
npm --version
```

Or use nvm (Node Version Manager):

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh
# reopen your terminal
nvm install 20
nvm use 20
```

nvm is the better choice long-term because it lets you switch Node versions per project.

Angular CLI.

```
npm install --global @angular/cli
ng version
```

If you installed Node globally without nvm, you may need `sudo` :

```
sudo npm install --global @angular/cli
```

Better: configure npm to install globals in your home directory:

```
mkdir ~/.npm-global
npm config set prefix '~/.npm-global'
# then add ~/.npm-global/bin to your PATH
```

Editor. VS Code has `.deb` and `.rpm` packages at `code.visualstudio.com`, or install via Snap: `sudo snap install code --classic`.

Git. Almost always preinstalled. If not:
`sudo apt install git.`

Recommended VS Code extensions

None are strictly required, but these make working with Angular much nicer:

- **Angular Language Service** (official Angular team) — inline errors and autocomplete in templates.
- **ESLint** — surfaces lint issues in the editor.
- **Prettier** — auto-formats code on save.
- **GitLens** — line-level git blame and history.
- **Path Intellisense** — autocompletes import paths.

Install via Extensions panel or `code --install-extension <id>.`

Troubleshooting

"ng is not recognized" / "command not found."

npm installed the CLI but your PATH doesn't include npm's global bin directory. Reopen your terminal first. If still failing:

```
npm config get prefix
```

Add the returned path's `bin` subdirectory to your PATH (in `~/.zshrc`, `~/.bashrc`, or your shell's config).

Node version too old.

You installed Node ages ago and Angular now needs a newer one. On Windows/macOS run the LTS installer again; it upgrades. On Linux, use nvm as described above.

Long install times.

`ng new` downloads a lot of packages the first time. On slow connections this can take five minutes. Subsequent projects share npm's cache and are faster.

"EACCES" errors from npm.

You are trying to install a global package without permission. Either use `sudo` (Linux; macOS with system Node), configure npm's prefix to your home directory (better), or use nvm (best).

Corporate proxy or firewall.

`npm` respects the environment variables `HTTP_PROXY` and `HTTPS_PROXY`. If your workplace requires them, set them in your shell config. Some corporate firewalls block Node's certificate lookups; ask your IT team for the internal cert bundle and set `NODE_EXTRA_CA_CERTS`.

HMR / dev server won't hot-reload.

Some file systems (Windows shared drives, some Docker mounts) don't notify Node of changes. Try `ng serve --poll=1000` (polls every second). Slower but reliable.

Antivirus slows every build to a crawl (Windows).

Add your project folder and `%APPDATA%\npm-cache` to your antivirus exclusions. Do not disable antivirus.

"Cannot find module" after `git pull`.

Someone else added a dependency. Run `npm install` to pick it up.

Port 4200 is already in use.

Another `ng serve` is running, or another process took the port. `ng serve --port 4201`. Find and kill the process: on macOS/Linux `lsof -i :4200`; on Windows `netstat -ano | findstr 4200` and `taskkill /PID <id>`.

A minimum quality-of-life setup

If you are new to shell environments, spend fifteen minutes on these once:

- Use an editor with syntax highlighting for TypeScript. (VS Code has it out of the box.)
- Turn on "format on save" so your code is uniformly indented and quoted.
- Use a terminal with pretty colors (Windows Terminal, iTerm2, GNOME Terminal). Reading long stack traces in monochrome is unpleasant.
- Learn your editor's file-navigation shortcut. In VS Code: `Cmd/Ctrl+P` → type a filename fragment. This alone doubles your speed in a large project.

Node versions across projects

Once you have two Angular projects on your machine, they may target different Node versions. `nvm` is the cleanest solution: put a `.nvmrc` file in each project (`echo 20 > .nvmrc`), and `nvm use` picks up the right version automatically.

Windows has `nvm-windows`. macOS/Linux use `nvm-sh`. Both work; both integrate well with VS Code.

Appendix B. Debugging Angular apps

Everything in this book worked when you typed it, at some point. Everything you build after this book will break. This appendix is what to reach for when it does.

The three questions

When something is wrong, ask, in order:

1. **What did I expect to see?**
2. **What am I seeing instead?**
3. **What could bridge the gap?**

Skip step 1 and you will fix the wrong problem. Skip step 2 and you will guess. Answer both explicitly before you touch any code.

The browser DevTools tour

Every debugging session touches at least one of:

- **Console** — errors, warnings, and your `console.log` output. `console.dir(obj)` prints an object's properties, expandable. `console.table(arr)` prints arrays of objects as a table. `console.trace()` prints the call stack at that point.

- **Network** — every HTTP request. Filter by "XHR" for API calls. Click a request to see headers, body, and response. Check status codes: 2xx good, 4xx client error, 5xx server error.
- **Elements** — the live DOM. Click an element to see its attributes and computed styles. `$0` in the console refers to the currently-selected element.
- **Sources** — the loaded JavaScript, source-mapped back to your TypeScript. Set breakpoints; step through code.
- **Performance** — record a session, see the flame chart. Chapter 16 covered this.
- **Application** — inspect `localStorage`, `sessionStorage`, cookies, IndexedDB, service workers, and cache storage. Useful when persistence code misbehaves.

Learn the keyboard shortcut for DevTools: F12 in most browsers, Cmd+Option+I on macOS.

Angular DevTools

Install the Angular DevTools browser extension. It adds two tabs to the browser DevTools:

- **Components** — the live component tree, with props, state, and injected services. Click a component to see its inputs and outputs.
- **Profiler** — record change detection cycles. See which components re-rendered, when, and how long each took.

If the Components tab shows "Angular is not detected," you are looking at a page that isn't Angular, or Angular is running in production mode (which by default disables devtools support). Compass's dev build supports it out of the box.

Reading Angular error messages

Angular errors are three parts: an **NG code** (like `NG0100`), a **short message**, and a **detailed message with a link**.

The NG code is your best friend. Search for it: `NG0100 site:angular.dev`. The official docs have an explanation with common causes for every code. (`angular.io` is a legacy domain that redirects to `angular.dev`; search results still work, but the current canonical domain is `.dev`.)

The most common codes:

- **NG0100 — ExpressionChangedAfterItHasBeenCheckedError.** Something changed a bound value during change detection. Look for a value being written in `ngAfterViewInit` or in an expression that computes different results on consecutive calls.
- **NG0200 — Circular dependency in DI.** Two services depend on each other. Redesign.
- **NG0201 — No provider found.** You injected something the current injector doesn't know about. Add `providedIn: 'root'` on the service, or `providers: [...]` in `app.config.ts`.

- **NG0950 — Required input is missing.** A component declared `input.required<T>()` but the parent didn't supply a value. Usually a `[value]` binding you forgot on `<app-thing>`.
- **NG05104 — Root element was not found.** The element `bootstrapApplication` was looking for (matching the root component's selector) isn't in `index.html`. Check that `<app-root>` (or whatever your selector is) exists in `src/index.html`.

The "detailed message with a link" is generated at runtime and is often the most useful part. It tells you which component, which template line, which expression.

Reading TypeScript errors

TypeScript's compiler errors are dense but precise. Read the whole message; the last line is usually the punchline.

```
error TS2322: Type 'string | null' is not assignable to type 'string'.  
  Type 'null' is not assignable to type 'string'.
```

The pattern: "type X is not assignable to type Y." Y is what a function or field expects. X is what you handed it. The fix is either to narrow X (`if (x !== null) { ... }`), assert it (`x!` — sparingly), or change Y to accept the wider type.

Two extra tricks:

- **Hover over a red squiggle in VS Code.** The full error message appears, sometimes with quick-fix suggestions.
- **Cmd/Ctrl-click a type name** to jump to its definition. This works on `Task`, `HttpClient`, everything.

`console.log` productively

You will `console.log` a lot. Two techniques:

Label your logs. `console.log('tasks after add:', tasks)` is much easier to find in the console than `console.log(tasks)` when there are twenty of them.

Log signals with `()`. `console.log('tasks:', this.tasks())` prints the current value. `console.log('tasks:', this.tasks)` prints the signal function itself, which is rarely what you want.

Remove them before committing. Or wrap them: `if (!environment.production) console.log(...)`.

The Sources debugger

`console.log` is fast; the debugger is stronger. In VS Code, place a red dot next to a line by clicking in the gutter, then run `ng serve` and interact with the page — execution stops at the breakpoint, and you can inspect every variable in scope, step over, step into, and continue.

Debug with the browser's DevTools instead if you prefer: open Sources, navigate to your `.ts` file (source maps make them visible), click the line number to set a breakpoint. Same experience, browser-native.

For particularly tricky bugs, `debugger;` in your code acts as a hardcoded breakpoint. Remove it before committing (or your linter will).

Common Angular gotchas

A grab bag of things that trip people up.

"Signal shows as a function, not its value."

You forgot the parentheses. `{{ tasks }}` prints the function object; `{{ tasks() }}` prints its current value.

"Template says nothing changed, but I clearly changed the data."

You mutated an array or object in place. Angular tracks references. Use spread to produce a new one: `this.tasks.update(t => [...t, newTask])`.

"My HTTP call fires twice."

You subscribed to a cold Observable twice. Every subscribe re-fires the request. Use `firstValueFrom`, an `async` pipe, or `shareReplay(1)`.

"My subscription runs forever."

You didn't tear it down. Use `takeUntilDestroyed()` or the `async` pipe.

"The template compiler says a component doesn't exist."

You didn't add it to `imports` in the parent component's `@Component`. Standalone components explicitly declare their dependencies.

"`ng serve` doesn't refresh."

Save the file (Cmd/Ctrl+S). Some editors have unsaved-file indicators; if you see a dot next to the filename, save.

"Two-way binding doesn't update."

`[(ngModel)]` requires `FormsModule` to be imported into the component. Or use a signal-based `model()`.

"Route doesn't match."

Trailing slashes, missing leading slash, or the wildcard route above your target. `path: 'foo/bar/'` won't match `/foo/bar`.

"CORS error in the console."

Not an Angular problem. The backend needs to send permissive `Access-Control-Allow-Origin` headers. Fix on the server, not the client.

"Route params are `undefined`."

You are reading them synchronously in the constructor, but the router hasn't populated them yet. Either use `withComponentInputBinding() + input()`, or subscribe to `route.paramMap`.

"Everything is fine locally but broken in production."

Something is different. Common culprits: environment variable substitution not configured (Chapter 19), a package installed as `devDependencies` that is actually needed at runtime, aggressive minification breaking a library that relies on function names.

Zone.js weirdness

If you are still on Zone.js (which the current default `ng new` is), sometimes async code updates a signal but the DOM doesn't refresh. The signal API is generally safe; but some older subscription patterns emit outside Angular's zone.

Fix: wrap the offending code in `NgZone.run()`, or migrate to `signals + provideExperimentalZonelessChangeDetection`. Zoneless removes an entire class of bug.

When to ask for help, and how

You will get stuck. When you do:

1. **Search the exact error message.** Someone has hit it. Their StackOverflow answer is your fastest path.
2. **Check the official Angular docs.** `angular.dev` has an excellent search.
3. **Ask on the Angular Discord** or on StackOverflow. Include a minimal reproduction: the smallest code that shows

the problem. Half the time you find the fix while distilling the repro.

Do not:

- Post twenty files of code hoping someone will read all of them.
- Say "it doesn't work." Say what you expected, what you saw, and what you already tried.
- Copy an error message without context. Include the surrounding code.

The rule of the room: come with the three questions from the top of this appendix answered, and someone will help you fast.

Reading stack traces

A stack trace goes bottom-up: the last line is the outermost caller, the first line is where the error was thrown. Read it in reverse for the "why," and forward for the "where."

Angular stack traces are noisier than plain JS traces because they include internal framework calls. Ignore anything under `zone.js`, `platform-browser`, or `core.mjs` — those are Angular's plumbing. Look for the first line that names *your* code.

Source maps make the file names in the trace refer to your `.ts` files, not the compiled JS. If you see filenames like `main-a1b2c3.js`, source maps aren't working — usually because you built without them or the map files aren't being served.

The "delete and see" technique

When something works and you don't know why, or breaks and you don't know why, delete pieces of code until behavior changes. Whatever you last deleted is the cause.

This works especially well for CSS: rules cascading unexpectedly, styles bleeding across components, weird layout. Comment out sections; watch what changes.

When you have exhausted every idea

Take a walk. Genuinely. The debugger's most powerful feature is your own attention, and attention degrades with fatigue. A ten-minute break saves an hour of guessing.

If the walk doesn't work: read the code you wrote most recently, aloud, from the top. Not skim — read. You will notice the typo or the transposed argument on the first pass, and you will feel foolish, and it will happen again next month. This is part of the job.

Appendix C. Legacy Angular idioms you'll still meet

This book taught modern Angular: standalone components, signals, the new control flow, `inject()`, and `input() / output()` functions. Angular codebases in the wild are older than this book, and many use idioms that were the default when they were written. This appendix maps the old shapes to the new ones so you can read older code without confusion.

Everything in this appendix still works. Angular's compatibility record is excellent. New code should use the modern forms; old code you inherit can be modernized incrementally or left alone.

NgModules

Old:

```
@NgModule({
  declarations: [TaskListComponent, TaskRowComponent, TimeAgoPipe],
  imports: [CommonModule, ReactiveFormsModule],
  exports: [TaskListComponent],
})
export class TasksModule {}
```

Modern: standalone components each declare their own imports. There is no module. Where you previously registered a group of com-

ponents together, they now export themselves and consumers import them individually.

If you are inside a codebase built around NgModules, look for:

- `@NgModule` decorators.
- Components without `standalone: true` (or `standalone: false` explicitly).
- `SharedModule` files that re-export common declarations.

Migration path: `ng generate @angular/core:standalone` is a schematic that converts an NgModule-based app to standalone components in one pass. It works well; take a snapshot beforehand.

Constructor DI vs `inject()`

Old:

```
export class TaskList {  
  constructor(private store: TaskStore, private router: Router) {  
  }  
}
```

Modern:

```
export class TaskList {  
  private store = inject(TaskStore);  
  private router = inject(Router);  
}
```

Both work. `inject()` composes better with signals, factory functions, and outside-of-constructor situations. Constructor DI is fine and is still the default in many code style guides.

If you are converting: don't do them all at once. Convert as you touch a file.

@Input() and **@Output()** decorators

Old:

```
export class TaskRow {  
  @Input() task!: Task;  
  @Output() toggled = new EventEmitter<string>();  
  
  onClick() { this.toggled.emit(this.task.id); }  
}
```

Modern:

```
export class TaskRow {  
  task = input.required<Task>();  
  toggled = output<string>();  
  
  onClick() { this.toggled.emit(this.task().id); }  
}
```

The decorator forms still work. Signal inputs (`input()`) give you a reactive value; decorator inputs give you a plain property. Signal outputs (`output()`) and `EventEmitter` are wire-compatible — a parent listens with (`toggled`) either way.

***ngIf**, ***ngFor**, ***ngSwitchCase**

Old:

```
<div *ngIf="user; else guest">
  Hello, {{ user.name }}.
</div>
<ng-template #guest>
  <a routerLink="/login">Sign in</a>
</ng-template>

<ul>
  <li *ngFor="let task of tasks; trackBy: trackTask">
    {{ task.title }}
  </li>
</ul>

<div [ngSwitch]="task.status">
  <span *ngSwitchCase="'todo'">To do</span>
  <span *ngSwitchCase="'done'">Done</span>
  <span *ngSwitchDefault>?</span>
</div>
```

Modern:

```
@if (user) {  
  <div>Hello, {{ user.name }}.</div>  
} @else {  
  <a routerLink="/login">Sign in</a>  
}  
  
<ul>  
  @for (task of tasks; track task.id) {  
    <li>{{ task.title }}</li>  
  }  
</ul>  
  
@switch (task.status) {  
  @case ('todo') { <span>To do</span> }  
  @case ('done') { <span>Done</span> }  
  @default { <span>?</span> }  
}
```

The new control flow is cleaner, faster, and doesn't need `CommonModule` in imports. There is a migration schematic: `ng generate @angular/core:control-flow`.

The old forms require `CommonModule` (or the specific directives) in the component's imports; the new forms don't require anything.

Observables where signals now fit

Old:

```

export class TaskList implements OnInit, OnDestroy {
  private destroy$ = new Subject<void>();
  tasks: Task[] = [];

  constructor(private store: TaskStore) {}

  ngOnInit() {
    this.store.tasks$
      .pipe(takeUntil(this.destroy$))
      .subscribe(t => this.tasks = t);
  }

  ngOnDestroy() {
    this.destroy$.next();
    this.destroy$.complete();
  }
}

```

Modern:

```

export class TaskList {
  private store = inject(TaskStore);
  tasks = this.store.tasks; // just a signal read
}

```

Or, if the store still exposes Observables:

```

tasks = toSignal(this.store.tasks$, { initialValue: [] });

```

The `destroy$ + takeUntil(destroy$)` pattern is the classic Angular observable-teardown idiom. Modern replacement: `takeUntilDestroyed()`, which reads the current `DestroyRef` from DI and completes automatically.

HttpClient returning Observables

Still the same. Angular's `HttpClient` methods still return Observables. What changed: modern code often unwraps them to Promises via `firstValueFrom`, or feeds them through `toSignal`, or uses the newer `httpResource`. You will still see `.subscribe(=> ...)` in older code; it still works, but pair it with `takeUntilDestroyed`.

Providers and `forRoot()`

Old:

```
imports: [RouterModule.forRoot(routes), HttpClientModule]
```

Modern:

```
providers: [provideRouter(routes), provideHttpClient()]
```

Every Angular feature that used to have a `forRoot()` / `forChild()` module now has a `provide*()` function. The old modules still exist for backward compatibility but new code uses the new forms.

Template reference variables and `ViewChild`

Old:

```
@ViewChild('searchInput') searchInput!: ElementRef<HTMLInputElement>

ngAfterViewInit() {
  this.searchInput.nativeElement.focus();
}
```

Modern:

```
searchInput = viewChild<ElementRef<HTMLInputElement>>('searchInput')

constructor() {
  afterNextRender(() => this.searchInput()?.nativeElement.focus())
}
```

The signal-based `viewChild()` gives you a reactive reference. `afterNextRender` replaces `ngAfterViewInit` for the common "run after DOM has rendered" case.

`ChangeDetectorRef.detectChanges()` / `markForCheck()`

Old: components on `ChangeDetectionStrategy.OnPush` sometimes call `this.cdr.markForCheck()` after mutating state to tell Angular to re-check them.

Modern: signals notify Angular automatically. If you have moved to signals, you should not need `markForCheck()`. If you find yourself reaching for `ChangeDetectorRef`, you are probably managing state outside the reactive system.

Renderer2

Still current. `Renderer2` is Angular's platform-agnostic way to manipulate the DOM (safe for SSR). Use it when you would otherwise reach for `nativeElement.style.foo = ...`; it works on the server, too.

`ngClass` and `ngStyle`

Old:

```
<li [ngClass]="{ done: task.done, overdue: isOverdue(task) }">  
  <span [ngStyle]="{ color: task.done ? '#999' : 'inherit' }">
```

Modern:

```
<li [class]="{ done: task.done, overdue: isOverdue(task) }">  
  <span [style]="{ color: task.done ? '#999' : 'inherit' }">
```

Or the compact shortcuts we covered in Chapter 6:

```
<li [class.done]="task.done" [class.overdue]="isOverdue(task)">  
  <span [style.color]="task.done ? '#999' : 'inherit'">
```

`[ngClass]` and `[ngStyle]` still work; they require `CommonModule`. The plain `[class]` and `[style]` bindings do not.

Router: `RouterModule` VS `provideRouter`

Old:

```
@NgModule({  
  imports: [RouterModule.forRoot(routes)],  
  exports: [RouterModule],  
})
```

Modern:

```
providers: [provideRouter(routes)]
```

In templates, `RouterOutlet`, `RouterLink`, and `RouterLinkActive` are still directives you import per component in standalone mode.

Forms: `FormsModule` VS `ReactiveFormsModule`

Both still exist. Older codebases sometimes import both to allow either style; newer codebases pick one. This book uses `ReactiveFormsModule` exclusively.

Lifecycle hooks

The classic hooks — `ngOnInit`, `ngOnDestroy`, `ngOnChanges`, `ngAfterViewInit`, `ngAfterContentInit`, `ngDoCheck` — all still work. Modern Angular's guidance is:

- Prefer `effect` over `ngOnInit` for "react to inputs."
- Prefer `DestroyRef.onDestroy` (via `inject`) or `takeUntilDestroyed()` over `ngOnDestroy`.

- Prefer `afterNextRender` / `afterRender` over `ngAfterViewInit`.

The old hooks are not deprecated; they are just less needed. When you inherit code that uses them, don't rewrite for the sake of it.

`ngOnChanges` and `SimpleChanges`

Old:

```
export class TaskRow implements OnChanges {
  @Input() task!: Task;

  ngOnChanges(changes: SimpleChanges) {
    if (changes['task']) {
      this.recompute();
    }
  }
}
```

Modern:

```
export class TaskRow {
  task = input.required<Task>();

  computed = computed(() => this.recompute(this.task()));
}
```

`ngOnChanges` used to be how you reacted to input changes. Signal inputs make `computed` and `effect` the more natural replacements.

EventEmitter

Still current; `output()` returns an `OutputEmitterRef` which behaves like an `EventEmitter` for the parent's listener. Older code writing `new EventEmitter<T>()` still works. New code prefers `output<T>()`.

Interceptors: class-based vs function-based

Old:

```
@Injectable()
export class AuthInterceptor implements HttpInterceptor {
  intercept(req: HttpRequest<any>, next: HttpHandler): Observable<any> {
    // ...
  }
}

// registered with HTTP_INTERCEPTORS:
{ provide: HTTP_INTERCEPTORS, useClass: AuthInterceptor, multi: true }
```

Modern:

```
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  // ...
  return next(req);
};

// registered with withInterceptors:
provideHttpClient(withInterceptors([authInterceptor]))
```

Both work.

Rule of thumb for reading legacy code

Old code was written when it was current. It solves the same problems modern Angular solves, in the shapes that were available then. Angular's team has consistently made *new* shapes without breaking *old* ones. That means:

- Read the old shape. It is not wrong, just older.
- Understand what it does.
- Convert only if you have a reason: you are already editing the file, or you are pulling it into a modernized area of the codebase.
- Never rewrite a whole file just because it uses `@Input()` instead of `input()`. That change costs a code review round without adding value.

The Angular team publishes migration schematics for each major shift (`ng update` applies them). If a large migration is worth doing, they usually made a tool for it.